

بازیابی اطلاعات

دانشگاه آزاد شیراز • دانشکده مهندسی کامپیوتر

ترم دوم • سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل اول: Boolean Retrieval

استاد درس

دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی)

امیرمحمد اسدجو

علی نیکوان

علی مجاهد

امیرحسین همتی

حمید نامجو

فهرست مطالب

0 Introduction to IR

1.1 An example information retrieval problem

1.2 A first take at building an inverted index

1.3 Processing Boolean queries

1.4 The extended Boolean model versus ranked retrieval

1.5 Chapter 1 Summary

1.6 End of Chapter Exercises

۱. مقدمه‌ای بر بازیابی اطلاعات: سفر به قلب وب امروزی

تصور کنید یک مغز دیجیتال فوق‌العاده

شما هر روز چند بار از خود می‌پرسید: «اون ایمیل مهم کجا رفت؟»، «مقاله‌ای که دیروز خوندم رو کجا ذخیره کردم؟» یا «چطور می‌تونم یه فیلم خوب با بازیگرای مورد علاقه‌م پیدا کنم؟». در پشت هر یک از این سوالات ساده، یک گول نامرئی به نام بازیابی اطلاعات (Information Retrieval) کار می‌کند. این علم، هنر یافتن سوزن اطلاعات در انبار کاه دیجیتال ماست.

دهه‌ها پیش، این یک مهارت تخصصی برای کتابداران و وکلای پژوهشگر بود. امروز، اما صدها میلیون نفر هر روز از آن استفاده می‌کنند—هر بار که در گوگل عبارتی را تایپ می‌کنند یا در صندوق ورودی ایمیل خود کلمه‌ای را جستجو می‌نمایند. بازیابی اطلاعات، دیگر یک فعالیت حاشیه‌ای نیست؛ بلکه به dominant form of information access تبدیل شده است.

چالش اصلی: داده‌های بی‌نظم و نامنظم

قلب این علم، کار با داده‌های **ناساختاریافته** (Unstructured Data) است. داده‌های ساختاریافته، مانند یک جدول اکسل منظم یا یک پایگاه داده فروشگاه، ستون‌ها و ردیف‌های مشخصی دارند. اما بیشتر اطلاعات دنیای ما—مانند ایمیل‌ها، مقالات، اخبار و صفحات وب—مثل یک اتاق پر از وسایل هستند که هرچیز جایی ریخته و هیچ برچسبی ندارد. علم بازیابی اطلاعات به کامپیوتر یاد می‌دهد که چطور در این اتاق شلوغ، مورد مورد نظر ما را پیدا کند.

البته، تقریباً هیچ داده‌ای کاملاً بی‌ساختار نیست. متون هم دارای ساختارهای پنهان زبانی هستند. سیستم‌های بازیابی مدرن از این ساختارها مانند عناوین، پاراگراف‌ها و حتی لینک‌ها (hyperlinks) بهره می‌برند تا جستجو را **نیمه‌ساختاریافته** (Semi-structured) و هوشمندانه‌تر کنند.

فراتر از جستجو: سازماندهی و طبقه‌بندی

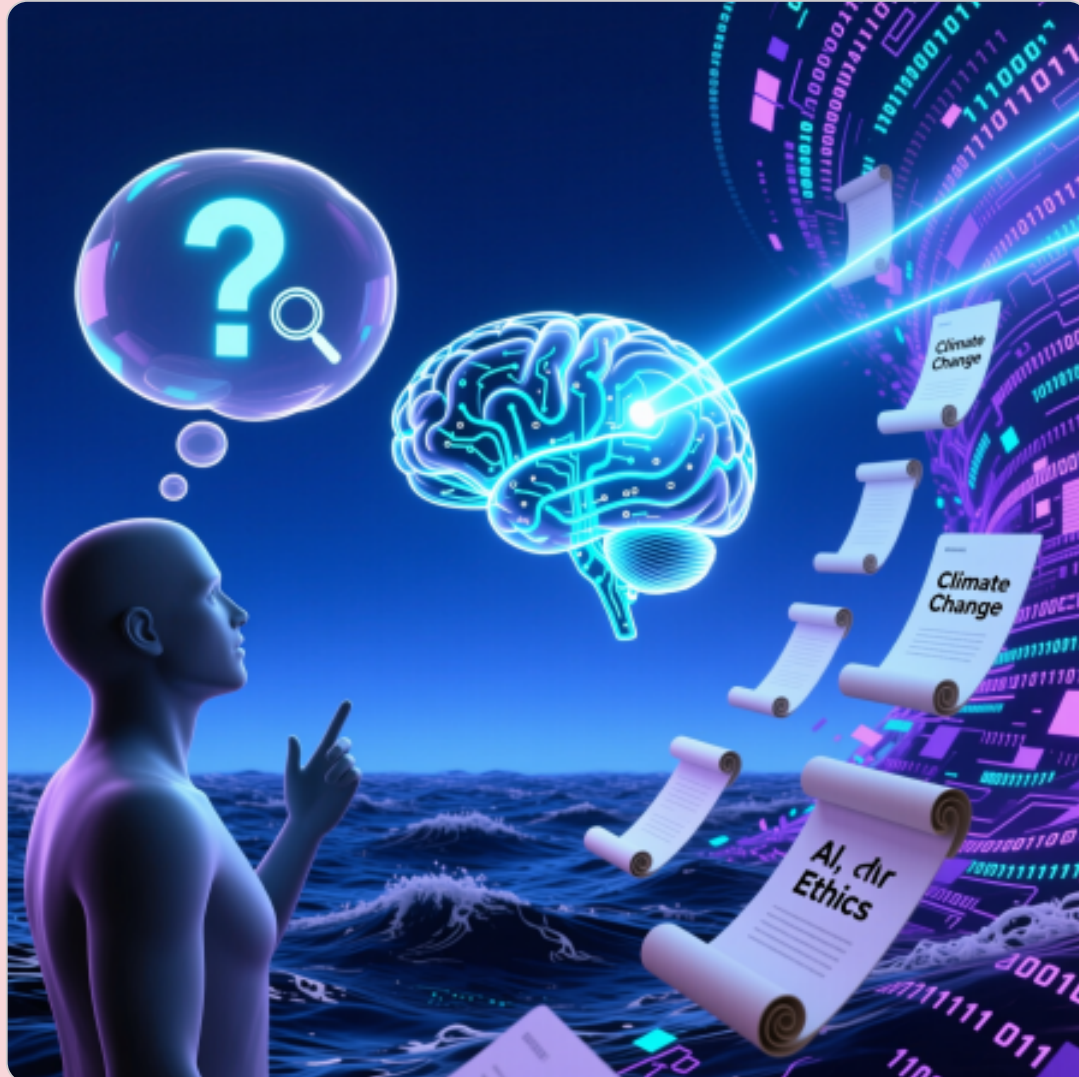
- بازیابی اطلاعات فقط پیدا کردن چند سند مرتبط نیست. این شامل وظایف هوشمندانه‌تری است:
- **خوشه‌بندی (Clustering):** تصور کنید صدها سند علمی در اختیار دارید. سیستم می‌تواند آن‌ها را به صورت خودکار به گروه‌های موضوعی تقسیم کند، درست مانند زمانی که کتاب‌های یک قفسه را بر اساس موضوع مرتب می‌کنید.
 - **طبقه‌بندی (Classification):** سیستم ایمیل شما هر روز این کار را برای شما انجام می‌دهد. به صورت خودکار تشخیص می‌دهد که کدام ایمیل اسپم است و کدامیک مهم و آن را در پوشه مناسب قرار می‌دهد.

با توجه به مقیاس عملکرد، سیستم‌های بازیابی اطلاعات به سه دسته اصلی تقسیم می‌شوند:

1. **جستجوی وب (Web Scale):** بزرگترین و پیچیده‌ترین حوزه. این سیستم‌ها باید بر روی میلیاردها سند در میلیون‌ها کامپیوتر جستجو کنند. چالش‌ها عظیم هستند: از جمع‌آوری اسناد با خزنده‌های وب (Crawlers) گرفته تا مقابله با ترندهای سایتی برای دستکاری رتبه‌ها.
2. **جستجوی شخصی (Personal Search):** همان جستجویی که در کامپیوتر خود انجام می‌دهید (مانند Spotlight در مک یا Instant Search در ویندوز). اینجا تمرکز بر سرعت، سبکی و کارایی با منابع محدود است.
3. **جستجوی سازمانی/حوزه‌محور (Enterprise/Domain-Specific):** جستجو در مجموعه‌های متمرکز مانند اسناد داخلی یک شرکت، پایگاه‌های ثبت اختراع یا مقالات تخصصی شیمی. این سیستم‌ها معمولاً روی چند سرور قدرتمند اجرا می‌شوند.

یک تصویر به ارزش هزاران کلمه

برای درک عمیق‌تر این مفهوم، تصور کنید کاربری که در اقیانوس بی‌پایان داده‌ها (سمت راست) یک نیاز اطلاعاتی دارد (سمت چپ). یک سیستم بازیابی اطلاعات هوشمند (مغز دیجیتال) باید بتواند از میان این حجم عظیم داده‌های نامنظم، دقیقاً همان اسناد مرتبط را استخراج و به کاربر ارائه دهد. این تصویر، قلب تلاش ما در این فصل است: چطور این مغز دیجیتال را بسازیم.



در این کتاب، ما به بررسی ایده‌ها و تکنیک‌هایی می‌پردازیم که این مغز دیجیتال را می‌سازند؛ از ساختارهای داده‌ای اولیه گرفته تا الگوریتم‌های پیچیده‌ای که دنیای وب را می‌چرخانند.

۱.۱ یک مسئله نمونه از بازیابی اطلاعات: جستجو در اقیانوس دیجیتال 🧐

🗨️ تصور کنید مدیر یک سرویس استریم فیلمنامه

شما مدیر آرشیو دیجیتال غول‌پیکری هستید که شامل تمام فیلمنامه‌های نوشته‌شده در تاریخ است. یک روز، یک کاربر پژوهشگر با یک درخواست بسیار خاص به شما مراجعه می‌کند: «می‌خواهم تمام فیلمنامه‌هایی را پیدا کنم که در آن‌ها **بروتوس و سزار** هر دو حضور دارند، اما **کالپورنیا** در هیچ‌کدام حضور ندارد.» چطور این جستجو را به صورت کارآمد انجام می‌دهید؟

یک راه اولیه و شاید بلافاصله به ذهن می‌رسد، استخدام یک کتابدار بسیار پرتلاش و خواندن تک‌تک تمام فیلمنامه‌ها! این روش در دنیای کامپیوتر به **پیمایش خطی** یا **گرپ کردن (grepping)** معروف است. برای مجموعه‌ای به کوچکی مانند آثار کامل شکسپیر، این روش حتی با سرعت کامپیوترهای امروزی هم قابل قبول است. اما برای میلیارد‌ها سند یا برای پرسمان‌های پیچیده‌تر، این روش مانند تلاش برای پیدا کردن سوزن در انبار کاه است!

📅 **ایده اول: ماتریس وقوع بولی (یک برگه بازرسی غول‌پیکر)**

اولین راهکار هوشمندانه، ساخت یک **ماتریس وقوع بولی** است. تصور کنید یک برگه غول‌پیکر که در آن هر ردیف به یک واژه (مثل «بروتوس») و هر ستون به یک سند (مثل «هملت») اختصاص دارد. در خانه‌ای که واژه در سند حضور دارد، عدد ۱ و در غیر این صورت ۰ می‌نویسیم.

برای پاسخ به پرسمان Brutus AND Caesar AND NOT Calpurnia، کافی است بردارهای بولی مربوط به این سه واژه را برداشته و یک عملیات بیتی روی آن‌ها انجام دهیم:

$$[1, 1, 0, 1, 0, 0] \wedge [1, 1, 0, 1, 1, 1] \wedge [1, 0, 1, 1, 1, 1] = [1, 0, 0, 1, 0, 0]$$

نتیجه این محاسبه به ما می‌گوید که سندهای شماره ۱ (هملت) و ۳ (مکبث) پاسخ‌های درست هستند. این روش، در تئوری بی‌نقص است، اما در عمل با یک مشکل اساسی روبرو است: **اندازه!**

❌ فاجعه مقیاس: ماتریس بیش از حد بزرگ!

فرض کنید مجموعه ما فقط یک میلیون سند با ۵۰۰ هزار واژه منحصر به فرد دارد. ماتریس ما ابعاد $500,000 \times 1,000,000$ خواهد داشت! این ماتریس حدود ۵۰۰ میلیارد درایه دارد. حتی اگر هر درایه فقط یک بایت باشد، به بیش از ۴۶۰ گیگابایت فضا نیاز دارد. مشکل بزرگ‌تر این است که این ماتریس فوق‌العاده **تنک (sparse)** است. چون هر سند فقط حدود ۱۰۰۰ واژه دارد، از ۵۰۰ میلیارد درایه، کمتر از ۱ میلیارد آن‌ها ۱ هستند و بقیه (یعنی ۹۹.۸٪) همه صفر هستند! ما در حال ذخیره یک اقیانوس از صفرها هستیم!

💡 راه حل نهایی: ایندکس معکوس (نگاهی از زاویه‌ای دیگر)

به جای پرسیدن «کدام اسناد شامل واژه X هستند؟»، مسئله را برعکس می‌کنیم و می‌پرسیم: «واژه X در کدام اسناد ظاهر شده است؟» این تغییر دیدگاه ساده، به **ایندکس معکوس (Inverted Index)** منجر می‌شود.

این ساختار هوشمندانه از دو بخش اصلی تشکیل شده است:

1. **دیکشنری (Dictionary):** یک فهرست مرتب‌شده از تمام واژه‌هایی که در مجموعه وجود دارند. این مانند

فهرست مطالب کتاب است.

2. **لیست ارسال (Postings List):** برای هر واژه در دیکشنری، لیستی از تمام اسنادی که آن واژه در آن‌ها یافت

شده است. این لیست‌ها معمولاً بر اساس شماره سند (docID) مرتب هستند.

برای مثال ما، ایندکس معکوس به این شکل خواهد بود:

Brutus → [1, 2, 4] | Caesar → [1, 2, 4, 5, 6] | Calpurnia → [2]

حالا برای پاسخ به پرسمان، دیگر نیازی به محاسبات پیچیده و ماتریس عظیم نداریم. فقط کافی است لیست ارسال «بروتوس» و «سزار» را گرفته و اشتراکشان را محاسبه کنیم، و سپس از نتیجه، سند شماره ۲ (که در لیست ارسال «کالپورنیا» است) را حذف کنیم. این فرآیند فوق‌العاده سریع و بهینه است.

🖼️ تصویر نهایی: لحظه «آها!»

تصویر زیر (شکل ۱.۲ در کتاب) دقیقاً همان نتیجه‌ای است که از روش ایندکس معکوس به دست می‌آید. این تصویر، لحظه‌ای است که تلاش فکری ما به نتیجه ملموس تبدیل می‌شود. این تصویر نشان می‌دهد که اسناد ۱ و ۳ (هملت و مکبث) پاسخ صحیح پرسمان ما هستند، دقیقاً همان چیزی که با محاسبات بیتی به دست آوردیم. این تصویر، پلی است بین تئوری ماتریس و کاربرد عملی ایندکس معکوس.


```
continue
elif token not in ['AND', 'OR']:
    positive_terms.append(token)
i += 1

# اگر واژه‌ای وجود نداشته، هیچ سندی برگردانده نشود
if not positive_terms:
    print("نتیجه پرسیمان: []")
    return []

# شروع با اسناد حاوی اولین واژه مثبت
first_term = positive_terms[0]
if first_term not in matrix:
    result_docs = set()
else:
    result_docs = set(doc for doc, exists in matrix[first_term].items() if exists)

# برای سایر واژه‌های مثبت AND اعمال
for term in positive_terms[1:]:
    if term in matrix:
        term_docs = set(doc for doc, exists in matrix[term].items() if exists)
        result_docs = result_docs.intersection(term_docs)
    else:
        result_docs = set() # نتیجه خالی → واژه‌ای نیست
        break

# اعمال NOT
for term in negative_terms:
    if term in matrix:
        term_docs = set(doc for doc, exists in matrix[term].items() if exists)
        result_docs = result_docs.difference(term_docs)

print(f"نتیجه پرسیمان: {sorted(result_docs)}")
return sorted(result_docs)

def build_inverted_index(matrix):
    """
    .این تابع یک ایندکس معکوس از روی ماتریس وقوع می‌سازد
    .این تابع، پل ارتباطی بین ماتریس و ایندکس معکوس را نشان می‌دهد
    """
    print("\nمرحله ۳: ساخت ایندکس معکوس از ماتریس\n")
    inverted_index = {}

    for term, doc_dict in matrix.items():
        # استخراج اسنادی که واژه در آنها وجود دارد
        docs_with_term = [doc for doc, exists in doc_dict.items() if exists]
        if docs_with_term:
            inverted_index[term] = sorted(docs_with_term)

    print(".ایندکس معکوس ساخته شد.")
    # print(inverted_index)
    return inverted_index

# --- اجرای مثال ---

if __name__ == "__main__":
    # مرحله ۱: ساخت ماتریس وقوع بولی
    term_doc_matrix = build_term_document_matrix(documents)

    # نمایش ماتریس برای چند واژه نمونه
    sample_terms = ['Brutus', 'Caesar', 'Calpurnia']
    print(f"\nنمونه‌ای از ماتریس وقوع برای واژه‌های {sample_terms}:")
    for term in sample_terms:
        if term in term_doc_matrix:
            print(f"{term}: {term_doc_matrix[term]}")

    # مرحله ۲: اجرای پرسیمان بولی
    query = ['Brutus', 'AND', 'Caesar', 'AND', 'NOT', 'Calpurnia']
    result = boolean_query(term_doc_matrix, query)

    # مرحله ۳: ساخت ایندکس معکوس
    inverted_index = build_inverted_index(term_doc_matrix)

    # نمایش ساختار ایندکس معکوس
    sample_terms = ['Brutus', 'Caesar', 'Calpurnia']
    print(f"\nنمونه‌ای از ماتریس وقوع برای واژه‌های {sample_terms}:")
    for term in sample_terms:
        lower_term = term.lower()
        if lower_term in inverted_index:
            print(f"{term}: {inverted_index[lower_term]}")
        else:
            print(f"{term}: (وجود ندارد در ماتریس)")

    print("\nاین فرایند، چگونگی تبدیل یک مسئله جستجوی خطی به یک راحل بهینه (ایندکس معکوس) را نشان می‌دهد\n")
```

{ 'Antony': 'Agrippa [Aside to Domitius Enobarbus]: Why, Enobarbus, When Antony found Julius Caesar dead, He cried almost to roaring; and he wept When at Philippi he found Brutus slain.', 'Hamlet': "Lord Polonius: I did enact Julius Caesar: I was killed i' the Capitol; Brutus killed me.", 'Othello': 'Had I but one hour tonight-- Othello, I am not well.', 'Macbeth': "If it were done when 'twere done, then 'twere well it were done quickly."}

...مرحله ۱: ساخت ماتریس وقوع بولی
ماتریس وقوع بولی ساخته شد

['Brutus', 'Caesar', 'Calpurnia']: نمونه‌ای از ماتریس وقوع برای واژه‌های

Brutus AND Caesar AND NOT Calpurnia: مرحله ۲: اجرای پرسمان بولی
['Antony', 'Hamlet']: نتیجه پرسمان

...مرحله ۳: ساخت ایندکس معکوس از ماتریس
ایندکس معکوس ساخته شد

['Brutus', 'Caesar', 'Calpurnia']: نمونه‌ای از ماتریس وقوع برای واژه‌های
Brutus: {'Antony': 1, 'Hamlet': 1, 'Othello': 0, 'Macbeth': 0}
Caesar: {'Antony': 1, 'Hamlet': 1, 'Othello': 0, 'Macbeth': 0}
Calpurnia: (وجود ندارد در ماتریس)

این فرآیند، چگونگی تبدیل یک مسئله جستجوی خطی به یک رامحل بهینه (ایندکس معکوس) را نشان می‌دهد

🔍 1.2 ساخت اولیه فهرست معکوس: پشت صحنه جادوی جستجو!

🚀 یک چالش دنیای واقعی

تصور کنید مدیر داده یک شبکه اجتماعی بزرگ مانند «Instagram» هستید. میلیون‌ها پست روزانه منتشر می‌شود. یک کاربر می‌خواهد تمام پست‌هایی را ببیند که همزمان دارای هشتگ‌های **#سفر** و **#غروب_آفتاب** هستند. چگونه می‌توانید این کار را در کسری از ثانیه انجام دهید؟ پیمایش تمام میلیون‌ها پست برای هر جستجو غیرممکن است! اینجااست که جادوی **فهرست معکوس** وارد می‌شود.

برای بهره‌گیری از سرعت فوق‌العاده در زمان جستجو، باید این فهرست را **از قبل** بسازیم. این فرآیند مانند تهیه یک دستور آشپزی دقیق است و چهار مرحله اصلی دارد:

1. ۱. جمع‌آوری مواد اولیه (اسناد)

همه پست‌های IG را جمع می‌کنیم. این «مجموعه اسناد» ماست.

2. ۲. خرد کردن (توکن‌سازی)

هر پست را به واحدهای کوچک‌تری به نام «توکن» (Token) می‌شکنیم. در دنیای ما، این یعنی جدا کردن تمام کلمات و هشتگ‌ها. 🌿

3. ۳. تمیزکاری و آماده‌سازی (پیش‌پردازش زبانی)

توکن‌ها را نرمال‌سازی می‌کنیم: همه را به حروف کوچک تبدیل می‌کنیم، علائم نگارشی مانند «!» و «#» را حذف می‌کنیم و ریشه کلمات را پیدا می‌کنیم. 🗨️

4. ۴. ساخت دفترچه تلفن هوشمند (ایندکس‌سازی)

حالا مهم‌ترین بخش کار: ساختن خود فهرست معکوس. 📖

📖 جادوی مرتب‌سازی: از آشفتگی تا نظم

چگونه این دفترچه تلفن را می‌سازیم؟ روش اولیه و هوشمندانه، **ایندکس‌سازی مبتنی بر مرتب‌سازی** است:

مرحله الف: آشفت‌گاه اولیه

ابتدا یک لیست بزرگ از تمام جفت‌های (توکن، شناسه_پست) برای کل مجموعه اسناد می‌سازیم. مثلاً:

(sunset, 1542)

(travel, 1123)

(سفر, 1123)

(good, 8801)

مرحله ب: نظم جادویی

حالا این لیست شلوغ را بر اساس توکن‌ها (الفبا) مرتب می‌کنیم. سپس موارد تکراری را گروه‌بندی می‌کنیم. نتیجه چیزی شبیه به این می‌شود:

```
good → 8801
sunset → 1542
travel → 1123
سفر → 1123
غروب_آفتاب → 1542
...
```

این ساختار مرتب، قلب تپنده فهرست معکوس ماست.

در نهایت، این دفترچه تلفن هوشمند به دو بخش اصلی تقسیم می‌شود:

📁 دو بخش اصلی فهرست معکوس

- دیکشنری (Dictionary):** این بخش مانند فهرست نام‌ها در دفترچه تلفن است. شامل تمام توکن‌های منحصر به فرد (مانند سفر، غروب_آفتاب) است. برای هر توکن، یک آمار کلیدی هم ذخیره می‌شود: **فراوانی سند** (Document Frequency یا df_t)؛ یعنی تعداد کل پست‌هایی که آن توکن در آن‌ها وجود دارد. این دیکشنری آنقدر کوچک است که می‌توانیم آن را کاملاً در حافظه اصلی (RAM) نگه داریم تا دسترسی به آن فوری باشد.
- لیست ارسال (Postings List):** این بخش مانند شماره تلفن‌های مربوط به هر نام در دفترچه تلفن است. برای هر توکن در دیکشنری، لیستی از شناسه‌های پست‌هایی (docID) که آن توکن در آن‌ها وجود دارد، ذخیره می‌شود. این لیست‌ها بسیار حجیم هستند و روی دیسک ذخیره می‌شوند، اما **مرتب‌شده بر اساس docID** هستند. این مرتب‌سازی، کلید سرعت پردازش پرسman‌های بولی است که در بخش قبل دیدیم.

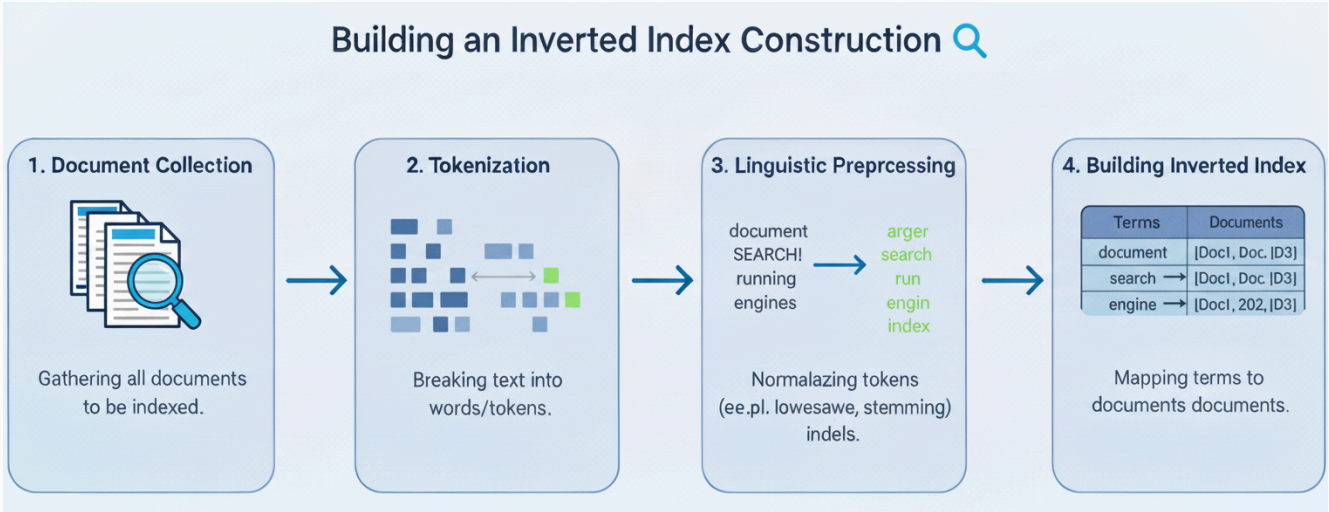
🤖 عمیق‌تر: بهترین ساختار برای لیست ارسال چیست؟

برای ذخیره لیست ارسال در حافظه، دو گزینه اصلی وجود دارد و انتخاب بین آن‌ها یک معامله بین انعطاف‌پذیری، سرعت و فضا است:

- **لیست پیوندی تک‌طرفه (Singly Linked List):** مثل قطاری از واگن‌ها که به هم متصل‌اند. **مزیت:** اضافه کردن یک پست جدید به لیست بسیار سریع و ارزان است. **عیب:** به حافظه پراکنده نیاز دارد و پیمایش آن ممکن است کندتر باشد. این ساختار، پایه‌ای برای تکنیک‌های پیشرفته‌تر مانند *Skip Lists* است.
- **آرایه با طول متغیر (Variable Length Array):** مثل یک قطار یکپارچه با تعداد واگن متغیر. **مزیت:** بسیار فشرده‌تر است و چون در حافظه پشت سر هم قرار گرفته، پیمایش آن در پردازنده‌های مدرن بسیار سریع‌تر است. **عیب:** اضافه کردن عضو جدید ممکن است نیازمند ساختن کل آرایه جدیدی باشد.

در دنیای واقعی، اگر به‌روزرسانی زیاد باشد، از لیست پیوندی استفاده می‌کنیم. اما اگر اسناد کمتر تغییر کنند، آرایه‌ها به دلیل فشرده‌گی و سرعت بالا، برنده هستند! 📦

این ساختار به ظاهر ساده، **بهره‌مندترین و کارآمدترین روش** برای پشتیبانی از جستجوی متنی موردی (ad-hoc) است. هر بار که در گوگل عبارتی را جستجو می‌کنید یا در یک پایگاه داده حقوقی به دنبال یک سند خاص می‌گردید، در حال استفاده از نسخه‌ای بسیار پیچیده‌تر از همین ایده ساده و زیبا هستید.



1.3 پردازش پرسمان‌های بولی

داستانی از کلاس درس 🎓

"روز اول کلاس بازیابی اطلاعات بود. دکتر اسکندری وارد شد و پرسید: 'چطور می‌توانم در این کتابخانه‌های دیجیتال، کتاب‌هایی را پیدا کنم که هم درباره‌ی هوش مصنوعی و هم درباره‌ی VLM‌ها باشند؟' سکوت حاکم شد. سپس دکتر لبخندی زد و گفت: 'این همان چیزی است که امروز می‌خواهیم یاد بگیریم - جادوی پرسمان‌های بولی!'"

در مدل بولی، هر پرسمان ترکیبی از واژه‌ها با عملگرهای AND، OR، و NOT است. این مدل ساده اما قدرتمند، پایه‌ی بسیاری از موتورهای جستجوی اولیه بود و هنوز هم در سیستم‌های اطلاعاتی مدرن کاربرد دارد.

فکر کن! 💡

تا حالا به این فکر کرده‌اید که چطور گوگل نتایج جستجو را فیلتر می‌کند؟ وقتی تایپ می‌کنید "رستوران ایتالیایی نزدیک من" AND "باز تا ساعت ۱۰ شب"، در حال استفاده از منطق بولی هستید! 🌙👁️

برای پاسخ به پرسمان ساده‌ای مانند:

Brutus $\wedge$ Calpurnia

مراحل زیر را دنبال می‌کنیم:

1. یافتن Brutus در دیکشنری 🔍
2. بازیابی لیست ارسال آن 📄
3. یافتن Calpurnia در دیکشنری 🔍
4. بازیابی لیست ارسال آن 📄
5. محاسبهٔ **اشتراک** دو لیست – یعنی یافتن اسنادی که هر دو واژه در آن‌ها ظاهر شده‌اند 🔗

تصور کن... 🎬

دو لیست شماره تلفن داریم: یکی از دوستان به نام "علی" و دیگری از دوستان که در "تهران" زندگی می‌کنند. می‌خواهی بدانی کدام علی‌ها در تهران زندگی می‌کنند. چه کار می‌کنی؟ دقیقاً! هر دو لیست را نگاه می‌کنی و شماره‌هایی که در هر دو

چون لیست‌های ارسال مرتب‌شده بر اساس **docID** هستند، می‌توانیم از الگوریتم ادغام دوطرفه (two-pointer merge) برای یافتن اشتراک در زمان خطی استفاده کنیم. این الگوریتم در شکل ۱.۶ کتاب آمده است:

```

INTERSECT( $p_1, p_2$ )
1   $answer \leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $\text{ADD}(answer, \text{docID}(p_1))$ 
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7  else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8      then  $p_1 \leftarrow \text{next}(p_1)$ 
9      else  $p_2 \leftarrow \text{next}(p_2)$ 
10 return  $answer$ 

```

► **Figure 1.6** Algorithm for the intersection of two postings lists p_1 and p_2 .

🚀 تحلیل پیچیدگی

پیچیدگی زمانی این الگوریتم برابر $O(x + y)$ است، که x و y طول لیست‌های ارسال هستند. این بسیار سریع‌تر از اسکن خطی تمام مجموعه است، هرچند از نظر نماد Θ هم‌مرتبه باشد – اما ثابت عملی آن بسیار کوچک‌تر است.

تفکر سریع: اگر لیست اول ۱۰۰۰ آیتم و لیست دوم ۵۰۰ آیتم داشته باشد، در بدترین حالت چند مقایسه نیاز داریم؟ 🤔

◀ پاسخ

برای پرسمان‌های چندجمله‌ای مانند:

$\text{Brutus} \wedge \text{Caesar} \wedge \text{Calpurnia}$

بهینه‌ترین استراتژی، پردازش واژه‌ها به ترتیب فراوانی صعودی است: ابتدا واژه‌ای که در کمترین تعداد سند آمده را بررسی می‌کنیم، چون نتیجه میانی از همان ابتدا کوچک خواهد بود و کار کمتری برای اشتراک بعدی لازم است. برای مثال، اگر:

• Calpurnia: 2 سند

• Brutus: 174 سند

• Caesar: 132 سند

🎯 استراتژی بهینه

آنگاه بهتر است پرسمان را به صورت زیر ارزیابی کنیم:

$(\text{Calpurnia} \wedge \text{Brutus}) \wedge \text{Caesar}$

🏠 مثال کاربردی

تصور کنید در یک فروشگاه آنلاین کار می‌کنید و مشتری به دنبال محصولی است که همزمان این ویژگی‌ها را داشته باشد: "قیمت کمتر از ۱۰۰ هزار تومان" AND "رنگ آبی" AND "برند X". اگر بدانید که فقط ۵ محصول رنگ آبی دارند، ۵۰ محصول برند

X هستند و ۲۰۰ محصول قیمت کمتر از ۱۰۰ هزار تومان دارند، از کدام فیلتر شروع می‌کنید؟ طبیعتاً از رنگ آبی چون نتایج اولیه را به ۵ محصول محدود می‌کند! 🛒

این تکنیک، **بهینه‌سازی پرسمان** (Query Optimization) نام دارد و نیازمند ذخیرهٔ **فراوانی سند** (df_t) در دیکشنری است.

برای پرسمان‌های پیچیده‌تر مانند:

$$(\text{Brutus} \vee \text{Caesar}) \wedge \neg(\text{Antony} \vee \text{Cleopatra})$$

ابتدا هر زیرعبارت را ارزیابی می‌کنیم:

- Brutus OR Caesar: ادغام دو لیست (UNION)
- Antony OR Cleopatra: ادغام دو لیست دیگر
- NOT (...): مکمل گرفتن نسبت به تمام اسناد مجموعه

⚠ چالش NOT

اما **پردازش NOT مشکل‌ساز است**، چون نیاز به دانستن مجموعهٔ کامل اسناد دارد – و اگر مجموعه بزرگ باشد، ساخت مکمل بسیار هزینه‌بر است. راه‌حل‌های هوشمندانه‌تری (مثل استفاده از ایندکس معکوس معکوس!) نیاز است، که در اینجا به‌صورت ساده پیاده‌سازی می‌شود.

تقویت درک: تصور کنید در یک مهمانی هستید و می‌خواهید کسانی را پیدا کنید که "نوشیدنی نمی‌خورند". آیا راحت‌تر است همه‌ی مهمان‌ها را بررسی کنید یا لیست کسانی را که "نوشیدنی می‌خورند" تهیه کرده و از کل لیست مهمان‌ها کم کنید؟ 🍹

الگوریتم کلی برای پرسمان‌های ترکیبی در شکل ۱.۷ کتاب آمده است – که واژه‌ها را بر اساس فراوانی مرتب کرده و به‌صورت تدریجی اشتراک می‌گیرد. این، پایهٔ اصلی موتورهای جست‌وجوی بولی است.

```
INTERSECT( $\langle t_1, \dots, t_n \rangle$ )
1   $terms \leftarrow \text{SORTBYINCREASINGFREQUENCY}(\langle t_1, \dots, t_n \rangle)$ 
2   $result \leftarrow \text{postings}(\text{first}(terms))$ 
3   $terms \leftarrow \text{rest}(terms)$ 
4  while  $terms \neq \text{NIL}$  and  $result \neq \text{NIL}$ 
5  do  $result \leftarrow \text{INTERSECT}(result, \text{postings}(\text{first}(terms)))$ 
6      $terms \leftarrow \text{rest}(terms)$ 
7  return  $result$ 
```

► **Figure 1.7** Algorithm for conjunctive queries that returns the set of documents containing each term in the input list of terms.

🌐 کاربردهای امروزی

اگرچه مدل بولی ساده به نظر می‌رسد، اما هنوز در بسیاری از سیستم‌های مدرن استفاده می‌شود. برای مثال:

- در پایگاه‌های داده حقوقی برای جستجوی دقیق قوانین و احکام
- در جستجوی اسناد علمی برای یافتن مقالاتی که همزمان چند مفهوم را پوشش می‌دهند
- در ویژگی‌های جستجوی پیشرفته گوگل (مانند جستجوی عبارت دقیق با استفاده از کوتیشن)
- در سیستم‌های توصیه‌گر برای فیلتر کردن محتوا بر اساس علایق کاربران

حالا که با مفاهیم آشنا شدی، بیا یک سناریوی واقعی را حل کنیم:

شما توسعه‌دهنده یک اپلیکیشن دوست‌یابی هستید. کاربر می‌خواهد فردی را پیدا کند که این ویژگی‌ها را داشته باشد: "علاقه‌مند به سفر" AND "علاقه‌مند به کتاب" AND NOT "سیگار می‌کشد". با توجه به آمار زیر، بهینه‌ترین استراتژی برای جستجو چیست؟

- علاقه‌مند به سفر: ۱۰,۰۰۰ کاربر
- علاقه‌مند به کتاب: ۵,۰۰۰ کاربر
- سیگار می‌کشد: ۲,۰۰۰ کاربر

راهنمایی

پاسخ

برای مطالعه بیشتر

اگر به این موضوع علاقه‌مند شدی، پیشنهاد می‌کنم مقاله "Beyond Boolean: The Evolution of Information Retrieval Models" را مطالعه کنی. این مقاله نشان می‌دهد که چگونه مدل‌های بولی به مدل‌های پیچیده‌تر مانند مدل فضای برداری و مدل‌های احتمالی تبدیل شده‌اند. 

برای مطالعه بیشتر

اگر به این موضوع علاقه‌مند شدی، پیشنهاد می‌کنم مقاله "Beyond Boolean: The Evolution of Information Retrieval Models" را مطالعه کنی. این مقاله نشان می‌دهد که چگونه مدل‌های بولی به مدل‌های پیچیده‌تر مانند مدل فضای برداری و مدل‌های احتمالی تبدیل شده‌اند. 

In [2]: # --- تعریف ساختار داده‌ها ---

```
# نمونه‌ای از یک ایندکس معکوس (ساده‌شده)
# مرتب‌شده هستند (postings lists) کلیدها واژه‌ها و مقادیر، لیست‌های ارسال
# این ساختار، قلب یک موتور جستجوی بولی است
inverted_index = {
    "Brutus": [1, 2, 4, 11, 31, 45, 173, 174],
    "Calpurnia": [2, 31, 54, 101],
    "Caesar": [1, 2, 4, 5, 6, 16, 57, 132],
    "Antony": [5, 10, 31],
    "Cleopatra": [2, 3, 31, 64]
}

# فرض می‌کنیم مجموعه اسناد ما شامل شناسه‌های ۱ تا ۲۰۰ است
# ضروری است تا بتوانیم مکمل یک مجموعه را محاسبه کنیم NOT این لیست برای عملگر
ALL_DOCUMENTS = list(range(1, 201))
```

--- پیاده‌سازی الگوریتم‌های اصلی ---

```
def intersect(p1, p2):
    """
    الگوریتم ادغام دوطرفه برای یافتن اشتراک دو لیست مرتب‌شده.
    این پیاده‌سازی دقیقاً مطابق با شیءکد کتاب و توضیحات مارک‌دان سل است.
    عمل می‌کند O(len(p1) + len(p2)) این الگوریتم به صورت خطی و با حداکثر کارایی
    """
    print(f"{p1} و {p2}: در حال محاسبه اشتراک لیست‌ها")
    i, j = 0, 0 # دو نشانگر (pointer) دو نشانگر #
    result = []
    تا زمانی که هر دو نشانگر به انتها نرسیده‌اند، به مقایسه ادامه می‌دهیم
    while i < len(p1) and j < len(p2):
        اگر آیتم‌ها برابر بودند، در نتیجه ذکر شده و هر دو نشانگر جلو می‌روند
        if p1[i] == p2[j]:
            result.append(p1[i])
            i += 1
            j += 1
        اگر آیتم لیست اول کوچکتر بود، فقط نشانگر اول را جلو می‌بریم
```

```
        elif p1[i] < p2[j]:
            i += 1
            # در غیر این صورت، نشانگر دوم را جلو می‌بریم
        else:
            j += 1
    print(f"نتیجه اشتراک: {result}")
    return result

def union(p1, p2):
    """
    الگوریتم ادغام برای یافتن اجتماع دو لیست مرتب‌شده (بدون تکرار).
    استفاده می‌شود OR این تابع برای پردازش عملگر
    """
    print(f"{p1} و {p2}: در حال محاسبه اجتماع لیست‌ها")
    i, j = 0, 0
    result = []
    # تا زمانی که هر دو لیست آیتم دارند
    while i < len(p1) and j < len(p2):
        if p1[i] == p2[j]:
            result.append(p1[i])
            i += 1
            j += 1
        elif p1[i] < p2[j]:
            result.append(p1[i])
            i += 1
        else:
            result.append(p2[j])
            j += 1
    # اضافه کردن آیتم‌های باقی‌مانده در هر دو لیست
    result.extend(p1[i:])
    result.extend(p2[j:])
    print(f"نتیجه اجتماع: {result}")
    return result

def complement(postings_list):
    """
    برای پیاده‌سازی عملگر نفی (NOT) تابع مکمل
    این عملیات هزینه‌بر است چون به کل مجموعه اسناد نیاز دارد
    """
    print(f"{ postings_list}: در حال محاسبه مکمل لیست")
    p1_iter = iter(postings_list)
    current_doc_to_exclude = next(p1_iter, None)
    result = []
    for doc_id in ALL_DOCUMENTS:
        if current_doc_to_exclude is not None and doc_id == current_doc_to_exclude:
            current_doc_to_exclude = next(p1_iter, None)
        else:
            result.append(doc_id)
    # برای نمایش بهتر، فقط ۱۰ سند اول نتیجه را چاپ می‌کنیم
    print(f"{result[:10]}...")
    return result

def get_postings(term):
    """تابع کمکی برای گرفتن لیست ارسال یک واژه"""
    return inverted_index.get(term, [])

def process_conjunctive_query(terms):
    """
    با بهینه‌سازی (conjunctive query) پردازش یک پرسمان ترکیبی
    .واژه‌ها بر اساس فراوانی (طول لیست ارسال) به ترتیب صعودی پردازش می‌شوند
    .این مهمترین تکنیک بهینه‌سازی برای پرسمان‌های بولی است
    """
    print(f"\n--- پردازش پرسمان AND: {' AND '.join(terms)} ---")

    # مرحله ۱: مرتب‌سازی واژه‌ها بر اساس طول لیست ارسال (بهینه‌سازی کلیدی)
    terms_with_freq = [(term, len(get_postings(term))) for term in terms]
    terms_with_freq.sort(key=lambda x: x[1])
    sorted_terms = [term for term, freq in terms_with_freq]
    print(f"{sorted_terms}: ترتیب بهینه پردازش واژه‌ها (بر اساس فراوانی صعودی)")

    # مرحله ۲: شروع با کوچکترین لیست و اشتراک گرفتن تدریجی
    result = get_postings(sorted_terms[0])
    print(f"{sorted_terms[0]}': {result}")

    for term in sorted_terms[1:]:
        # اگر نتیجه میانی خالی شد، دیگر ادامه نده
        if not result:
            print("نتیجه میانی خالی شد، پردازش متوقف شد")
            break
        result = intersect(result, get_postings(term))

    print(f"نتیجه نهایی پرسمان: {result}")
    return result

# --- اجرای مثال‌ها ---

if __name__ == "__main__":

    # مثال ۱: پرسمان ساده (Brutus AND Calpurnia)
    print("="*50)
    print("مثال ۱: پرسمان ساده")
    brutus_postings = get_postings("Brutus")
    calpurnia_postings = get_postings("Calpurnia")
    intersect(brutus_postings, calpurnia_postings)

    # مثال ۲: پرسمان ترکیبی با بهینه‌سازی (Brutus AND Caesar AND Calpurnia)
    print("\n" + "="*50)
    print("مثال ۲: پرسمان ترکیبی با بهینه‌سازی")
    process_conjunctive_query(["Brutus", "Caesar", "Calpurnia"])

    # مثال ۳: پرسمان پیچیده با NOT و OR (Brutus OR Caesar) AND NOT (Antony OR Cleopatra)
    print("\n" + "="*50)
    print("NOT و OR مثال ۳: پرسمان پیچیده با")
    # (Brutus OR Caesar) AND NOT (Antony OR Cleopatra)
```

```
# محاسبه ۱: Brutus OR Caesar)
brutus_caesar_union = union(get_postings("Brutus"), get_postings("Caesar"))

# محاسبه ۲: Antony OR Cleopatra)
antony_cleopatra_union = union(get_postings("Antony"), get_postings("Cleopatra"))

# محاسبه ۳: NOT (Antony OR Cleopatra)
# هشدار: این عملیات بسیار سنگین است!
# antony_cleopatra_complement = complement(antony_cleopatra_union)

# مرحله ۴: محاسبه نهایی (اشتراک)
# نتیجه نهایی = intersect(brutus_caesar_union, antony_cleopatra_complement)
# آن را کامنت می‌کنیم، NOT برای جلوگیری از اجرای عملیات سنگین
print("محاسبه نهایی انجام نمی‌شود، NOT برای جلوگیری از عملیات سنگین")
print("استفاده می‌کنند NOT در عمل، موتورهای جستجو از تکنیک‌های هوشمندانه‌تری برای پردازش")
```

=====

مثال ۱: پرسمان ساده

در حال محاسبه اشتراک لیست‌ها: [1, 2, 4, 11, 31, 45, 173, 174] و [2, 31, 54, 101]

نتیجه اشتراک: [2, 31]

=====

مثال ۲: پرسمان ترکیبی با بهینه‌سازی

--- AND: Brutus AND Caesar AND Calpurnia ---

ترتیب بهینه پردازش واژه‌ها (بر اساس فراوانی صعودی): ['Calpurnia', 'Brutus', 'Caesar']

'Calpurnia': [2, 31, 54, 101] شروع با لیست کوچکترین واژه

در حال محاسبه اشتراک لیست‌ها: [2, 31, 54, 101] و [1, 2, 4, 11, 31, 45, 173, 174]

نتیجه اشتراک: [2, 31]

در حال محاسبه اشتراک لیست‌ها: [2, 31] و [1, 2, 4, 5, 6, 16, 57, 132]

نتیجه اشتراک: [2]

نتیجه نهایی پرسمان: [2]

=====

NOT و OR مثال ۳: پرسمان پیچیده با

در حال محاسبه اجتماع لیست‌ها: [1, 2, 4, 11, 31, 45, 173, 174] و [1, 2, 4, 5, 6, 16, 57, 132]

نتیجه اجتماع: [1, 2, 4, 5, 6, 11, 16, 31, 45, 57, 132, 173, 174]

در حال محاسبه اجتماع لیست‌ها: [5, 10, 31] و [2, 3, 31, 64]

نتیجه اجتماع: [2, 3, 5, 10, 31, 64]

محاسبه نهایی انجام نمی‌شود، NOT برای جلوگیری از عملیات سنگین

استفاده می‌کنند NOT در عمل، موتورهای جستجو از تکنیک‌های هوشمندانه‌تری برای پردازش

🔍 1.4 مدل بولی گسترش‌یافته در مقابل بازیابی رتبه‌بندی‌شده

💡 دو دستیار در ذهن شما تصور کنید

دستیار اول بسیار دقیق و خشک است. اگر بگویید «قهوه و کیک»، دقیقاً اسنادی را پیدا می‌کند که هر دو کلمه را داشته باشند، نه یکی بیشتر. این دستیار، **مدل بولی** است.

دستیار دوم هوشمند و شهودی است. اگر بگویید «یک کافه دنج برای عصر جمعه»، او با توجه به کل جملات، مفهوم «دنج» و «عصر جمعه» را می‌فهمد و بهترین گزینه‌ها را به شما پیشنهاد می‌دهد، حتی اگر دقیقاً این کلمات در متن نباشند. این دستیار، **مدل بازیابی رتبه‌بندی‌شده (Ranked Retrieval)** است.

برای دهه‌ها، دنیای جستجوی حرفه‌ای، به‌ویژه در حوزه‌های تخصصی مانند حقوق، تحت سلطه دستیار اول (مدل بولی) بود. اما نسخه‌ی ساده‌ی بولی که با `AND` و `OR` آشنا شدیم، برای کاربران حرفه‌ای بسیار محدود بود. آن‌ها به یک نسخه‌ی **ابرسوپر** نیاز داشتند!

مهم‌ترین ارتقا، اضافه شدن **عملگرهای نزدیکی (Proximity Operators)** بود. این عملگرها به دستیار ما کمک می‌کنند تا بفهمد دو کلمه فقط در یک سند وجود ندارند، بلکه «کنار هم» معنای خاصی ایجاد می‌کنند.

🎬 مطالعه موردی: دنیای جستجوی حقوقی (Westlaw)

تصور کنید یک وکیل برجسته هستید و باید پرونده‌ای پیچیده درباره «مسئولیت میزبان در قبال مهمانان مست» را تحقیق کنید. در سیستم Westlaw، شما یک پرسمان غول‌پیکر و دقیق می‌نویسید، نه یک جستجوی ساده:

```
host! /p (responsib! liab!) /p (intoxicat! drunk!) /p guest
```

بیاپید این جادو را با هم بشکافیم:

- `!host` : هر کلمه‌ای که با `host` شروع شود (مثل `host`, `host's`).
- `p/` : معجون اصلی! یعنی «در همان پاراگراف».
- `(!responsib! liab)` : کلماتی مثل `responsible` یا `liability`.

این پرسمان می‌گوید: «اسنادی را پیدا کن که در یک پاراگراف، کلماتی از خانواده `host`، در پاراگراف دیگری کلماتی از خانواده مسئولیت، و در پاراگراف سوم کلماتی از خانواده مستی و مهمان وجود داشته باشد.» این سطح از کنترل، برای یک متخصص طلاست! 🏆

اما یک بحث بزرگ همیشه وجود داشته: کدام روش بهتر است؟

📊 نبرد دقت (Precision) در مقابل بازیابی (Recall)

مدل بولی یک مشکل اساسی دارد:

- استفاده زیاد از `AND` → نتایج بسیار **مربوط** ولی **کم**. (دقت بالا، بازیابی پایین)
- استفاده زیاد از `OR` → نتایج بسیار **زیاد** ولی بسیاری از آن‌ها **غیرمربوط**. (دقت پایین، بازیابی بالا)

یافتن نقطه‌ی طلایی بین این دو، مثل راه رفتن روی تیغ یک کارد است! به همین دلیل، مطالعات (مانند تحقیق Turtle در ۱۹۹۴) نشان دادند که حتی برای وکلا، پرسمان‌های متنی آزاد (دستیار دوم) اغلب نتایج بهتری تولید می‌کنند.

پس راه حل چیست؟ چگونه می‌توانیم بهترین هر دو دنیا را داشته باشیم؟ اینجاست که ما به **چهار ستون موتورهای جستجوی مدرن** می‌رسیم:

1. 🔍 **جستجوی بخشنده (Tolerant Search)**: سیستمی که اشتباهات املایی شما را می‌بخشد و می‌فهمد که «ماشین» و «دستگاه» می‌توانند مترادف باشند.
2. 🌿 **جستجوی بافتار (Proximity & Phrase Search)**: سیستمی که تفاوت بین «سگ گاز می‌گیرد» و «انسان به سگ گاز می‌گیرد» را می‌فهمد. این یعنی درک نزدیکی و ترتیب کلمات.
3. 📊 **اهمیت فراوانی (Term Frequency)**: سیستمی که می‌داند اگر کلمه «هوش مصنوعی» در یک سند ۱۰ بار تکرار شده، آن سند احتمالاً از سندی که فقط یک بار به آن اشاره کرده، مرتبط‌تر است.
4. 🏆 **هنر رتبه‌بندی (Ranking)**: سیستمی که به جای گفتن «بله/خیر»، به هر سند یک امتیاز از ۱۰۰ می‌دهد و بهترین‌ها را در بالای لیست قرار می‌دهد.

🌐 سفر از کتابخانه حقوق تا گوگل

این چهار ستون، فناوری است که امروزه دنیا را تسخیر کرده است. بار بعدی که در گوگل عبارتی را جستجو می‌کنید، به یاد داشته باشید که در پشت این کادر ساده، این ایده‌ها با قدرت در حال کار هستند. گوگل ممکن است ظاهراً یک جستجوی متنی آزاد ارائه دهد، اما در پشت صحنه از جستجوی عبارت (دستور دوم) و نسخه‌های ساده‌شده‌ای از منطق بولی استفاده می‌کند.

```
In [3]: # --- تعریف ساختار داده‌ها ---

# کلاس ساده برای نمایش یک سند با محتوا و پاراگراف‌های آن
class Document:
    def __init__(self, doc_id, content):
        self.doc_id = doc_id
        # تقسیم محتوا به پاراگراف‌ها بر اساس خطوط خالی
        self.paragraphs = [p.strip() for p in content.split('\n\n') if p.strip()]
```

```
ایجاد لیستی از کلمات در هر پاراگراف برای جستجوی سریع‌تر #
self.paragraph_words = [p.lower().split() for p in self.paragraphs]
```

```
def __str__(self):
    return f"سند {self.doc_id}: {self.paragraphs[0][:50]}..."
```

نمونه‌ای از مجموعه اسناد کوچک

```
documents = [
    Document(1, "مهمانان در مهمانی مست بودند. میزبان مسئولیت مواد الکلی را بر عهده داشت. مهمانان پس از ترک مهمانی تصادف کردند"),
    Document(2, "میزبان مهمانی دیگری برگزار کرد. در این مهمانی، مهمانان الکل مصرف نکردند. مسئولیت میزبان در این مورد کمتر بود"),
    Document(3, "قوانین مربوط به مسئولیت میزبان در بسیاری از کشورها وجود دارد. اگر مهمان مست باشد و حادثه‌ای رخ دهد، میزبان مسئول است"),
    Document(4, "مهمان و میزبان توافق‌نامه‌ای قبل از مهمانی امضا کردند. این توافق‌نامه مسئولیت‌ها را مشخص می‌کرد"),
    Document(5, "در این مهمانی، میزبان به مهمانان الکل نداد. با این حال، یکی از مهمانان مشروب خود را آورده بود و مست شد")
]
```

--- پیاده‌سازی الگوریتم‌های جستجو ---

```
def boolean_search(documents, query_terms):
    """
    جستجوی بولی ساده: اسنادی را برمی‌گرداند که تمام واژه‌های پرسمان را دارند
    """

    results = []
    for doc in documents:
        # تبدیل کل سند به حروف کوچک برای جستجوی حساس‌به‌حروف بزرگ/کوچک نباشیم
        doc_text = ' '.join(doc.paragraphs).lower()
        if all(term.lower() in doc_text for term in query_terms):
            results.append(doc)
    return results
```

```
def proximity_search_paragraph(doc, term1, term2):
    """
    بررسی می‌کند که آیا دو واژه در یک پاراگراف از سند وجود دارند یا نه
    است (همان پاراگراف) p / این پیاده‌سازی برای عملگر
    """

    for words in doc.paragraph_words:
        has_term1 = any(term1.lower() in word for word in words)
        has_term2 = any(term2.lower() in word for word in words)
        if has_term1 and has_term2:
            return True
    return False
```

```
def extended_boolean_search(documents, term1, term2, operator):
    """
    جستجوی بولی گسترش‌یافته با عملگرهای نزدیکی
    """

    results = []
    for doc in documents:
        if operator == "AND":
            # جستجوی بولی ساده
            doc_text = ' '.join(doc.paragraphs).lower()
            if term1.lower() in doc_text and term2.lower() in doc_text:
                results.append(doc)
        elif operator == "/p":
            # جستجوی نزدیکی در همان پاراگراف
            if proximity_search_paragraph(doc, term1, term2):
                results.append(doc)
    return results
```

```
def term_frequency(doc, term):
    """
    تعداد تکرار یک واژه در یک سند را می‌شمارد
    """

    count = 0
    for paragraph in doc.paragraphs:
        words = paragraph.lower().split()
        count += sum(1 for word in words if term.lower() in word)
    return count
```

```
def ranked_retrieval(documents, query_terms):
    """
    بازیابی رتبه‌بندی‌شده: اسناد را بر اساس مجموع فراوانی واژه‌های پرسمان رتبه‌بندی می‌کند
    """

    doc_scores = []
    for doc in documents:
        score = sum(term_frequency(doc, term) for term in query_terms)
        فقط اسنادی که حداقل یکی از واژه‌ها را دارند #
        doc_scores.append((doc, score))

    مرتب‌سازی اسناد بر اساس امتیاز (از بیشترین به کمترین) #
    doc_scores.sort(key=lambda x: x[1], reverse=True)
    return doc_scores
```

--- اجرای مثال‌ها ---

```
if __name__ == "__main__":

    مثال ۱: مقایسه جستجوی بولی ساده و جستجوی نزدیکی #
    print("*60")
    print("مثال ۱: مقایسه جستجوی بولی ساده و جستجوی نزدیکی")
    term1 = "میزبان"
    term2 = "مهمان"

    # جستجوی بولی ساده (AND)
    boolean_results = boolean_search(documents, [term1, term2])
    print(f"\nنتایج جستجوی بولی ساده({term1} AND {term2}):")
    for doc in boolean_results:
        print(f" - {doc}")

    # جستجوی نزدیکی (/p)
    proximity_results = extended_boolean_search(documents, term1, term2, "/p")
    print(f"\nنتایج جستجوی نزدیکی({term1} /p {term2}):")
    for doc in proximity_results:
        print(f" - {doc}")
```

```
# مثال ۲: مقایسه مدل بولی و مدل رتبه‌بندی‌شده
print("\n" + "="*60)
print("مثال ۲: مقایسه مدل بولی و مدل رتبه‌بندی‌شده")
query_terms = ["میزبان", "مسئولیت"]

# نتایج مدل بولی
boolean_results = boolean_search(documents, query_terms)
print(f"\n(بدون رتبه‌بندی):")
for doc in boolean_results:
    print(f"    - {doc}")

# نتایج مدل رتبه‌بندی‌شده
ranked_results = ranked_retrieval(documents, query_terms)
print(f"\n(با امتیاز):")
for doc, score in ranked_results:
    print(f"    - امتیاز {score}: {doc}")

# مثال ۳: نمایش فراوانی واژه در اسناد مختلف
print("\n" + "="*60)
print("مثال ۳: فراوانی واژه در اسناد مختلف")
term = "میزبان"
print(f"\n(در اسناد '{term}' فراوانی واژه):")
for doc in documents:
    freq = term_frequency(doc, term)
    print(f"    - {doc}: {freq} بار")
```

مثال ۱: مقایسه جستجوی بولی ساده و جستجوی نزدیکی

:(مهمان AND میزبان) نتایج جستجوی بولی ساده

- ...سند 1: مهمانان در مهمانی مست بودند. میزبان مسئولیت مواد ا -
- ...سند 2: میزبان مهمانی دیگری برگزار کرد. در این مهمانی، مهم -
- ...سند 3: قوانین مربوط به مسئولیت میزبان در بسیاری از کشورها -
- ...سند 4: مهمان و میزبان توافق‌نامه‌ای قبل از مهمانی امضا کر -
- ...سند 5: در این مهمانی، میزبان به مهمانان الکل نداد. با این -

:(مهمان p / میزبان) نتایج جستجوی نزدیکی

- ...سند 1: مهمانان در مهمانی مست بودند. میزبان مسئولیت مواد ا -
- ...سند 2: میزبان مهمانی دیگری برگزار کرد. در این مهمانی، مهم -
- ...سند 3: قوانین مربوط به مسئولیت میزبان در بسیاری از کشورها -
- ...سند 4: مهمان و میزبان توافق‌نامه‌ای قبل از مهمانی امضا کر -
- ...سند 5: در این مهمانی، میزبان به مهمانان الکل نداد. با این -

مثال ۲: مقایسه مدل بولی و مدل رتبه‌بندی‌شده

:نتایج مدل بولی (بدون رتبه‌بندی)

- ...سند 1: مهمانان در مهمانی مست بودند. میزبان مسئولیت مواد ا -
- ...سند 2: میزبان مهمانی دیگری برگزار کرد. در این مهمانی، مهم -
- ...سند 3: قوانین مربوط به مسئولیت میزبان در بسیاری از کشورها -
- ...سند 4: مهمان و میزبان توافق‌نامه‌ای قبل از مهمانی امضا کر -

:نتایج مدل رتبه‌بندی‌شده (با امتیاز)

- ...امتیاز 3: سند 2: میزبان مهمانی دیگری برگزار کرد. در این مهمانی، مهم -
- ...امتیاز 3: سند 3: قوانین مربوط به مسئولیت میزبان در بسیاری از کشورها -
- ...امتیاز 2: سند 1: مهمانان در مهمانی مست بودند. میزبان مسئولیت مواد ا -
- ...امتیاز 2: سند 4: مهمان و میزبان توافق‌نامه‌ای قبل از مهمانی امضا کر -
- ...امتیاز 1: سند 5: در این مهمانی، میزبان به مهمانان الکل نداد. با این -

مثال ۳: فراوانی واژه در اسناد مختلف

:فراوانی واژه 'میزبان' در اسناد

- سند 1: مهمانان در مهمانی مست بودند. میزبان مسئولیت مواد ا...: 1 بار
- سند 2: میزبان مهمانی دیگری برگزار کرد. در این مهمانی، مهم...: 2 بار
- سند 3: قوانین مربوط به مسئولیت میزبان در بسیاری از کشورها...: 2 بار
- سند 4: مهمان و میزبان توافق‌نامه‌ای قبل از مهمانی امضا کر...: 1 بار
- سند 5: در این مهمانی، میزبان به مهمانان الکل نداد. با این...: 1 بار

جمع‌بندی فصل ۱: بازیابی بولی

در این فصل با مفاهیم پایه‌ای بازیابی اطلاعات آشنا شدیم:

- مدل بولی را به‌عنوان سنگ بنای سیستم‌های جستجو معرفی کردیم. این مدل همان منطقی است که در کتابخانه‌ها با

جستجوی «و» و «یا» استفاده می‌کنیم - مثلاً جستجوی «هوش مصنوعی و یادگیری ماشین» در پایگاه داده مقالات علمی.

- ماتریس وقوع بولی و محدودیت‌های آن را بررسی کردیم. تصور کنید یک کتابخانه با هزاران کتاب دارید و می‌خواهید

بدانید کدام کتاب‌ها کلمات کلیدی خاصی دارند - ماتریس وقوع این کار را انجام می‌دهد، اما برای کتابخانه‌های بزرگ

بسیار ناکارآمد است!

- فهرست معکوس را به‌عنوان راه‌حلی کارآمد برای ذخیره‌سازی و جست‌وجو معرفی کردیم. این همان فهرستی است که در

انتهای کتاب‌ها می‌بینیم، اما برای جستجوی کامپیوتری بهینه شده است. گوگل در نسخه‌های اولیه خود از این روش

استفاده می‌کرد.

- الگوریتم‌های اشتراک (AND)، ادغام (OR) و بهینه‌سازی ترتیب پردازش را پیاده‌سازی کردیم. وقتی در اسنپ فود «پیتزا و نوشابه» را جستجو می‌کنید، در واقع از الگوریتم AND استفاده می‌کنید - فقط نتایجی نمایش داده می‌شوند که هر دو کلمه را داشته باشند.
- تفاوت مدل بولی ساده، بولی گسترش‌یافته، و بازیابی رتبه‌بندی‌شده را بررسی کردیم. این همان تفاوتی است که بین جستجوی ساده در فایل‌های ویندوز و جستجوی پیشرفته در گوگل وجود دارد.

بیا یاد این مفاهیم را با چند مثال واقعی مقایسه کنیم:

- ♦ **بازیابی بولی ساده:** تصور کنید در دیجی‌کالا به دنبال «گوشی سامسونگ با حافظه ۲۵۶ گیگابایت و قیمت زیر ۱۰ میلیون تومان» می‌گردید. این جستجو دقیقاً از منطق بولی استفاده می‌کند: (گوشی AND سامسونگ) AND (حافظه ۲۵۶ AND (قیمت زیر ۱۰ میلیون)). سیستم ابتدا تمام گوشی‌های سامسونگ را پیدا می‌کند، سپس از میان آن‌ها موارد با حافظه ۲۵۶ را جدا کرده و در نهایت آن‌هایی را نمایش می‌دهد که قیمت مناسب داشته باشند. نتیجه یا هست یا نیست؛ هیچ رتبه‌بندی وجود ندارد.
- ♦ **بازیابی بولی گسترش‌یافته:** یک وکیل در پایگاه داده قوانین و آراء دادگاه به دنبال یک مورد خاص می‌گردد. او جستجوی زیر را انجام می‌دهد:
«مسئولیت قراردادی NEAR/10 خسارت»
در اینجا، `NEAR/10` یک عملگر گسترش‌یافته است که به سیستم می‌گوید فقط سندهایی را پیدا کن که عبارت «مسئولیت قراردادی» و «خسارت» در فاصله کمتر از ۱۰ کلمه از هم قرار دارند. این کار از پیدا کردن سندهایی که این دو کلمه را به صورت پراکنده در آن آورده‌اند، بسیار دقیق‌تر است.
- ♦ **بازیابی رتبه‌بندی‌شده:** شما در گوگل عبارت «بهترین کافه‌های قصرالدشت» را جستجو می‌کنید. گوگل هزاران صفحه را پیدا می‌کند که این کلمات را دارند. اما به جای نمایش همه آن‌ها، آن‌ها را **رتبه‌بندی** می‌کند. نتیجه اول احتمالاً کافه‌ای است که امتیاز بالایی دارد، نظرات مثبت زیادی گرفته، به موقعیت مکانی شما نزدیک است و در وبسایت‌های معتبر به آن اشاره شده است. نتیجه پنجم ممکن است کافه خوبی باشد اما امتیاز کمتری دارد. این مدل، نتایج را بر اساس «مرتبطی» به شما نمایش می‌دهد، نه فقط بر اساس تطابق کلمات.

این فصل پایه‌های سخت‌افزاری و ساختاری سیستم‌های جست‌وجو را فراهم می‌کند. در فصل‌های بعدی، با مفاهیمی مانند:

- وزن‌دهی به واژه‌ها (tf-idf) - همان روشی که گوگل تشخیص می‌دهد کلمه «گوشی» در یک صفحه مهم‌تر از کلمه «و» است
- مدل فضای برداری - تکنیکی که به سیستم‌های جست‌وجو اجازه می‌دهد مفاهیم مشابه را پیدا کنند، حتی اگر کلمات یکسانی استفاده نشده باشد
- ارزیابی سیستم‌ها (دقت، بازیابی، F1) - معیارهایی برای سنجش اینکه آیا موتور جست‌وجوی ما واقعاً خوب کار می‌کند یا نه
- خوشه‌بندی و طبقه‌بندی متن - روشی که گوگل اخبار را به دسته‌های ورزشی، سیاسی و اقتصادی تقسیم می‌کند

آشنا خواهیم شد - که همگی بر همین پایه فهرست معکوس بنا شده‌اند.

End of Chapter Exercises

مسئله 1.1: تحلیل پیچیدگی زمانی در نمایه وارونه

تفاوت اصلی در رویکرد جستجوی خطی با استفاده از «نمایه وارونه» (**Inverted Index**) در چیست؟
در چه مقیاسی از حجم داده‌ها، این تفاوت از نظر پیچیدگی زمانی (**Time Complexity**) حیاتی می‌شود؟

مسئله 1.3: معماری و اجزای اصلی نمایه وارونه

اجزای اصلی یک **Inverted Index** را تعریف کنید و وظیفه دقیق هر کدام را در فرآیند پیدا کردن یک کلمه شرح دهید.

مسئله 1.4: نقش پیش‌پردازش زبانی و توکنایزیشن در ساخت نمایه

در مرحله ساخت نمایه (**Index Construction**)، چرا انجام مراحلی مانند **Tokenization** و اعمال ماژول‌های زبانی (**Linguistic Modules**) پیش از ایجاد لیست‌های ارجاع ضروری است؟
یک مثال از خروجی سیستم بدون این مراحل بزنید.

مسئله 1.5: بهینه‌سازی پرسمان و ترتیب اجرای عملیات اشتراک

فرض کنید در دیکشنری، فرکانس سند (**DF**) برای کلمات به این شرح است:
Data = 1000، **Mining** = 500، **Web** = 10000 و **Search** = 200.
کاربر پرسمان (**Web AND Data**) AND (**Mining AND Search**) را وارد کرده است.
ترتیب بهینه اجرای عملیات اشتراک (**Query Optimization**) را مرحله به مرحله بنویسید تا کمترین هزینه پردازش به سیستم تحمیل شود.

مسئله 1.6: پیامد عدم مرتب‌سازی لیست‌های ارجاع در الگوریتم ادغام

در پروژه کلاسی، اگر دانشجویی لیست‌های ارجاع (**Postings Lists**) را بدون مرتب‌سازی (**Sorting**) شناسه‌های سند (**DocID**) در فایل ذخیره کند، الگوریتم استاندارد دو اشاره‌گره (**Standard Merge Algorithm**) در زمان اجرای پرسمان **A AND B** چه رفتاری از خود نشان می‌دهد؟

مسئله 1.7: محاسبه تعداد مقایسه‌ها در الگوریتم ادغام

دو کلمه **A** و **B** به ترتیب دارای لیست ارجاع (**Postings Lists**) **[1, 3, 5, 7, 9, 11]** و **[2, 3, 4, 5, 11, 15]** هستند. برای یافتن اشتراک این دو لیست با الگوریتم استاندارد ادغام (**Standard Merge Algorithm**)، اشاره‌گرها دقیقاً چند مقایسه (**Comparison**) انجام می‌دهند تا به انتهای لیست برسند؟
مسیر حرکت آن‌ها را ردیابی کنید.

مسئله 1.8: تحلیل پردازش پرسمان‌های منطقی و هزینه عملگر NOT

کاربر پرسمانی به شکل **(A OR B) AND NOT C** را وارد کرده است. با استفاده از عملگرهای مجموعه، نحوه پردازش این پرسمان روی لیست‌های ارجاع (**Postings Lists**) را شرح دهید.
چرا هزینه پردازش عملگر **NOT** در موتورهای جستجو معمولاً بالاتر از **AND** ارزیابی می‌شود؟

مسئله 1.9: ارتقای نمایه برای پشتیبانی از پرسمان‌های عبارتی

سیستم نمایه‌سازی فعلی فقط شناسه‌های سند (DocID) را ذخیره می‌کند.

برای پشتیبانی از پرسمان‌های عبارتی دقیق (Phrase Queries) مانند جستجوی عبارت "Machine Learning" (دقیقاً پشت سر هم)، ساختار ذخیره‌سازی لیست‌های ارجاع (Postings Lists) باید چه تغییری کند و چه اطلاعات اضافه‌ای باید نگهداری شود؟

مسئله 1.10: تحلیل کارایی صف اولویت در بهینه‌سازی پرسمان‌های OR

یکی از تیم‌های دانشجویی پیشنهاد داده است که برای بهینه‌سازی پرسمان‌هایی که دارای چندین عبارت OR هستند (مثل A OR B OR C OR D)، به جای محاسبه دو به دو، از ساختمان داده صف اولویت (Priority Queue) استفاده شود. صحت و کارایی این پیشنهاد را از نظر پیچیدگی زمانی (Time Complexity) تحلیل کنید.

مسئله 1.11: پیاده‌سازی و تحلیل سربار عملگرهای مجاورت

در مدل بولین توسعه‌یافته (Extended Boolean)، چگونه می‌توان با استفاده از عملگرهای مجاورت (Proximity Operators) پرسمانی نوشت که کلمه «دانشگاه» و «تهران» را با فاصله حداکثر 2 کلمه از یکدیگر پیدا کند؟ پیاده‌سازی این عملگر در زمان اجرا چه سرباری دارد؟

مسئله 1.12: محدودیت‌های تطابق دقیق در بازیابی بولین کلاسیک

چرا مدل ارزیابی در بازیابی بولین کلاسیک به صورت 0 و 1 (Exact Match) است؟ یک سناریوی ساده مثال بزنید که در آن کاربری به دنبال یک سند خاص است، اما به دلیل ماهیت 0 و 1 بودن این مدل، با خطای منطقی مواجه شده و هیچ نتیجه‌ای دریافت نمی‌کند.

مسئله 1.13: توزیع نمایه وارونه و مقایسه معماری‌های پارتیشن‌بندی ★

فرض کنید حجم اسناد به قدری زیاد است که نمایه وارونه روی یک سرور جا نمی‌شود و باید آن را بین چندین سرور توزیع کنید (Sharding). دو رویکرد Term-partitioning (تقسیم بر اساس کلمات) و Document-partitioning (تقسیم بر اساس اسناد) را با هم مقایسه کنید. برای اجرای پرسمان A AND B، شبکه در کدام روش بیشتر درگیر می‌شود و گلوگاه (Bottleneck) سیستم کجاست؟

مسئله 1.14: مثال نقض و تحلیل سربار در اشاره‌گرهای پرشی ★

در یک جلسه دفاع شفاهی، دانشجویی ادعا می‌کند که اضافه کردن Skip Pointers همیشه و در هر شرایطی زمان اشتراک لیست‌ها را کاهش می‌دهد. یک مثال نقض عددی طراحی کنید (شامل طول گام پرش و عناصر دو لیست) که در آن، استفاده از پرشگرها نه تنها سرعت را بالا نبرده، بلکه به دلیل سربار بررسی شرط‌های پرش، تعداد کل مقایسه‌ها را نسبت به حالت خطی معمولی افزایش می‌دهد.

مسئله 1.15: معماری کش در موتورهای جستجو (تحلیل Trade-off) ★

در یک موتور جستجوی واقعی، پرسمان‌های تکراری زیادی وجود دارد. اگر بخواهید نتایج پرسمان‌های رایج را کش کنید، آیا از نظر معماری بهتر است نتیجه نهایی پرسمان (DocID های خروجی) را کش کنید یا لیست‌های ارجاع (Postings Lists) کلمات آن پرسمان را در RAM نگه دارید؟
با تحلیل Trade-off بین فضای حافظه و سرعت به‌روزرسانی سیستم به این چالش پاسخ دهید.

مسئله 1.16: تحلیل مشکل قحطی نتیجه (Famine) و گذر به بازیابی رتبه‌بندی شده ★

یک فروشگاه اینترنتی گزارش داده که در پرسمان laptop AND gaming AND asus AND 32GB هیچ نتیجه‌ای به کاربر نمایش داده نمی‌شود (Famine)، در حالی که لپ‌تاپ‌های گیمینگ ایسوس 16 گیگابایتی موجود هستند که کاربر احتمالاً یکی را می‌خرد.

از منظر تئوری بازیابی، توضیح دهید چرا مدل Boolean در این سناریو به بن‌بست خورده است و چگونه گذر به مدل Ranked Retrieval و جستجوی متن آزاد (Free text) این مشکل را حل می‌کند؟

مسئله 1.17: معماری دوگانه نمایه و مدیریت به‌روزرسانی بلادرنگ ★

در سایتی که روزانه هزاران خبر جدید منتشر می‌کند، نمایه وارونه اصلی روی دیسک کاملاً فشرده است و آپدیت لحظه‌ای آن غیرممکن است. یک معماری دوگانه شامل Main Index و Auxiliary Index طراحی کنید که مشکل جستجوی بلادرنگ اخبار جدید را حل کند.
چالش ادغام این دو نمایه و مدیریت اسناد حذف شده (Deletion) چگونه باید رفع شود؟

مسئله 1.18: استراتژی پردازش پرسمان ترکیبی و تحلیل هزینه ★

در پرسمان A AND (B OR C)، فرض کنید کلمه A بسیار نادر است (فقط در 10 سند) اما کلمات B و C کلماتی بسیار پرتکرار هستند (حضور در 1 میلیون سند). الگوریتم بهینه‌ساز پرسمان (Query Optimizer) چه تصمیمی باید بگیرد؟
آیا محاسبه عبارت داخل پرانتز (B OR C) پیش از اعمال عملگر AND منطقی است؟
یک رویکرد جایگزین برای پردازش سریع‌تر ارائه دهید.

مسئله 1.19: تحلیل پیچیدگی زمانی اشتراک چندگانه و ارزیابی جدول درهم‌سازی ★

اگر بخواهیم اشتراک k لیست ارجاع (Postings Lists) را (که هرکدام به طور میانگین N طول دارند) محاسبه کنیم، مرتب‌سازی لیست‌ها بر اساس طول فرکانس و سپس اشتراک متوالی آن‌ها، در بدترین حالت چه مرتبه زمانی (O) خواهد داشت؟
آیا استفاده از جدول درهم‌سازی (Hash Table) برای DocID ها جایگزین بهتری برای الگوریتم استاندارد Merge است؟
معایب استفاده از Hash در این سناریو را بیان کنید.

مسئله 1.20: تحلیل خطای پیاده‌سازی عملگر AND و تداخل مدل‌های بازیابی ★

شما در حال بررسی و داوری کدهای یک تیم دانشجویی هستید. متوجه می‌شوید که خروجی الگوریتم جستجوی آن‌ها در پرسمان‌های چندکلمه‌ای کاملاً بی‌ربط است. با بررسی سورس کد، می‌بینید آن‌ها به جای پیاده‌سازی عملگر AND منطقی (اشتراک مجموعه‌ها)، فرکانس کلمات پرسمان را در هر سند جمع زده‌اند و اسنادی که جمع فرکانس بالاتری دارند را برگردانده‌اند.

با اصطلاحات علمی فصل اول توضیح دهید چرا این پیاده‌سازی از اساس در مدل **Boolean Retrieval** غلط است و این گروه چه مفهومی از سیستم‌های رتبه‌بندی را به اشتباه وارد مدل کلاسیک بولین کرده‌اند؟

مسئله 1.21: تحلیل شکست تجاری مدل بولین و گذر به بازیابی رتبه‌بندی شده ★

تیم فروش یک فروشگاه اینترنتی گزارش داده که سیستم جستجوی فعلی (که بر پایه مدل **Extended Boolean** کار می‌کند) باعث افت شدید فروش شده است. بررسی لاگ‌ها نشان می‌دهد کاربری پرسمان

laptop AND gaming AND asus AND 16GB را جستجو کرده و سیستم دقیقاً 0 نتیجه برگردانده است. این در حالی است که فروشگاه ده‌ها لپ‌تاپ گیمینگ ایسوس با حافظه **32GB** و لپ‌تاپ‌های گیمینگ لنوو با حافظه **16GB** در انبار دارد که کاربر در صورت مشاهده قطعاً یکی را می‌خرد.

الف) به عنوان معمار سیستم، از نظر تئوری بازیابی اطلاعات، توضیح دهید چرا مدل **Boolean** در این سناریو به بن‌بست خورده و شکست تجاری خورده است؟

ب) چگونه گذر از **Boolean Retrieval** به مدل **Ranked Retrieval** و استفاده از رویکرد **Free-text query** می‌تواند این مشکل را حل کند؟

نحوه برخورد سیستم جدید با این پرسمان را به صورت مفهومی شرح دهید.

مسئله 1.22: طراحی سناریوی بدترین حالت در اشاره‌گرهای پرشی ★

برای تسریع عملیات اشتراک (**AND**) در لیست‌های ارجاع (**Postings Lists**) بسیار طولانی، از ساختار **Skip Pointers** استفاده کرده‌ایم. می‌دانیم که این ساختار معمولاً جستجو را سریع‌تر می‌کند. اما سیستم همیشه در حالت ایده‌آل کار نمی‌کند. سناریویی را طراحی کنید (با ارائه یک مثال عددی از دو لیست **L_1** و **L_2** همراه با محل قرارگیری پرش‌ها) که نشان دهد در بدترین حالت (**Worst-case**)، استفاده از **Skip Pointers** نه‌تنها به افزایش سرعت کمکی نمی‌کند، بلکه به دلیل سربار (**Overhead**) بررسی شرط پرش‌ها، الگوریتم را از حالت خطی معمولی نیز کندتر می‌کند.

طول گام پرش‌ها در این حالت بدبینانه چه تاثیری دارد؟

مسئله 1.23: معماری نمایه پویا و جستجوی بلادرنگ ★

شما سیستم بازیابی اطلاعات یک خبرگزاری بزرگ را طراحی کرده‌اید که علاوه بر آرشیو، روزانه ده‌ها هزار خبر جدید دریافت می‌کند. نمایه وارونه اصلی شما بسیار حجیم است و روی هارد دیسک (**Disk**) ذخیره شده است.

اگر بخواهید اخبار جدید در لحظه (**Real-time**) قابل جستجو باشند، آپدیت کردن مستقیم نمایه وارونه روی دیسک به ازای ورود هر خبر، سیستم را به شدت کند می‌کند.

به عنوان یک مهندس نرم‌افزار، چه راهکار معماری و داده‌ساختاری برای مدیریت این مشکل پیشنهاد می‌دهید که هم اخبار قدیمی و هم اخبار ثانیه‌های اخیر با بالاترین سرعت جستجو شوند؟

(راهنمایی: به استفاده همزمان از حافظه اصلی (**RAM**) و ساختارهای کمکی فکر کنید).

پاسخنامه مسئله 1.1: تحلیل و مقایسه رویکردهای جستجوی خطی

مقدمه و طرح مسئله

در سیستم‌های بازبازی اطلاعات (**Information Retrieval**)، رویکرد الگوریتمی به فرآیند جستجو، تعیین‌کننده اصلی کارایی سیستم است. برای یافتن یک عبارت خاص در مجموعه‌ای از اسناد (**Corpus**)، دو پارادایم کلی وجود دارد: جستجوی ترتیبی (**Sequential**) و استفاده از ساختارهای داده پیش‌پردازش‌شده. این مسئله به بررسی تفاوت بنیادین این دو روش و تحلیل مقیاس‌پذیری آن‌ها می‌پردازد.

1. جستجوی خطی (**Linear Search**) و ابزارهایی نظیر **grep**

در رویکرد جستجوی خطی، سیستم در زمان ثبت درخواست (**Query Time**)، محتوای اسناد را به صورت کاراکتر به کاراکتر یا کلمه به کلمه پیمایش می‌کند تا عبارت مورد نظر را بیابد. عملکرد ابزارهایی مانند **grep** در سیستم‌عامل‌های مبتنی بر یونیکس بر این اساس استوار است.

پیچیدگی زمانی: اگر حجم کل داده‌ها (تعداد کل کلمات یا کاراکترهای موجود در تمامی اسناد) را برابر با **N** در نظر بگیریم، پیچیدگی زمانی این رویکرد از مرتبه **O(N)** خواهد بود. این بدان معناست که با افزایش حجم متون، زمان اجرای الگوریتم نیز به صورت خطی افزایش می‌یابد.

2. رویکرد نمایه وارونه (**Inverted Index**)

نمایه وارونه ساختار داده‌ای است که پیش از زمان ثبت درخواست و طی فاز پیش‌پردازش (**Pre-processing**) ایجاد می‌شود. در این رویکرد، یک دیکشنری از تمامی کلمات یکتای موجود در اسناد تشکیل شده و هر کلمه (**Term**) به لیستی از شناسه‌های اسناد (**Document IDs**) که آن کلمه در آن‌ها رخ داده است (**Postings List**)، متصل می‌شود.

پیچیدگی زمانی: هنگام جستجو، سیستم نیازی به پیمایش متن اسناد ندارد؛ بلکه مستقیماً به دیکشنری مراجعه می‌کند. اگر از ساختار جدول درهم‌سازی (**Hash Table**) برای پیاده‌سازی دیکشنری استفاده شود، زمان جستجوی کلمه از مرتبه **O(1)** و در صورت استفاده از درخت‌های جستجو، از مرتبه **O(log K)** خواهد بود (که **K** تعداد کلمات یکتای دیکشنری است). در این حالت، هزینه زمانی اصلی تنها مربوط به واکنش و اشتراک‌گیری لیست‌های پیوندی (**Postings Lists**) است و جستجو کاملاً مستقل از حجم کل داده‌ها (**N**) انجام می‌پذیرد.

3. نقطه بحرانی در مقیاس داده‌ها (**Scalability Threshold**)

تفاوت این دو رویکرد از نظر پیچیدگی زمانی، وابستگی مستقیمی به مقیاس و حجم داده‌ها دارد:

مقیاس خرد و محلی (داده‌های در سطح مگابایت یا گیگابایت پایین): در این مقیاس (مانند جستجو در فایل‌های متنی یک سیستم شخصی یا سورس‌کدهای یک پروژه)، استفاده از جستجوی خطی کاملاً بهینه و منطقی است. در این حالت، سربار (**Overhead**) محاسباتی و حافظه‌ای لازم برای ایجاد و نگهداری ساختار نمایه وارونه توجیه فنی ندارد.

مقیاس کلان و سازمانی (داده‌های وب، ترابایت و فراتر از آن): با ورود به مقیاس کلان اطلاعاتی، تفاوت پیچیدگی **O(N)** در برابر **O(1)** از نظر عملیاتی حیاتی می‌شود. در حجم داده‌های وب یا پایگاه‌داده‌های عظیم، جستجوی خطی منجر به زمان‌های پاسخ‌گویی غیرقابل قبول (از چند ساعت تا چند روز) می‌شود و عملاً سیستم را ناکارآمد می‌سازد. در این نقطه، پردازش خطی از نظر محاسباتی غیرعملی (**Intractable**) تلقی شده و استفاده از نمایه وارونه از یک «انتخاب بهینه‌ساز» به یک «الزام معماری» برای پاسخ‌گویی درنگ‌درنگ (**Real-time**) تبدیل می‌گردد.

نتیجه‌گیری: تفاوت اصلی این دو روش در انتقال بار محاسباتی است؛ جستجوی خطی تمام هزینه را در زمان

جستجو (Query Time) پرداخت می‌کند، در حالی که نمایه وارونه، با استفاده از حافظه بیشتر، هزینه محاسباتی را

پاسخنامه مسئله 1.2: ماتریس تلاقی ترم-سند و تحلیل پدیده خلوت بودن (Sparsity)

1. تشکیل ماتریس تلاقی ترم-سند (Term-Document Incidence Matrix)

برای درک این ساختار، ابتدا فرض می‌کنیم سه سند کوتاه (D1 تا D3) با محتوای زیر در اختیار داریم و می‌خواهیم حضور یا عدم حضور 5 کلمه کلیدی مشخص را در آن‌ها بررسی کنیم:

سند D1: «مبانی بازیابی اطلاعات»

سند D2: «طراحی موتور جستجو»

سند D3: «الگوریتم‌های بازیابی وب»

کلمات کلیدی (Terms) انتخاب شده: بازیابی، اطلاعات، موتور، جستجو، وب.

ماتریس تلاقی به صورت یک ماتریس دودویی (Binary) تشکیل می‌شود که در آن سطرها نشان‌دهنده کلمات کلیدی، ستون‌ها نشان‌دهنده اسناد و درایه‌های ماتریس مقدار 1 (حضور کلمه در سند) یا 0 (عدم حضور) را می‌گیرند:

کلمه کلیدی (Term)	سند D1	سند D2	سند D3
بازیابی	1	0	1
اطلاعات	1	0	0
موتور	0	1	0
جستجو	0	1	0
وب	0	0	1

2. چالش ذخیره‌سازی در سیستم‌های واقعی و پدیده خلوت بودن (Sparsity)

در یک سیستم بازیابی اطلاعات واقعی (نظیر موتورهای جستجوی وب)، مقیاس این ماتریس به شدت بزرگ است. فرض کنید مجموعه‌ای متشکل از یک میلیون سند ($N = 1,000,000$) داریم و واژه‌نامه یکتای ما شامل 500,000 کلمه ($M = 500,000$) است.

اندازه ماتریس تلاقی برابر با $M \times N$ خواهد بود؛ یعنی به یک ماتریس با 5×10^{11} درایه (نصف یک تریلیون سلول) نیاز داریم. اگر هر درایه تنها 1 بیت حافظه اشغال کند، ذخیره این ماتریس به ده‌ها گیگابایت حافظه اصلی (RAM) یکپارچه نیاز دارد.

اما مشکل اصلی کجاست؟ **پدیده خلوت بودن (Sparsity)**.

هر سند متنی، معمولاً تنها شامل کسر بسیار کوچکی از کل واژگان موجود در زبان است (مثلاً یک سند 1000 کلمه‌ای، نهایتاً از 1000 کلمه یکتای واژه‌نامه 500,000 کلمه‌ای استفاده کرده است). این یعنی بیش از 99.9% از درایه‌های ماتریس تلاقی ترم-سند، مقدار 0 دارند. اختصاص دادن منابع عظیم حافظه برای ذخیره میلیاردها صفر، از نظر معماری سیستم و پیچیدگی فضایی، کاملاً غیرمنطقی و ناکارآمد است.

3. مدیریت Sparsity از طریق نمایه وارونه (Inverted Index)

برای حل مشکل خلوت بودن ماتریس، نمایه وارونه رویکردی هوشمندانه اتخاذ می‌کند: **تغییر از ذخیره‌سازی وضعیت به ذخیره‌سازی رخدادی**.

به جای ذخیره کل ماتریس (شامل تمامی صفرها و 1ها)، سیستم نمایه وارونه تنها درایه‌هایی که مقدار 1 دارند را ثبت می‌کند. این کار با استفاده از ساختار داده‌های لیست پیوندی (**Linked List**) یا آرایه‌های پویا انجام می‌شود. در این ساختار: هر کلمه (**Term**) به عنوان یک رکورد در دیکشنری ذخیره می‌شود و به جای یک سطر کامل از ماتریس، تنها یک **لیست رخداد (Postings List)** به آن متصل می‌گردد. این لیست صرفاً شامل شناسه اسنادی (**Doc IDs**) است که کلمه در آن‌ها وجود دارد.

نتیجه‌گیری ساختاری: استفاده از نمایه وارونه مشکل Sparsity را به طور کامل برطرف می‌کند، زیرا هیچ حافظه‌ای برای ثبت عدم حضور (صفرها) تخصیص داده نمی‌شود. این بهینه‌سازی، پیچیدگی فضایی (Space Complexity) سیستم را به طرز چشم‌گیری کاهش داده و امکان پیاده‌سازی سیستم‌های بازیابی در مقیاس وب را فراهم می‌آورد.

پاسخنامه مسئله 1.3: معماری و اجزای اصلی نمایه وارونه

مقدمه

نمایه وارونه (**Inverted Index**) به عنوان هسته مرکزی سیستم‌های بازیابی اطلاعات، از دو ساختار داده‌ی مجزا اما کاملاً وابسته به یکدیگر تشکیل شده است: واژه‌نامه (**Dictionary**) و لیست‌های رخداد (**Postings Lists**). تفکیک این دو بخش، امکان بهینه‌سازی جستجو از نظر پیچیدگی زمانی و مدیریت حافظه را فراهم می‌آورد. در ادامه، ماهیت و وظیفه دقیق هر بخش تشریح می‌گردد.

1. واژه‌نامه (Dictionary) یا (Lexicon)

واژه‌نامه یک ساختار داده جستجوپذیر (نظیر جدول درهم‌سازی، درخت جستجوی دودویی یا **B-Tree**) است که شامل تمامی کلمات یکتای استخراج‌شده از کل مجموعه اسناد (**Corpus**) است.

محتویات: این ساختار برای هر کلمه کلیدی (**Term**)، خود آن کلمه، فرکانس تکرار آن در کل اسناد (**Document Frequency**) و یک اشاره‌گر (**Pointer**) به لیست رخداد مربوط به آن کلمه را ذخیره می‌کند.

وظیفه در فرآیند جستجو: واژه‌نامه به عنوان «دروازه ورود» به سیستم عمل می‌کند. وظیفه اصلی آن، پاسخ‌گویی درنگ‌درنگ (**Real-time**) به این سوال است که: «آیا کلمه مورد جستجو اصلاً در اسناد ما وجود دارد یا خیر؟ و اگر وجود دارد، اطلاعات آن کجاست؟» از آنجا که سرعت در این مرحله حیاتی است، واژه‌نامه معمولاً به صورت کامل در حافظه اصلی (**RAM**) بارگذاری می‌شود تا زمان جستجوی کلمه در آن از مرتبه $O(1)$ یا $O(\log K)$ باشد.

2. لیست‌های رخداد (Postings Lists)

لیست‌های رخداد، مجموعه‌ای از آرایه‌های پویا یا لیست‌های پیوندی (**Linked Lists**) هستند. هر کلمه در واژه‌نامه، یک لیست رخداد اختصاصی برای خود دارد.

محتویات: هر لیست شامل شناسه‌های اسنادی (**Document IDs**) است که آن کلمه در آن‌ها مشاهده شده است. در سیستم‌های پیشرفته‌تر، علاوه بر شناسه سند، اطلاعاتی نظیر موقعیت دقیق کلمه در متن (**Position**) و میزان تکرار کلمه در

آن سند (**Term Frequency**) نیز در این لیست‌ها ذخیره می‌شود تا در فرآیند رتبه‌بندی (**Ranking**) استفاده گردد. این شناسه‌ها همواره به صورت صعودی مرتب (**Sorted**) می‌شوند.

وظیفه در فرآیند جستجو: لیست رخداد، حامل «پاسخ نهایی» است. وظیفه این بخش، ارائه دقیق آدرس تمامی اسنادی است که باید به عنوان نتیجه جستجو به کاربر نمایش داده شوند. به دلیل حجم بالای این لیست‌ها، معمولاً روی دیسک (**Hard Drive/SSD**) ذخیره شده و تنها در زمان نیاز، واکشی (**Fetch**) می‌شوند.

3. تشریح مرحله به مرحله فرآیند پیدا کردن یک کلمه

برای درک بهتر تعامل این دو جزء، فرآیند پردازش یک تقاضای جستجو (**Query**) را به صورت الگوریتمی بررسی می‌کنیم:

گام اول (جستجو در Dictionary): کاربر کلمه‌ای (مثلاً «الگوریتم») را جستجو می‌کند. سیستم ابتدا به واژه‌نامه (که در **RAM** قرار دارد) مراجعه کرده و کلمه «الگوریتم» را در میان کلیدها جستجو می‌کند.

گام دوم (ارزیابی وضعیت): اگر کلمه در واژه‌نامه یافت نشد، سیستم بلافاصله جستجو را خاتمه داده و اعلام می‌کند سندی یافت نشد. اگر کلمه پیدا شد، سیستم اشاره‌گر (**Pointer**) متصل به آن کلمه را می‌خواند.

گام سوم (مراجعه به Postings List): سیستم با استفاده از اشاره‌گر، به سراغ حافظه (دیسک) رفته و دقیقاً همان لیست رخداد (**Postings List**) مختص به کلمه «الگوریتم» را واکشی می‌کند.

گام چهارم (بازگرداندن نتایج): شناسه‌های اسناد موجود در آن لیست (مثلاً اسناد شماره 12، 45 و 89) خوانده شده و محتوای آن اسناد به عنوان نتیجه نهایی به کاربر ارجاع داده می‌شود.

نتیجه‌گیری: طراحی دومرحله‌ای نمایه وارونه باعث می‌شود تا سیستم نیازی به پیمایش متن اسناد نداشته باشد. دیکشنری وظیفه «مکان‌یابی سریع اطلاعات» و لیست رخداد وظیفه «ذخیره‌سازی جامع داده‌های بازیابی» را بر عهده دارد. ادغام این دو، پردازش کارآمد روی کلان‌داده‌ها را ممکن می‌سازد.

پاسخنامه مسئله 1.4: نقش پیش پردازش زبانی و توکنایزیشن در ساخت نمایه

مقدمه

در معماری سیستم‌های بازیابی اطلاعات، مرحله ساخت نمایه (**Index Construction**) نمی‌تواند مستقیماً روی جریان خام کاراکترها (**Raw Text**) اعمال شود. پیش از آنکه سیستم بتواند شناسه‌های اسناد را به لیست‌های رخداد (**Postings Lists**) اضافه کند، باید دقیقاً بداند که چه «موجودیت‌هایی» قرار است به عنوان کلید در واژه‌نامه (**Dictionary**) ذخیره شوند. اینجاست که فاز پیش‌پردازش شامل نشانه‌سازی (**Tokenization**) و ماژول‌های زبانی (**Linguistic Modules**) به عنوان یک لایه حیاتی وارد عمل می‌شود.

1. نشانه‌سازی یا تقطیع (Tokenization)

فرآیند نشانه‌سازی، جریان پیوسته‌ای از کاراکترها را به واحدهای معنادار و مجزا به نام نشانه (**Token**) یا کلمه تقسیم می‌کند. سیستم کامپیوتری به صورت پیش‌فرض درکی از کلمه ندارد و متن را صرفاً رشته‌ای از بایت‌ها می‌بیند.

ضرورت: بدون نشانه‌سازی، سیستم نمی‌داند یک کلمه کجا شروع و کجا پایان می‌یابد. جدا کردن کلمات بر اساس فاصله‌ها (Spaces) و علائم نگارشی (Punctuation) اولین قدم برای استخراج موجودیت‌های قابل جستجو است.

2. ماژول‌های پردازش زبانی (Linguistic Modules)

پس از استخراج نشانه‌ها، این کلمات هنوز برای ورود به واژه‌نامه استاندارد نیستند. ماژول‌های زبانی شامل فرآیندهایی نظیر حذف کلمات توقف (Stop-words Removal)، یکسان‌سازی حروف (Normalization) و ریشه‌یابی (Stemming / Lemmatization) هستند.

ضرورت: زبان‌های طبیعی پر از تنوع ظاهری برای یک مفهوم واحد هستند. هدف این ماژول‌ها، تبدیل کلمات مشتق‌شده و مختلف به یک «ریشه استاندارد» (Base Form) است. این کار دو مزیت بزرگ دارد:

1. کاهش حجم واژه‌نامه: جلوگیری از ذخیره ده‌ها حالت مختلف برای یک کلمه.
2. افزایش جامعیت (Recall): اطمینان از اینکه کاربر با جستجوی فعل مفرد، نتایج شامل افعال جمع یا مشتقات آن را نیز پیدا می‌کند.

3. بررسی یک مثال عملی: خروجی سیستم بدون فاز پیش‌پردازش

برای درک فاجعه‌ای که در غیاب این مراحل رخ می‌دهد، فرض کنید سند زیر را در سیستم داریم و کاربر قصد جستجوی عبارت «کتاب» را دارد:

متن سند: «او کتاب‌هایش را روی میز گذاشت.»

حالت الف: فقدان Tokenization و Linguistic Modules

اگر هیچ پیش‌پردازشی انجام نشود، سیستم کل این جمله را به عنوان یک رشته یکپارچه می‌بیند.

ورودی واژه‌نامه: [او کتاب‌هایش را روی میز گذاشت].

نتیجه جستجو: کاربر کلمه «کتاب» را جستجو می‌کند. چون سیستم کل جمله را به عنوان یک کلید واحد ثبت کرده است، هیچ تطابقی یافت نمی‌شود (Miss) و سند بازیابی نمی‌گردد.

حالت ب: انجام Tokenization اما فقدان Linguistic Modules

حال فرض کنید سیستم کلمات را با فاصله‌ها جدا کرده است، اما ماژول‌های زبانی (مانند ریشه‌یابی و حذف علائم) اعمال نشده‌اند.

ورودی‌های واژه‌نامه: [او]، [کتاب‌هایش]، [را]، [روی]، [میز]، [گذاشت].

نتیجه جستجو: مجدداً کاربر کلمه «کتاب» را جستجو می‌کند. سیستم به واژه‌نامه مراجعه کرده و به دنبال تطابق دقیق (Exact Match) می‌گردد. چون در واژه‌نامه تنها کلمه [کتاب‌هایش] وجود دارد، سیستم متوجه ارتباط معنایی این دو نمی‌شود و مجدداً سند بازیابی نمی‌گردد. (همچنین به نقطه متصل به کلمه [گذاشت] دقت کنید که به عنوان بخشی از کلمه ذخیره شده است).

نتیجه‌گیری ساختاری: عدم استفاده از نشانه‌سازی و ماژول‌های زبانی منجر به پدیده‌ای به نام «انفجار واژه‌نامه» (Dictionary Blowup) و «افت شدید جامعیت» (Low Recall) می‌شود. این مراحل پیش‌پردازشی، متون خام و

ساختارنیافته را به مجموعه‌ای از مفاهیم استاندارد و قابل ردیابی تبدیل می‌کنند تا ساخت نمایه وارونه عملاً توجیه و کارایی داشته باشد.

پاسخنامه مسئله 1.5: بهینه‌سازی پرسمان و ترتیب اجرای عملیات اشتراک (Query Optimization and Intersection Order)

مقدمه و بیان منطق بهینه‌سازی

در سیستم‌های بازیابی اطلاعات، عملگر منطقی **AND** (اشتراک) دارای خاصیت جابه‌جایی (**Commutative**) و شرکت‌پذیری (**Associative**) است. این یعنی ترتیب قرارگیری پرانتزها در پرسمان (**Web AND Data**) **AND** (**Mining AND Search**) از نظر منطقی تفاوتی در نتیجه نهایی ایجاد نمی‌کند و سیستم می‌تواند پرانتزها را نادیده گرفته و پرسمان را به صورت **Web AND Data AND Mining AND Search** پردازش کند.

هزینه محاسباتی (**Cost**) اشتراک‌گیری بین دو لیست رخداد با طول‌های **x** و **y**، در بدترین حالت از مرتبه $O(x + y)$ است. طول لیست رخداد حاصل از این اشتراک، هرگز نمی‌تواند از طول کوچک‌ترین لیست اولیه بزرگتر باشد؛ یعنی حداکثر برابر با $\min(x, y)$ خواهد بود.

قاعده طلایی در بهینه‌سازی پرسمان این است: «برای کمینه‌سازی هزینه کل پردازش، همواره اشتراک‌گیری را از کوچک‌ترین لیست‌های رخداد (کمترین Document Frequency) آغاز کنید.»

1. مرتب‌سازی لیست‌ها بر اساس فرکانس سند (DF)

در گام نخست، سیستم مقادیر **DF** (طول لیست رخداد) هر کلمه را از واژه‌نامه (**Dictionary**) واکشی کرده و آن‌ها را به صورت صعودی مرتب می‌کند:

1. 2. 3. 4. ترتیب بهینه برای پردازش به این شکل خواهد بود: (**Web AND Data AND Mining AND Search**)

2. اجرای مرحله‌به‌مرحله و تحلیل هزینه پردازش

اکنون عملیات اشتراک را بر اساس ترتیب بهینه اجرا کرده و هزینه هر مرحله را محاسبه می‌کنیم:

مرحله اول: اشتراک دو لیست کوچک‌تر

عملکرد: سیستم لیست‌های **Search** و **Mining** را واکشی و با هم اشتراک می‌گیرد.

هزینه پردازش این مرحله: $O(200 + 500) = O(700)$

خروجی: یک لیست میانی (**Temporary List**) ایجاد می‌شود. با توجه به اصل تقاطع، اندازه این لیست میانی حداکثر برابر با 200 خواهد بود (حالت بدبینانه). آن را **R₁** می‌نامیم.

مرحله دوم: ادغام لیست میانی با کلمه بعدی

عملکرد: سیستم لیست میانی **R₁** را با لیست کلمه **Data** اشتراک می‌گیرد.

هزینه پردازش این مرحله: $O(200 + 1000) = O(1200)$

خروجی: لیست میانی جدیدی به نام **R₂** ایجاد می‌شود که اندازه آن مجدداً حداکثر 200 خواهد بود.

مرحله سوم: ادغام نهایی با بزرگترین لیست

عملکرد: در نهایت، سیستم لیست میانی R_2 را با لیست عظیم **Web** اشتراک می‌گیرد.

هزینه پردازش این مرحله: $O(200 + 10000) = O(10200)$

خروجی: لیست نهایی اسناد (پاسخ پرسمان) به دست می‌آید.

3. نتیجه‌گیری و مقایسه هزینه (چرا این روش بهینه است؟)

مجموع هزینه پردازش در روش بهینه برابر است با: $700 + 1200 + 10200 = 12100$ عملیات.

سناریوی فاجعه‌بار (بدون بهینه‌سازی):

اگر سیستم بخواهد پرسمان را دقیقاً مطابق با پرانتزگذاری کاربر پردازش کند:

1. ابتدا **Web** و **Data** را اشتراک می‌گیرد: هزینه $O(10000 + 1000) = O(11000)$. لیست میانی حاصل حداکثر طول 1000

خواهد داشت.

2. سپس **Mining** و **Search** را اشتراک می‌گیرد: هزینه $O(500 + 200) = O(700)$. لیست میانی حاصل حداکثر طول 200

خواهد داشت.

3. در نهایت دو لیست میانی را اشتراک می‌گیرد: هزینه $O(1000 + 200) = O(1200)$.

مجموع هزینه: $11000 + 700 + 1200 = 12900$ عملیات.

نتیجه‌گیری ساختاری: در این مثال با وجود کوچک بودن اعداد، روش بهینه باعث صرفه‌جویی قابل توجهی شده

است. در سیستم‌های واقعی که فرکانس کلماتی مانند **Web** ممکن است در مقیاس میلیارد باشد، شروع عملیات از

کلمات عمومی و پرکاربرد منجر به ایجاد لیست‌های میانی غول‌پیکر و در نتیجه سرریز شدن حافظه (**Memory**)

(**Overflow**) و افت شدید سرعت سیستم می‌گردد. الگوریتم مرتب‌سازی بر اساس **DF** از بروز این مشکل جلوگیری

می‌کند.

پاسخنامه مسئله 1.6: پیامد عدم مرتب‌سازی لیست‌های ارجاع در الگوریتم ادغام (Consequences of

Unsorted Postings in Merge Algorithm)

مقدمه و پیش‌فرض‌های الگوریتم استاندارد

در قلب سیستم‌های بازیابی اطلاعات، کارایی خیره‌کننده عملگرهای منطقی (مانند **AND**) وام‌دار «الگوریتم استاندارد ادغام دو

اشاره‌گره» (**Standard Two-Pointer Merge**) است. پیش‌نیاز قطعی و حیاتی برای اجرای این الگوریتم با پیچیدگی زمانی

مطلوب $O(x + y)$ ، صعودی و مرتب بودن (**Sorted**) تمامی شناسه‌های سند (**DocID**) در لیست‌های رخداد (**Postings Lists**)

است.

اگر ساختار مرتب‌سازی در زمان ذخیره‌سازی داده‌ها رعایت نشود، منطق خطی این الگوریتم به طور کامل دچار فروپاشی ساختاری

(**Structural Breakdown**) می‌شود. در ادامه، رفتار دقیق سیستم در مواجهه با این خطای پیاده‌سازی بررسی می‌گردد.

1. منطق تصمیم‌گیری در الگوریتم دو اشاره‌گره

این الگوریتم بر اساس یک قانون ساده اما قدرتمند کار می‌کند: در هر مرحله، دو اشاره‌گر مقادیر فعلی خود را با هم مقایسه می‌کنند. اشاره‌گری که به **DocID** کوچکتر اشاره می‌کند، یک گام به جلو می‌رود (چون قطعاً مقدار کوچکتر در ادامه لیست دیگر که صعودی است یافت نخواهد شد). اگر مقادیر برابر بودند، یک اشتراک یافت شده است و هر دو اشاره‌گر پیش می‌روند.

2. رفتار الگوریتم در مواجهه با لیست‌های نامرتب (بروز پدیده False Negative)

وقتی لیست‌ها نامرتب باشند، پیش‌فرض بنیادین الگوریتم (یعنی عدم بازگشت به مقادیر کوچکتر در ادامه لیست) نقض می‌شود. در این حالت، الگوریتم به جای یافتن تمامی اشتراک‌ها، به دلیل تصمیم‌گیری‌های غلط در حرکت دادن اشاره‌گرها، بسیاری از شناسه‌های مشترک را نادیده گرفته و اصطلاحاً آن‌ها را «رد می‌کند» (**Skip**). نتیجه مستقیم این رفتار، افت شدید و غیرقابل پیش‌بینی **جامعیت (Recall)** سیستم است؛ یعنی سیستم اسنادی را که کاملاً مرتبط هستند و کلمات مورد نظر را دارند، به کاربر نشان نمی‌دهد (منجر به تولید خروجی‌های **False Negative** می‌شود).

3. اثبات رفتار الگوریتم با بررسی گام‌به‌گام (Trace)

برای درک دقیق این فروپاشی، دو لیست رخداد نامرتب را برای پرسمان **A AND B** در نظر بگیرید:

List A: [45, 12, 89]

List B: [12, 45, 70]

True Intersection: 12, 45

اکنون رفتار الگوریتم استاندارد را روی این دو لیست ردیابی می‌کنیم:

گام اول: اشاره‌گر **A** روی **45** و اشاره‌گر **B** روی **12** است. چون **45 > 12** است، اشاره‌گر **B** یک گام به جلو می‌رود.

گام دوم: اشاره‌گر **A** روی **45** و اشاره‌گر **B** روی **45** است. مقادیر برابرند. سند **45** به عنوان خروجی ثبت می‌شود. هر دو اشاره‌گر پیش می‌روند.

گام سوم: اشاره‌گر **A** روی **12** و اشاره‌گر **B** روی **70** است. چون **70 > 12** است، اشاره‌گر **A** یک گام به جلو می‌رود.

گام چهارم: اشاره‌گر **A** روی **89** و اشاره‌گر **B** روی **70** است. چون **89 > 70** است، اشاره‌گر **B** باید به جلو برود، اما لیست **B** به پایان رسیده است. الگوریتم متوقف می‌شود.

نتیجه اجرای الگوریتم: خروجی تولید شده تنها سند **[45]** است! الگوریتم به دلیل نامرتب بودن لیست، سند **12** را به طور کامل از دست داد.

4. پیامدهای سیستمی و تغییر هزینه پردازشی

وقتی لیست‌ها مرتب نباشند، امکان استفاده از الگوریتم بهینه دو اشاره‌گره از بین می‌رود و دانشجو مجبور به انتخاب یکی از دو رویکرد زیر است که هر دو فاجعه‌بار هستند:

استفاده از حلقه‌های تودرتو (Nested Loops): مقایسه هر عنصر از لیست **A** با تمام عناصر لیست **B**. پیچیدگی زمانی از حالت خطی **O(x+y)** به حالت مربعی **O(x*y)** تنزل می‌یابد که در لیست‌های میلیونی، باعث توقف کامل (**Hang**) سیستم می‌گردد.

استفاده از جدول درهم‌سازی (Hash Table): بارگذاری عناصر لیست کوچکتر در جدول درهم‌سازی. اگرچه زمان اجرا **O(x+y)**

باقی می‌ماند، اما سیستم متحمل یک پیچیدگی فضایی (**Space Complexity**) عظیم و غیرضروری از مرتبه $O(\min(x, y))$ می‌شود؛ در حالی که الگوریتم استاندارد برای اجرا نیازی به حافظه کمکی (**In-place**) ندارد.

نتیجه‌گیری: این فروپاشی خاموش، یکی از رایج‌ترین خطاهای پیاده‌سازی است. ارائه خروجی ناقص مشابه مثال بالا و درخواست از دانشجو برای عارضه‌یابی (**Troubleshooting**)، روشی بسیار دقیق برای سنجش درک عمیق آن‌ها از معماری **Inverted Index** است.

پاسخنامه مسئله 1.7: محاسبه تعداد مقایسه‌ها در الگوریتم ادغام

مقدمه و تنظیمات اولیه

برای یافتن اشتراک (**AND**) میان دو لیست رخداد، الگوریتم استاندارد از مکانیزم دو اشاره‌گر (**Two-Pointer**) استفاده می‌کند. فرض می‌کنیم اشاره‌گر **P_A** برای پیمایش لیست **A** و اشاره‌گر **P_B** برای پیمایش لیست **B** تخصیص یافته‌اند.

در نقطه شروع، هر دو اشاره‌گر روی اولین عنصر (اندیس صفر) لیست‌های خود قرار می‌گیرند. در هر گام، مقادیر تحت اشاره با یکدیگر مقایسه شده و سیستم بر اساس کوچکتر، بزرگتر یا مساوی بودن آن‌ها برای حرکت اشاره‌گرها تصمیم‌گیری می‌کند.

لیست **A**: [1, 3, 5, 7, 9, 11]

لیست **B**: [2, 3, 4, 5, 11, 15]

ردیابی گام‌به‌گام مسیر حرکت اشاره‌گرها

گام 1: **P_A** روی مقدار 1 و **P_B** روی مقدار 2 قرار دارد.

مقایسه: 1 کوچکتر از 2 است.

عملیات: اشاره‌گر **P_A** یک خانه به جلو می‌رود.

گام 2: **P_A** روی مقدار 3 و **P_B** روی مقدار 2 قرار دارد.

مقایسه: 3 بزرگتر از 2 است.

عملیات: اشاره‌گر **P_B** یک خانه به جلو می‌رود.

گام 3: **P_A** روی مقدار 3 و **P_B** روی مقدار 3 قرار دارد.

مقایسه: مقادیر برابرند.

عملیات: عدد 3 به عنوان اشتراک یافت‌شده در لیست خروجی ثبت می‌شود. هر دو اشاره‌گر **P_A** و **P_B** یک خانه پیش می‌روند.

گام 4: **P_A** روی مقدار 5 و **P_B** روی مقدار 4 قرار دارد.

مقایسه: 5 بزرگتر از 4 است.

عملیات: اشاره‌گر **P_B** یک خانه به جلو می‌رود.

گام 5: **P_A** روی مقدار 5 و **P_B** روی مقدار 5 قرار دارد.

مقایسه: مقادیر برابرند.

عملیات: عدد 5 در لیست خروجی ثبت می‌شود. هر دو اشاره‌گر **P_A** و **P_B** یک خانه پیش می‌روند.

گام 6: P_A روی مقدار 7 و P_B روی مقدار 11 قرار دارد.

مقایسه: 7 کوچکتر از 11 است.

عملیات: اشاره گر P_A یک خانه به جلو می‌رود.

گام 7: P_A روی مقدار 9 و P_B روی مقدار 11 قرار دارد.

مقایسه: 9 کوچکتر از 11 است.

عملیات: اشاره گر P_A یک خانه به جلو می‌رود.

گام 8: P_A روی مقدار 11 و P_B روی مقدار 11 قرار دارد.

مقایسه: مقادیر برابرند.

عملیات: عدد 11 در لیست خروجی ثبت می‌شود. هر دو اشاره گر P_A و P_B یک خانه پیش می‌روند.

شرط پایان و نتیجه‌گیری نهایی

پس از اجرای گام هشتم، اشاره گر P_A از مرز انتهای لیست A عبور می‌کند. از آنجا که شرط بقای حلقه (Loop Condition) در الگوریتم ادغام، نرسیدن اشاره‌گرها به انتهای هر دو لیست است، الگوریتم بلافاصله در همین نقطه متوقف می‌شود. در نتیجه، مقدار 15 در انتهای لیست B اساساً نیازی به بررسی پیدا نمی‌کند.

نتیجه‌گیری: تعداد دقیق مقایسه‌ها: سیستم برای تکمیل این پردازش دقیقاً به 8 عمل مقایسه (Comparison) نیاز داشت.

خروجی نهایی سیستم: لیست [3, 5, 11] به عنوان شناسه‌های مشترک بین دو سند بازگردانده می‌شود.

پاسخنامه مسئله 1.8: تحلیل پردازش پرسمان‌های منطقی و هزینه عملگر NOT (Analysis of Logical Query Processing and the Cost of NOT Operator)

مقدمه و نگاشت به جبر مجموعه‌ها (Set Algebra)

در مدل بازیابی بولی (Boolean Retrieval Model)، پرسمان‌های منطقی به طور مستقیم به عملگرهای نظریه مجموعه‌ها ترجمه می‌شوند. برای درک نحوه پردازش سیستم، ابتدا باید معادل ریاضی هر عملگر را مشخص کنیم:

عملگر OR معادل اجتماع ($\cup$) است.

عملگر AND معادل اشتراک ($\cap$) است.

عملگر NOT معادل متمم‌گیری نسبت به مجموعه مرجع (کل اسناد) است.

بنابراین، پرسمان $(A \text{ OR } B) \text{ AND NOT } C$ از نظر منطق ریاضی به شکل زیر فرمول‌بندی می‌شود:

$$(A \cup B) \cap C^c$$

که این رابطه در جبر مجموعه‌ها دقیقاً معادل عملگر تفاضل (Set Difference) است:

$$(A \cup B) - C$$

1. پردازش مرحله به مرحله روی لیست‌های ادعا (Postings Lists)

برای اجرای این پرسمان بدون تحمیل بار اضافی به حافظه، سیستم آن را در دو فاز و با استفاده از لیست‌های میانی (Temporary Lists) پردازش می‌کند:

گام اول (پردازش اجتماع - OR): سیستم ابتدا اشاره‌گرهایی برای لیست ادعای کلمه **A** و لیست ادعای کلمه **B** اختصاص می‌دهد. در الگوریتم ادغام اجتماع، هر شناسه‌ای (**DocID**) که در حداقل یکی از دو لیست وجود داشته باشد، خوانده شده و وارد یک لیست میانی (مثلاً **R₁**) می‌شود. در صورت وجود شناسه‌های تکراری (اشتراک **A** و **B**)، تنها یک نسخه از آن در **R₁** ثبت می‌گردد.

پیچیدگی زمانی: $O(|A| + |B|)$

طول لیست حاصل: حداکثر برابر با $|A| + |B|$

گام دوم (پردازش تفاضل - AND NOT):

اکنون سیستم لیست میانی **R₁** و لیست ادعای کلمه **C** را در اختیار دارد. دو اشاره‌گر در ابتدای این دو لیست قرار می‌گیرند. منطق حرکت در این مرحله، منطق «حذف» است:

اگر شناسه در **R₁** کوچکتر از **C** باشد، آن شناسه به عنوان نتیجه قطعی ثبت شده و اشاره‌گر **R₁** پیش می‌رود.

اگر شناسه‌ها برابر باشند، یعنی سند هم کلمات اولیه را دارد و هم شامل کلمه ممنوعه **C** است. پس این شناسه **نادیده گرفته شده** (حذف می‌شود) و هر دو اشاره‌گر پیش می‌روند.

اگر شناسه در **C** کوچکتر باشد، اشاره‌گر **C** صرفاً پیش می‌رود تا به شناسه‌های بزرگتر برسد.

2. تحلیل معماری: چرا هزینه پردازش NOT بسیار بالاتر از AND است؟

در موتورهای جستجو و سیستم‌های بازیابی مقیاس‌بزرگ، عملگر **NOT** یکی از پرهزینه‌ترین دستورات ارزیابی می‌شود که دلایل ساختاری آن به شرح زیر است:

الف) از دست رفتن پدیده خلوت بودن (Loss of Sparsity):

قدرت اصلی نمایه وارونه (**Inverted Index**) در استفاده از خاصیت **Sparsity** است؛ یعنی ما فقط اسنادی را پردازش می‌کنیم که یک کلمه در آن‌ها «وجود» دارد. عملگر **AND** ماهیتی **محدودکننده (Restrictive)** دارد و به سرعت طول لیست خروجی را کاهش می‌دهد. اما عملگر **NOT** (به تنهایی) ماهیتی **بسط‌دهنده (Expansive)** دارد. اگر سیستم بخواهد پرسمان **NOT C** را پردازش کند، باید لیست ادعای **C** را از مجموعه کل اسناد (**Universal Set** - که با **N** نمایش داده می‌شود) کم کند. این یعنی سیستم مجبور است یک لیست خروجی با ابعاد تقریبی $O(N)$ بسازد که در مجموعه‌های چند میلیارد سندی، منجر به سرریز کامل حافظه اصلی (**RAM**) و فروپاشی کارایی می‌گردد.

ب) تفاوت در شرایط توقف الگوریتم (Stopping Criteria):

در الگوریتم تقاطع استاندارد (**A AND B**)، به محض اینکه یکی از لیست‌ها به انتها برسد، الگوریتم به دلیل عدم امکان یافتن اشتراک جدید، فوراً متوقف می‌شود. اما در پردازش **A AND NOT C** (که بهینه‌ترین حالت استفاده از **NOT** است)، حتی اگر لیست **C** به پایان برسد، سیستم **نمی‌تواند متوقف شود**؛ بلکه مجبور است تمام شناسه‌های باقی‌مانده در انتهای لیست **A** را تا آخرین مورد به لیست خروجی اضافه کند. این پیمایش اجباری تا انتهای لیست پایه، هزینه پردازشی زمان اجرا (**Query Time**) را در مقایسه با **AND** به شدت افزایش می‌دهد.

نتیجه‌گیری:

به دلیل همین هزینه‌های ساختاری، موتورهای جستجوی مدرن اجرای عملگر **NOT** را به صورت مستقل (**Unary NOT**) ممنوع کرده یا نادیده می‌گیرند و تنها آن را در ترکیب با عملگرهای محدودکننده‌ای نظیر **AND** (به شکل فیلتر حذفی) مجاز می‌دانند.

پاسخنامه مسئله 1.9: ارتقای نمایه برای پشتیبانی از پرسمان‌های عبارتی (Upgrading Index to

Support Phrasal Queries)

مقدمه: محدودیت مدل استاندارد (Bag of Words)

در نمایه‌سازی استاندارد که تنها شناسه‌های سند (**DocID**) ذخیره می‌شوند، سیستم از مدل «کیسه کلمات» (**Bag of Words**) پیروی می‌کند. در این مدل، سیستم تنها از «حضور» یا «عدم حضور» یک کلمه در سند مطلع است و هیچ درکی از **ترتیب**، **مجاورت** یا **ساختار جملات** ندارد. در نتیجه، اگر کاربری عبارت دقیق "**Machine Learning**" را جستجو کند، سیستم استاندارد آن را به عنوان **Machine AND Learning** پردازش کرده و سندی با متن "**Learning to operate this Machine**" را نیز به عنوان یک پاسخ کاملاً مرتبط برمی‌گرداند (یک خطای **False Positive** بزرگ). برای حل این مشکل، معماری سیستم باید از سطح سند به سطح موقعیت کلمات ارتقا یابد.

1. تغییر ساختار ذخیره‌سازی: معرفی نمایه موقعیتی (Positional Index)

برای پشتیبانی از پرسمان‌های عبارتی، ساختار لیست‌های رخداد (**Postings Lists**) باید تغییر بنیادی کند. به جای تخصیص یک لیست ساده از شناسه‌ها به هر کلمه، سیستم باید **موقعیت (Position)** دقیق هر بار تکرار آن کلمه در متن سند را نیز ذخیره کند.

در این ساختار جدید (نمایه موقعیتی)، هر درایه از لیست رخداد، دیگر یک عدد منفرد (**DocID**) نیست؛ بلکه یک ساختار داده ترکیبی شامل شناسه سند و لیستی از موقعیت‌هاست:

مثال عملی:

فرض کنید کلمه "**Machine**" در سند شماره 12، کلمات دهم و پنجاهم متن است. کلمه "**Learning**" نیز در همان سند، کلمات یازدهم و نودم متن است. لیست رخداد در نمایه موقعیتی به این شکل ذخیره می‌شود:

Machine $\rightarrow [\dots, \text{DocID}_{12} : \langle 10, 50 \rangle, \dots]$

Learning $\rightarrow [\dots, \text{DocID}_{12} : \langle 11, 90 \rangle, \dots]$

2. الگوریتم پردازش پرسمان (اشتراک موقعیتی - Positional Intersect)

زمان اجرای پرسمان، سیستم دیگر نمی‌تواند فقط به یافتن اسناد مشترک اکتفا کند، بلکه باید یک «الگوریتم اشتراک موقعیتی» را اجرا نماید:

گام اول (یافتن اسناد مشترک): ابتدا مانند یک پرسمان **AND** معمولی، با استفاده از دو اشاره‌گر، اسنادی که هر دو کلمه "Machine" و "Learning" را دارند پیدا می‌کند (در اینجا $DocID_{12}$).

گام دوم (بررسی مجاورت): روی سند مشترک، سیستم موقعیت‌های کلمه اول را با کلمه دوم مقایسه می‌کند. شرط تطابق برای پرسمان عبارتی این است که موقعیت کلمه دوم، دقیقاً یک واحد بیشتر از کلمه اول باشد:

$$Pos(Term_2) - Pos(Term_1) = 1$$

گام سوم (ارزیابی مقادیر): سیستم جفت (11, 10) را بررسی می‌کند: $11 - 10 = 1$. تطابق یافت شد!

سپس جفت (90, 50) را بررسی می‌کند که مجاور نیستند.

به دلیل یافتن حداقل یک تطابق مجاور، سند 12 به عنوان پاسخ معتبر برای پرسمان عبارتی تایید و به کاربر بازگردانده می‌شود.

3. تحلیل پیچیدگی و هزینه‌های سیستمی این ارتقا

تبدیل نمایه استاندارد به نمایه موقعیتی، هزینه‌های معماری قابل توجهی به همراه دارد که تحلیل آن برای طراحی سیستم‌های مقیاس‌بزرگ حیاتی است:

پیچیدگی فضایی (Space Complexity): نمایه موقعیتی به شدت حجیم‌تر است. از آنجا که هر بار تکرار یک کلمه نیازمند ثبت موقعیت است، حجم نمایه وارونه بین 2 تا 4 برابر افزایش می‌یابد و بسته به زبان و ساختار داده‌ها، می‌تواند به 30 تا 50 درصد از حجم کل متن اصلی (**Corpus**) برسد. استفاده از تکنیک‌های فشرده‌سازی (مانند گاما کدینگ برای ذخیره فاصله بین موقعیت‌ها به جای عدد مطلق) در اینجا الزامی است.

پیچیدگی زمانی (Time Complexity): الگوریتم ادغام (**Merge**) دیگر محدود به طول لیست اسناد نیست. در بدترین حالت، زمان اجرای مقایسه‌ها وابسته به تعداد کل رخدادهای کلمات (فرکانس کلمه در کل متون یا **Collection Frequency**) خواهد بود.

4. رویکرد جایگزین (معماری نمایه دوکلمه‌ای - Bi-word Index)

قابل ذکر است که روش جایگزینی به نام **نمایه دوکلمه‌ای** وجود دارد که در آن جفت کلمات متوالی (مانند "Machine Learning") مستقیماً به عنوان یک کلید مستقل در واژه‌نامه (**Dictionary**) ثبت می‌شوند. این روش زمان اجرای پرسمان را به **O(1)** کاهش می‌دهد، اما منجر به پدیده «انفجار واژه‌نامه» (**Dictionary Blowup**) می‌شود؛ زیرا تعداد ترکیب‌های دوکلمه‌ای ممکن در یک زبان بی‌نهایت است.

نتیجه‌گیری ساختاری:

به همین دلیل، موتورهای جستجوی مدرن همواره از **نمایه موقعیتی (Positional Index)** به عنوان رویکرد استاندارد و مقیاس‌پذیر استفاده می‌کنند.

پاسخنامه مسئله 1.10: تحلیل کارایی صف اولویت در بهینه‌سازی پرسمان‌های OR

مقدمه و طرح مسئله

در سیستم‌های بازیابی اطلاعات، پردازش پرسمان‌هایی که شامل چندین عملگر **OR** متوالی هستند (مانند **A OR B OR C OR D**)، به معنای یافتن اجتماع (**Union**) چندین لیست رخداد (**Postings List**) است. به این مسئله در علوم کامپیوتر، **ادغام چندگانه (K-way Merge)** گفته می‌شود که در آن **K** تعداد لیست‌هاست. پیشنهاد تیم دانشجویی مبنی بر استفاده از صف اولویت (**Priority Queue**) برای حل این مسئله، یک رویکرد کاملاً مهندسی‌شده و یکی از راهکارهای استاندارد در طراحی موتورهای جستجو است. در ادامه، صحت و برتری این الگوریتم را از منظر تحلیل پیچیدگی زمانی اثبات می‌کنیم.

1. تحلیل رویکرد سنتی: ادغام دو به دو (Pairwise Merge)

برای درک ارزش پیشنهاد دانشجویان، ابتدا باید نقص روش سنتی را بررسی کنیم. فرض کنید **K** لیست رخداد داریم که میانگین طول هر کدام **N** است (مجموعاً $T = K \times N$ عنصر). اگر سیستم بخواهد لیست‌ها را به صورت متوالی و دو به دو ادغام کند:

ابتدا لیست اول و دوم را ادغام می‌کند: هزینه $O(2N)$

سپس حاصل را با لیست سوم ادغام می‌کند: هزینه $O(3N)$

این روند تا لیست **K**-ام ادامه می‌یابد: هزینه $O(KN)$

مجموع پیچیدگی زمانی این رویکرد به صورت یک تصاعد حسابی رشد کرده و از مرتبه $O(K^2 N)$ خواهد بود. در پرسمان‌هایی با تعداد متغیرهای بالا، این توان دوم (K^2) باعث افت شدید کارایی زمان اجرا (**Query Time**) می‌شود.

2. تشریح الگوریتم پیشنهادی دانشجویان: استفاده از صف اولویت (Min-Heap)

پیشنهاد استفاده از صف اولویت (که معمولاً با ساختار داده **Min-Heap** پیاده‌سازی می‌شود)، رویکرد حل مسئله را از حالت «دو به دو» به حالت «همزمان» تغییر می‌دهد. نحوه کارکرد این الگوریتم به شرح زیر است:

گام راه‌اندازی (Initialization): سیستم یک اشاره‌گر به ابتدای هر یک از **K** لیست اختصاص می‌دهد. سپس عنصر اول هر لیست (همراه با شناسه نشان‌دهنده این که عنصر متعلق به کدام لیست است) وارد یک **Min-Heap** می‌شود. از آنجا که درخت **Min-Heap** همواره کوچکترین مقدار را در ریشه خود نگه می‌دارد، کوچکترین شناسه سند (**DocID**) در کل لیست‌ها، بلافاصله در ریشه درخت قرار می‌گیرد. اندازه این صف همواره دقیقاً برابر با **K** است.

گام استخراج و پیشروی (Extract and Advance):

- سیستم عنصر ریشه (کوچکترین **DocID**) را استخراج کرده و به لیست خروجی نهایی (پاسخ پرسمان) اضافه می‌کند.
- اشاره‌گر لیستی که این عنصر از آن استخراج شده، یک خانه به جلو می‌رود.
- سیستم عنصر جدید آن لیست را وارد **Min-Heap** می‌کند. (اگر لیست به انتها رسیده باشد، عنصری اضافه نمی‌شود و اندازه صف کاهش می‌یابد).
- درخت **Min-Heap** خود را مرتب می‌کند (**Heapify**) تا کوچکترین عنصر جدید به ریشه بیاید.

گام تکرار: این فرآیند تا زمانی که صف اولویت کاملاً خالی شود (یعنی تمام لیست‌ها به انتها برسند) تکرار می‌گردد. برای مدیریت شناسه‌های تکراری (اشتراک‌ها در **OR**)، سیستم به سادگی چک می‌کند که اگر عنصر استخراج‌شده از ریشه برابر با آخرین عنصر اضافه‌شده به لیست خروجی بود، آن را نادیده بگیرد.

3. تحلیل پیچیدگی زمانی رویکرد صف اولویت (Min-Heap)

اکنون کارایی این پیشنهاد را با نماد **Big O** تحلیل می‌کنیم:

ساخت صف اولویت اولیه با K عنصر، نیازمند زمان $O(K)$ است. در مجموع، تمام عناصر موجود در تمامی لیست‌ها (که تعداد آن‌ها T است) دقیقاً یک بار وارد صف شده و یک بار از آن خارج می‌شوند.

در ساختار **Min-Heap**، هزینه هر بار درج (**Insert**) یا استخراج و مرتب‌سازی (**Extract-Min**)، وابسته به ارتفاع درخت است که با توجه به اندازه K -تایی صف، برابر با $O(\log K)$ خواهد بود.

از آنجا که ما T عنصر داریم و برای هر کدام عملیاتی از مرتبه $O(\log K)$ انجام می‌دهیم، زمان کل پردازش گام‌های تکرار شونده برابر با $O(T \log K)$ است.

بنابراین پیچیدگی زمانی کل این رویکرد برابر است با:

$$O(K) + O(T \log K) = O(T \log K)$$

4. نتیجه‌گیری و ارزیابی فنی:

پیشنهاد این تیم دانشجویی کاملاً صحیح و بسیار بهینه است.

با مقایسه پیچیدگی زمانی رویکرد سنتی $O(K^2 N)$ و رویکرد صف اولویت $O(K N \log K)$ (با جایگذاری $T = KN$)، مشخص می‌شود که جایگزینی ضرب K با $\log K$ ، یک بهبود دراماتیک در مقیاس‌پذیری ایجاد می‌کند. در سیستم‌های واقعی بازیابی وب که یک پرسمان ممکن است شامل ده‌ها مترادف و بسط کوئری (**Query Expansion**) با عملگر **OR** باشد، استفاده از صف اولویت نه تنها یک پیشنهاد خوب، بلکه استاندارد صنعتی برای جلوگیری از گلوگاه‌های محاسباتی (**Computational Bottlenecks**) است.

پاسخنامه مسئله 1.11: پیاده‌سازی و تحلیل سربار عملگرهای مجاورت (Implementation and

Overhead Analysis of Proximity Operators)

مقدمه

مدل بولین استاندارد، به دلیل ماهیت صفر و یکی خود (حضور یا عدم حضور یک کلمه در سند)، توانایی درک فاصله و ارتباط معنایی کلمات مجاور را ندارد. برای رفع این محدودیت، «مدل بولین توسعه‌یافته» (**Extended Boolean Model**) معرفی می‌شود که با استفاده از عملگرهای مجاورت (مانند **NEAR** یا **WITHIN**)، امکان جستجوی مبتنی بر فاصله کلمات را فراهم می‌کند. این قابلیت به ویژه برای یافتن اسامی خاص یا مفاهیم ترکیبی حیاتی است.

1. نگارش پرسمان (Query Formulation)

در سینتکس استاندارد سیستم‌های بازیابی اطلاعات، عملگر مجاورت معمولاً با کلمه کلیدی **NEAR** (مخفف **N**) یا **W** (مخفف **Within**) به همراه یک عدد نشان داده می‌شود که حداکثر تعداد کلمات مجاز بین دو عبارت را مشخص می‌کند.

این پرسمان به سیستم دستور می‌دهد: «اسنادی را بازبایی کن که در آن‌ها کلمات **Daneshgah** و **Tehran** حضور داشته باشند، به شرطی که حداکثر 2 کلمه دیگر بتواند بین آن‌ها قرار گیرد.»

2. منطق محاسباتی و شرط تطابق موقعیت‌ها

برای اجرای این پرسمان، سیستم نیازمند استفاده از «نمایه موقعیتی» (**Positional Index**) است. پس از یافتن اسنادی که هر دو کلمه را دارند، سیستم موقعیت (**Position**) آن‌ها را در متن مقایسه می‌کند.

اگر موقعیت کلمه اول P_1 و موقعیت کلمه دوم P_2 باشد و حداکثر کلمات مجاز بین آن‌ها k در نظر گرفته شود (در این مسئله $k = 2$)، شرط تطابق به صورت ریاضی این‌گونه تعریف می‌شود:

$$|P_1 - P_2| \leq k + 1$$

بنابراین برای این مسئله، اختلاف موقعیت دو کلمه باید کوچکتر یا مساوی 3 باشد. (به عنوان مثال، اگر **Daneshgah** کلمه دهم و **Tehran** کلمه سیزدهم باشد، $3 \leq 13 - 10$)، شرط برقرار است و دقیقاً 2 کلمه بین آن‌ها فاصله وجود دارد).

3. تحلیل سربار پردازشی در زمان اجرا (Query-Time Overhead)

پشتیبانی از عملگرهای مجاورت، بهای سنگینی برای معماری سیستم دارد و سربارهای (**Overheads**) زیر را در زمان اجرای پرسمان تحمیل می‌کند:

الف) سربار سنگین ورودی/خروجی (I/O Overhead):

همان‌طور که می‌دانیم، لیست‌های رخداد موقعیتی (**Positional Postings Lists**) بسیار حجیم‌تر از لیست‌های استاندارد هستند. وقتی کاربر از عملگر **NEAR** استفاده می‌کند، سیستم باید حجم عظیمی از داده‌های موقعیتی را از دیسک (**Disk**) خوانده و به حافظه اصلی (**RAM**) منتقل کند. این انتقال داده، گلوگاه (**Bottleneck**) اصلی در سرعت پاسخ‌گویی است.

ب) پیچیدگی زمانی و سربار محاسباتی (Computational Overhead):

در مدل بولین استاندارد، یافتن اشتراک دو سند نیازمند یک مقایسه ساده روی شناسه‌های سند (**DocID**) است. اما با عملگر مجاورت، سیستم باید یک «حلقه تودرتو» (**Nested Loop**) روی موقعیت‌های کلمات اجرا کند.

فرض کنید کلمه **A** در یک سند خاص tf_A بار تکرار شده و کلمه **B** در همان سند tf_B بار تکرار شده است. الگوریتم اشتراک موقعیتی برای بررسی شرط فاصله، باید این دو لیست از موقعیت‌ها را پیمایش کند که پیچیدگی زمانی آن برای هر تک‌سند مشترک، از مرتبه $O(tf_A + tf_B)$ است.

هنگامی که این محاسبه برای هزاران یا میلیون‌ها سند مشترک ضرب شود، بار محاسباتی عظیمی (**CPU Cycles**) به پردازنده‌های موتور جستجو تحمیل می‌گردد.

نتیجه‌گیری ساختاری:

به دلیل همین سربارهای نمایی در زمان اجرا، موتورهای جستجوی مقیاس‌بزرگ، جستجوی مجاورتی را به عنوان پیش‌فرض اعمال نمی‌کنند، بلکه آن را به عنوان یک فاز ثانویه (پس از فیلتر کردن اسناد کاندیدا با عملگر ساده AND) و تنها روی زیرمجموعه کوچکی از اسناد با بالاترین امتیاز (Top-K Documents) اجرا می‌کنند تا از افت کارایی جلوگیری شود.

پاسخنامه مسئله 1.12: محدودیت‌های تطابق دقیق در بازیابی بولین کلاسیک (Exact Match)

Limitations in Classic Boolean Retrieval

مقدمه

مدل بازیابی بولین کلاسیک (Classic Boolean Model)، نخستین و پایه‌ای‌ترین مدل در سیستم‌های بازیابی اطلاعات است. این مدل، با وجود سادگی و سرعت بالا در پردازش، از یک محدودیت ذاتی و بنیادین رنج می‌برد: ارزیابی میزان ارتباط اسناد به صورت قطعی و دودویی (باینری) انجام می‌شود و هیچ‌گونه درکی از «درجه ارتباط» (Degree of Relevance) یا وزن‌دهی به کلمات وجود ندارد.

1. چرا ارزیابی به صورت صفر و یک است؟

علت این رویکرد به بنیان‌های ریاضی این مدل بازمی‌گردد. مدل بولین مستقیماً بر پایه جبر بول (Boolean Algebra) و نظریه مجموعه‌ها (Set Theory) بنا شده است.

در این چارچوب ریاضی، جهان به صورت قطعی تعریف می‌شود: یک کلمه یا در یک سند وجود دارد (مقدار 1 یا True) یا وجود ندارد (مقدار 0 یا False). بر همین اساس، تابع ارزیابی سیستم، تطابق پرسمان کاربر با یک سند را صرفاً به عنوان یک معادله منطقی حل می‌کند که خروجی آن تنها می‌تواند 0 (سند نامرتب - عدم بازیابی) یا 1 (سند مرتبط - بازیابی قطعی) باشد. به این مکانیزم سخت‌گیرانه، تطابق دقیق (Exact Match) گفته می‌شود، زیرا سیستم تنها به دنبال کلماتی می‌گردد که دقیقاً با کاراکترهای پرسمان یکسان باشند.

2. بررسی یک سناریوی عملی: شکست منطقی مدل بولین

برای درک ضعف این رویکرد صفر و یکی، سناریوی پرکاربرد زیر را در نظر بگیرید:

نیاز اطلاعاتی کاربر: کاربری به دنبال یافتن یک مقاله جامع درباره «بهینه‌سازی الگوریتم‌ها در زبان پایتون» است.

پرسمان ثبت‌شده: کاربر برای رسیدن به دقیق‌ترین نتیجه، از عملگر محدودکننده AND استفاده کرده و پرسمان زیر را وارد می‌کند:

Python AND Algorithm AND Optimization

وضعیت سند هدف (Target Document): یک مقاله مرجع، بسیار غنی و دقیقاً مرتبط با نیاز کاربر در پایگاه داده وجود دارد. در این مقاله کلمه Python بیش از 100 بار و کلمه Algorithm بیش از 50 بار تکرار شده است. اما نویسنده مقاله، بر اساس سلیقه نگارشی خود، به جای استفاده از کلمه Optimization، در سراسر متن از کلمات مترادفی نظیر Efficiency یا Performance استفاده کرده است.

تحلیل رفتار سیستم (رخداد خطای منطقی):

هنگام اجرای الگوریتم، سیستم مدل بولین وضعیت این سند را به این شکل محاسبه می‌کند:

وجود Python در سند = 1 (True)

وجود Algorithm در سند = 1 (True)

وجود Optimization در سند = 0 (False)

$$1 \text{ AND } 1 \text{ AND } 0 = 0$$

3. پیامدهای ساختاری این رویکرد

همان‌طور که در سناریوی بالا مشاهده شد، خروجی نهایی برای این مقاله مرجع دقیقاً 0 خواهد بود و سیستم آن را به طور کامل از لیست نتایج حذف می‌کند (بروز خطای منفی کاذب یا **False Negative**).

مشکل اصلی اینجاست که در چشم‌انداز مدل بولین کلاسیک، بین این مقاله مرجع (که 99% با نیاز کاربر هم‌پوشانی معنایی دارد اما یک کلمه را ندارد) و یک مقاله کاملاً بی‌ربط درباره «دستور پخت پیتزا» (که هیچ‌کدام از کلمات را ندارد)، هیچ تفاوتی وجود ندارد؛ زیرا هر دو در نهایت امتیاز صفر می‌گیرند.

پیامدهای نهایی: این رفتار صفر و یکی منجر به دو پدیده مخرب در سیستم‌های بازیابی می‌شود:

1. **عدم امکان رتبه‌بندی (No Ranking):** تمام اسنادی که خروجی 1 می‌گیرند، در یک سطح قرار دارند. سیستم نمی‌تواند تشخیص دهد کدام سند مرتبط‌تر است (مثلاً سندی که کلمه را 1 بار دارد با سندی که آن را 100 بار دارد، یکسان ارزیابی می‌شود).

2. **قحطی یا ضیافت (Feast or Famine):** پرسرمان‌های کاربر معمولاً یا به قدری سخت‌گیرانه هستند (مانند استفاده زیاد از **AND**) که هیچ سندی بازیابی نمی‌شود (قحطی)، و یا به قدری گشاده‌دستانه هستند (استفاده از **OR**) که میلیون‌ها سند نامرتب به کاربر نمایش داده می‌شود (ضیافت)، که در هر دو حالت کاربر به نتیجه مطلوب نمی‌رسد.

پاسخنامه مسئله 1.13: توزیع نمایه وارونه و مقایسه معماری‌های پارتیشن‌بندی

مقدمه: ضرورت توزیع‌سازی (Distributed Indexing)

زمانی که حجم اسناد (**Corpus**) در سیستم‌های بازیابی اطلاعات به مقیاس وب می‌رسد، ذخیره‌سازی نمایه وارونه روی یک سرور منفرد به دلیل محدودیت‌های حافظه اصلی (**RAM**) و فضای ذخیره‌سازی غیرممکن می‌گردد. در این شرایط، پایگاه داده باید تکه‌تکه شده و روی خوشه‌ای از سرورها (**Cluster**) توزیع شود که به این عمل **Sharding** می‌گویند. برای این کار دو معماری بنیادین وجود دارد که رفتار سیستم را در زمان اجرای پرسرمان به کلی تغییر می‌دهند.

بخش 1) رویکرد تقسیم بر اساس اسناد (Document-Partitioning / Local Index)

در این معماری، مجموعه کل اسناد به بخش‌های مساوی تقسیم شده و هر بخش به یک سرور مجزا (**Node**) اختصاص می‌یابد.

ساختار: هر سرور یک نمایه وارونه محلی (**Local Index**) می‌سازد که صرفاً شامل کلمات موجود در اسناد اختصاص‌یافته به خودش است.

نحوه اجرای پرسرمان A AND B: وقتی کاربر این پرسرمان را ثبت می‌کند، سرور مرکزی (**Broker** یا **Aggregator**) این درخواست را به طور همزمان (**Broadcast**) برای تمامی سرورهای خوشه ارسال می‌کند. هر سرور به صورت مستقل و محلی، اشتراک لیست **A** و **B** را در نمایه خود محاسبه کرده و بهترین نتایج خود را (**Top-K**) به سرور مرکزی بازمی‌گرداند. در نهایت، سرور مرکزی این نتایج را ترکیب و مرتب کرده و به کاربر نشان می‌دهد.

بخش 2) رویکرد تقسیم بر اساس کلمات (Term-Partitioning / Global Index)

در این معماری، واژه‌نامه (**Dictionary**) قطعه‌قطعه می‌شود. به عنوان مثال، تمامی کلماتی که با حروف **A** تا **M** شروع می‌شوند به سرور اول و حروف **N** تا **Z** به سرور دوم سپرده می‌شوند.

ساختار: هر سرور دارای یک نمایه سراسری (**Global Index**) است؛ به این معنا که لیست رخداد (**Postings List**) یک کلمه در یک سرور، حاوی شناسه تمامی اسنادی است که آن کلمه در آن‌ها وجود دارد (از کل اسناد سیستم).

نحوه اجرای پرسمان A AND B: سرور مرکزی بررسی می‌کند که کلمه **A** در کدام سرور و کلمه **B** در کدام سرور ذخیره شده است. اگر **A** در سرور شماره 1 و **B** در سرور شماره 2 باشد، یکی از این دو سرور مجبور است **کل لیست رخداد خود را از طریق شبکه** برای سرور دیگر (یا برای سرور مرکزی) ارسال کند تا عملگر اشتراک (**Intersection**) بتواند روی دو لیست انجام شود.

بخش 3) مقایسه درگیری شبکه و تحلیل گلوگاه (Bottleneck Analysis)

تفاوت اصلی این دو رویکرد در زمان اجرای پرسمان‌های چندکلمه‌ای (مانند **A AND B**) به شکل چشم‌گیری نمایان می‌شود:

گلوگاه در روش Term-Partitioning (فاجعه شبکه):

در این روش، **شبکه (Network Bandwidth)** به گلوگاه اصلی سیستم تبدیل می‌شود. لیست‌های رخداد برای کلمات پرکاربرد می‌توانند شامل صدها میلیون شناسه سند باشند (چندین گیگابایت داده). انتقال این حجم عظیم از داده‌ها بین سرورها در هر بار جستجو، پهنای باند شبکه داخلی را به سرعت اشغال کرده و زمان پاسخ‌گویی (**Latency**) را به شدت افزایش می‌دهد. در اینجا سیستم به جای جابه‌جا کردن «کد پردازی»، مجبور به جابه‌جا کردن «داده‌های عظیم» است که در معماری توزیع‌یافته یک خطای طراحی محسوب می‌شود.

گلوگاه در روش Document-Partitioning (بهینگی شبکه): در این روش، درگیری شبکه بسیار ناچیز است. سرور مرکزی تنها یک رشته متنی کوتاه (**A AND B**) را به سرورها می‌فرستد و هر سرور تنها حجم کوچکی از نتایج نهایی (مثلاً **10** سند برتر) را برمی‌گرداند. لیست‌های عظیم رخداد هرگز در شبکه جابه‌جا نمی‌شوند و پردازش دقیقاً در همان جایی انجام می‌شود که داده‌ها قرار دارند.

با این حال، گلوگاه این روش **سرور مرکزی (Broker)** است؛ زیرا با افزایش تعداد سرورها (مثلاً به ده‌ها هزار نود)، ارسال درخواست برای همه آن‌ها و ادغام همزمان نتایج می‌تواند باعث سربار پردازشی روی **Broker** شود.

نتیجه‌گیری ساختاری: به دلیل هزینه غیرقابل جبران انتقال لیست‌های رخداد در شبکه (در روش

Term-Partitioning)، تقریباً تمامی موتورهای جستجوی مدرن و مقیاس‌پذیر (نظیر **Google** و **ElasticSearch**) از معماری **Document-Partitioning** استفاده می‌کنند. در این سیستم‌ها، مدیریت گلوگاه سرور مرکزی با استفاده از ساختار درختی (**Tree-based Aggregation**) و مسیربای‌های توزیع‌شده به راحتی قابل حل است.

پاسخنامه مسئله 1.14: مثال نقض و تحلیل سربار در اشاره‌گرهای پرشی (**Counterexample and**

Overhead Analysis in Skip Pointers)

1. بیان منطق الگوریتم و ریشه سربار (**Overhead**)

در الگوریتم ادغام با پرشگر، زمانی که اشاره‌گر لیست اول **P_A** به مقداری کوچکتر از لیست دوم **P_B** اشاره می‌کند، سیستم پیش از اینکه **P_A** را یک خانه به جلو ببرد، یک بررسی اضافه انجام می‌دهد:

شرط پرش: «آیا در این خانه پرشگر وجود دارد؟ و آیا مقصدی که پرشگر به آن اشاره می‌کند، همچنان کوچکتر یا مساوی با مقدار فعلی در P_B است؟»

اگر پاسخ به این شرط منطقی **False** باشد (یعنی مقدار مقصد بزرگتر از P_B باشد)، پرش لغو می‌شود و اشاره‌گر به صورت عادی یک خانه جلو می‌رود. اما سیستم برای بررسی این شرط، **یک مقایسه عددی اضافی** را به پردازنده تحمیل کرده است. اگر این اتفاق مدام تکرار شود، کارایی به شدت افت می‌کند.

2. طراحی مثال نقض عددی

برای ایجاد این نقطه ضعف، باید دو لیست با شناسه‌های به شدت متراکم (**Dense**) و درهم‌تنیده (**Interleaved**) طراحی کنیم.

لیست **A: [1, 3, 5, 7, 9, 11]** (طول لیست 6 است)

لیست **B: [2, 4, 6, 8, 10, 12]**

تنظیمات پرشگر برای لیست A:

طبق فرمول استاندارد، طول گام پرش برابر با جذر طول لیست (یعنی $2 \approx \sqrt{6}$) در نظر گرفته می‌شود. بنابراین در لیست **A**، هر دو خانه یک پرشگر قرار می‌دهیم:

خانه با اندیس 0 (مقدار 1) دارای پرشگر به اندیس 2 (مقدار 5) است.

خانه با اندیس 2 (مقدار 5) دارای پرشگر به اندیس 4 (مقدار 9) است.

خانه با اندیس 4 (مقدار 9) دارای پرشگر به انتهای لیست است (بدون پرش در نظر می‌گیریم).

3. ردیابی الگوریتم ادغام خطی استاندارد (بدون پرشگر)

در ادغام معمولی، ما فقط مقادیر P_A و P_B را با هم مقایسه می‌کنیم:

$1 < P_A$ پیش می‌رود (1 مقایسه)

$2 < P_B$ پیش می‌رود (1 مقایسه)

$3 < P_A$ پیش می‌رود (1 مقایسه)

$4 < P_B$ پیش می‌رود (1 مقایسه)

این روند زیگزاگی دقیقاً تا انتهای لیست ادامه می‌یابد.

تعداد کل مقایسه‌ها: به دلیل درهم‌تنیدگی کامل، برای رسیدن به انتهای لیست اول دقیقاً **11 مقایسه** انجام می‌شود.

4. ردیابی الگوریتم همراه با پرشگر (بروز افت کارایی)

اکنون همان دو لیست را با فعال بودن مکانیزم پرشگر ردیابی می‌کنیم:

گام اول ($P_A = 1$ و $P_B = 2$):

مقایسه اصلی: $1 < 2$ (نتیجه: 1 مقایسه)

چون $A < B$ است، سیستم شرط پرش را چک می‌کند: پرشگر به عدد 5 اشاره دارد. آیا $5 \leq 2$ است؟ خیر.

نتیجه شرط پرش: (1 مقایسه هدر رفته). P_A عادی پیش می‌رود.

گام دوم ($P_A = 3$ و $P_B = 2$):

مقایسه اصلی: $3 > 2$ (نتیجه: 1 مقایسه). P_B پیش می‌رود.

گام سوم ($P_A = 3$ و $P_B = 4$):

مقایسه اصلی: $3 < 4$ (نتیجه: 1 مقایسه). پرشگری وجود ندارد. P_A پیش می‌رود.

گام چهارم ($P_A = 5$ و $P_B = 4$):

مقایسه اصلی: $5 > 4$ (نتیجه: 1 مقایسه). P_B پیش می‌رود.

گام پنجم ($P_A = 5$ و $P_B = 6$):

مقایسه اصلی: $5 < 6$ (نتیجه: 1 مقایسه).

سیستم شرط پرش را چک می‌کند: پرشگر به عدد 9 اشاره دارد. آیا $9 \leq 6$ است؟ خیر.

نتیجه شرط پرش: (1 مقایسه هدر رفته). P_A عادی پیش می‌رود.

در هر گامی که لیست A کوچکتر است و پرشگر دارد، سیستم مجبور به انجام یک ارزیابی بی‌فایده می‌شود.

تعداد کل مقایسه‌ها با پرشگر: 11 مقایسه اصلی + 2 مقایسه باطل‌شده برای چک کردن پرشگرها = 13 مقایسه.

نتیجه‌گیری تحلیلی: این مثال نقض نشان می‌دهد که در ادغام دو لیست با توزیع عددی بسیار نزدیک و طول‌های مشابه، الگوریتم پرشگر نه تنها زمان اجرا را از مرتبه $O(N)$ کاهش نمی‌دهد، بلکه با افزودن شاخه‌های شرطی (Conditional Branches) بی‌حاصل، سربار پردازشی را افزایش داده و الگوریتم را از حالت بهینه خارج می‌کند. پرشگرها تنها زمانی مفیدند که یکی از لیست‌ها بسیار طولانی‌تر از دیگری بوده یا شکاف‌های عددی (Gaps) بزرگی بین شناسه‌های اسناد وجود داشته باشد.

پاسخنامه مسئله 1.15: معماری کش در موتورهای جستجو (Caching Architecture in Search Engines)

مقدمه: سطوح کش‌سازی در معماری موتورهای جستجو

در موتورهای جستجوی مقیاس‌پذیر، مدیریت پرسمان‌های تکراری از طریق حافظه نهان (Cache) یکی از ارکان کلیدی برای کاهش تاخیر (Latency) و صرفه‌جویی در منابع پردازشی است. معماران سیستم برای استقرار حافظه نهان با دو رویکرد بنیادین مواجه هستند: **کش‌سازی نتایج (Result Caching)** و **کش‌سازی لیست‌ها (List / Term Caching)**. انتخاب بین این دو نیازمند تحلیل دقیق بده‌بستان (Trade-off) میان مصرف حافظه، سرعت پاسخ‌گویی و هزینه به‌روزرسانی است.

1. رویکرد اول: کش کردن نتایج پرسمان (Result / Query Caching)

در این معماری، سیستم مستقیماً خروجی نهایی یک پرسمان کامل (مانند **A AND B**) را کش می‌کند. یعنی کلید کش، رشته متنی پرسمان است و مقدار آن، لیست کوچکی از شناسه‌های اسناد برتر (**Top-K DocIDs**) یا حتی کدهای **HTML** نتایج است.

مزیت زمانی: اگر کاربر دیگری دقیقاً همان پرسمان را وارد کند، پیچیدگی زمانی اجرای پرسمان از مرتبه پردازش اشتراک‌گیری، مستقیماً به $O(1)$ تقلیل می‌یابد؛ زیرا سیستم بدون هیچ محاسبه‌ای خروجی را از **RAM** می‌خواند.

مزیت فضایی در سطح پرسمان: فضای اشغال شده برای هر پرسمان بسیار اندک است، زیرا سیستم فقط نتایج برتر (مثلاً 10 سند اول) را ذخیره می‌کند، نه کل اسناد مشترک را.

2. رویکرد دوم: کش کردن لیست‌های رخداد (List / Term Caching)

در این معماری، سیستم پرسمان‌های ترکیبی را کش نمی‌کند، بلکه لیست رخداد (**Postings Lists**) کلماتِ پرکاربرد را از دیسک به حافظه اصلی (**RAM**) منتقل کرده و آنجا نگه می‌دارد.

مزیت نرخ اصابت (Hit Rate): بسیاری از پرسمان‌ها کاملاً جدید و منحصربه‌فرد هستند (**Long-tail Queries**)، اما کلمات تشکیل‌دهنده آن‌ها تکراری است. در این روش، حتی اگر کل پرسمان در سیستم سابقه نداشته باشد، لیست کلمات آن در کش وجود دارد و از هزینه سنگین خواندن از دیسک (**Disk I/O**) جلوگیری می‌شود.

چالش پردازشی: سرعت پاسخ‌گویی در این روش **O(1)** نیست. سیستم همچنان موظف است الگوریتم ادغام (مانند دو اشاره‌گر) را با پیچیدگی **O(x+y)** روی لیست‌های موجود در **RAM** اجرا کند.

3. تحلیل Trade-off: فضای حافظه در برابر به‌روزرسانی سیستم

تفاوت حیاتی و پاشنه آشیل این دو روش، زمانی آشکار می‌شود که پایگاه داده اسناد پویا باشد (اسناد جدید ایندکس شوند یا اسناد قدیمی حذف گردند):

تحلیل از منظر به‌روزرسانی و ابطال کش (Cache Invalidation):

در کش نتایج (Result Cache): مدیریت به‌روزرسانی یک فاجعه مهندسی است (**Cache Invalidation Nightmare**). اگر سندی به نام **D_{new}** به سیستم اضافه شود که حاوی کلمات **A** و **B** باشد، سیستم دقیقاً نمی‌داند این سند جدید روی کدامیک از میلیون‌ها پرسمان کش‌شده تاثیر می‌گذارد. برای حفظ دقت سیستم، با هر بار آپدیت شدن نمایه، معمولاً کل کش نتایج باید منقضی (**Flush**) شده و از نو ساخته شود که منجر به افت شدید کارایی موقت می‌گردد.

در کش لیست‌ها (List Cache): به‌روزرسانی بسیار نرم و کارآمد است. با ورود سند **D_{new}**، سیستم صرفاً شناسه این سند را به انتهای لیست‌های **A** و **B** در **RAM** اضافه می‌کند (**Append**). هیچ کشی باطل نمی‌شود و یکپارچگی سیستم با کمترین هزینه حفظ می‌گردد.

تحلیل از منظر فضای کل حافظه (Memory Space):

در کش نتایج: با وجود اینکه هر پرسمان فضای کمی می‌گیرد، اما به دلیل تنوع بی‌نهایت پرسمان‌های کاربران، ذخیره تمامی ترکیبات ممکن باعث انفجار فضای کش می‌شود.

در کش لیست‌ها: لیست‌های رخداد برای کلمات متداول نظیر «وب» یا «داده» بسیار غول‌پیکر هستند و نگهداری کامل آن‌ها در **RAM** فضای عظیمی از حافظه را اشغال می‌کند.

نتیجه‌گیری و راهکار:

در سیستم‌های بازیابی واقعی (مانند موتورهای جستجوی تجاری)، تکیه بر تنها یک رویکرد غیرمنطقی است. معماری استاندارد بر اساس یک سیستم کش چندسطحی (**Multi-level Caching**) بنا می‌شود:

1. سیستم از Result Caching تنها برای کسر بسیار کوچکی از پرسمان‌ها (شاید 1 تا 5 درصد) که به شدت تکراری و پرتعداد هستند (**Trending / Head Queries**) استفاده می‌کند تا بار پردازشی را به شدت کاهش دهد. برای رفع مشکل آپدیت، معمولاً این کش را با زمان‌های انقضای کوتاه (**TTL - Time to Live**) تنظیم می‌کنند تا داده‌های کهنه به صورت خودکار حذف شوند.

2. سیستم از List Caching برای کلمات مستقل پرکاربرد به عنوان لایه دوم استفاده می‌کند تا سرعت **I/O** را برای سایر

پرسمان‌های ترکیبی غیرتکراری تضمین نماید. این ترکیب، بهینه‌ترین حالت از تعادل میان حافظه پردازشی و سرعت به‌روزرسانی را فراهم می‌آورد.

پاسخنامه مسئله 1.16: تحلیل مشکل قحطی نتیجه و گذر به بازیابی رتبه‌بندی شده

مقدمه و بیان مشکل

یکی از چالش‌های بنیادین در طراحی سیستم‌های بازیابی اطلاعات (به‌ویژه در پلتفرم‌های تجارت الکترونیک)، مدیریت پرسمان‌های سخت‌گیرانه کاربران است. در این سناریو، یک خطای کلاسیک به نام **پدیده قحطی اطلاعات (Information Famine)** رخ داده است؛ جایی که سیستم به جای ارائه بهترین جایگزین‌های موجود، با اعمال منطق صفر و یکی، هیچ نتیجه‌ای به کاربر نشان نمی‌دهد و منجر به شکست تجربه کاربری (و از دست رفتن فروش) می‌گردد. در ادامه ریشه‌های تئوریک این بن‌بست و راهکار معماری آن تحلیل می‌شود.

بخش 1) ریشه بن‌بست در مدل بولین کلاسیک (Exact Match)

همان‌طور که در مسائل پیشین بررسی شد، مدل بولین بر پایه جبر گزاره‌ها و تطابق دقیق (**Exact Match**) عمل می‌کند. عملگر **AND** در این مدل، به معنای «اشتراک مطلق» است. فرض کنید سیستم در حال ارزیابی سندی با عنوان «لپ‌تاپ گیمینگ ایسوس **16** گیگابایت» برای پرسمان کاربر (**laptop AND gaming AND asus AND 32GB**) است. ارزیابی بولی این سند به شکل زیر انجام می‌شود:

کلمه **laptop** در سند وجود دارد **1**

کلمه **gaming** در سند وجود دارد **1**

کلمه **asus** در سند وجود دارد **1**

کلمه **32GB** در سند وجود ندارد **0**

ارزیابی نهایی سند برابر است با:

$$1 \text{ AND } 1 \text{ AND } 1 \text{ AND } 0 = 0$$

با وجود اینکه سند مورد نظر **75** درصد با نیاز کاربر هم‌پوشانی دارد و یک جایگزین بسیار مناسب است، اما به دلیل ماهیت **شکننده (Brittle)** مدل بولین، یک تک‌کلمه غایب (**32GB**) کل معادله را باطل کرده و سند کاملاً از لیست نتایج حذف می‌شود (**Famine**).

بخش 2) گذار معماری: جستجوی متن آزاد (Free-text Query)

برای حل این مشکل، موتورهای جستجوی مدرن از پارادایم جبر بولی فاصله گرفته و به سمت «جستجوی متن آزاد» حرکت می‌کنند. در این حالت، کاربر نیازی به استفاده از عملگرهای محدودکننده مانند **AND** یا **OR** ندارد. پرسمان صرفاً به مجموعه‌ای از کلمات (**Bag of Words**) تبدیل می‌شود:

[laptop, gaming, asus, 32GB]

این تغییر سینتکس، به سیستم اجازه می‌دهد تا به جای فیلتر کردن قطعی اسناد، به سمت «ارزیابی میزان ارتباط» حرکت کند.

در مدل‌های رتبه‌بندی‌شونده (نظیر مدل فضای برداری یا **Vector Space Model**)، ارزیابی اسناد دیگر یک فرآیند باینری (True / False) نیست، بلکه یک تابع پیوسته (**Continuous Function**) است که عددی بین 0 و 1 (یا یک امتیاز اعشاری) را به عنوان میزان شباهت بازمی‌گرداند.

این مدل‌ها از مفهوم تطابق نسبی (**Partial Match**) پشتیبانی می‌کنند. زمانی که پرسمان متن آزاد وارد سیستم می‌شود:

1. وزن‌دهی (**Term Weighting**): سیستم با استفاده از الگوریتم‌هایی نظیر **TF-IDF**، به هر کلمه در پرسمان وزن می‌دهد.

2. محاسبه شباهت (**Similarity Scoring**): سیستم میزان شباهت سند «لپ‌تاپ گیمینگ ایسوس 16 گیگابایت» را با پرسمان کاربر محاسبه می‌کند. از آنجا که 3 کلمه از 4 کلمه پرسمان در سند وجود دارد، این سند به جای دریافت امتیاز 0، یک امتیاز شباهت بالا (مثلاً 0.85) دریافت می‌کند.

3. رتبه‌بندی و نمایش (**Ranking**): از آنجا که لپ‌تاپ 32 گیگابایتی در فروشگاه موجود نیست تا امتیاز 1.0 بگیرد، همین لپ‌تاپ 16 گیگابایتی (با امتیاز 0.85) به عنوان بالاترین رتبه در جایگاه اول نتایج (**Top-1**) به کاربر نمایش داده می‌شود.

نتیجه‌گیری ساختاری: تفاوت این دو مدل، تفاوت میان "Data Retrieval" (بازیابی قطعی داده‌ها نظیر دستورات SQL) و "Information Retrieval" (بازیابی معنایی اطلاعات) است. مدل Ranked Retrieval با درک این موضوع که «عدم تطابق یک کلمه نباید به معنای بی‌ارزش بودن کل سند باشد»، پدیده **Famine** را از بین برده و با ارائه بهترین جایگزین‌های ممکن (مرتبط‌ترین اسناد)، نرخ موفقیت جستجو را به شدت افزایش می‌دهد.

پاسخنامه مسئله 1.17: معماری دوگانه نمایه و مدیریت بروزرسانی بلادرنگ

مقدمه و بیان مسئله

در سیستم‌های بازیابی اطلاعات پویای مقیاس‌بزرگ (مانند خبرگزاری‌ها یا شبکه‌های اجتماعی)، نمایه‌سازی ایستا (**Static Indexing**) پاسخگو نیست. نمایه وارونه اصلی برای کاهش حجم، با الگوریتم‌های پیچیده‌ای فشرده‌سازی شده و روی دیسک ذخیره می‌شود. ساختار این فشرده‌سازی‌ها به هم پیوسته است؛ یعنی درج یک سند جدید در میان یک لیست رخداد (**Postings List**) فشرده‌شده، نیازمند بازنویسی کل آن فایل روی دیسک است که از نظر محاسباتی و زمان **I/O** برای آپدیت‌های لحظه‌ای کاملاً غیرممکن محسوب می‌شود. راهکار استاندارد معماری برای این بن‌بست، استفاده از «سیستم نمایه‌سازی دوگانه» است.

بخش 1) طراحی معماری نمایه دوگانه (Dual Index Architecture)

برای پشتیبانی از درج بلادرنگ اسناد، سیستم نمایه‌سازی به دو بخش کاملاً مجزا با وظایف متفاوت تقسیم می‌شود:

نمایه اصلی (**Main Index**): این نمایه ساختاری عظیم، به شدت فشرده و فقط‌خواندنی (**Read-only**) دارد که روی دیسک سخت ذخیره شده است. هیچ سند جدیدی مستقیماً به این نمایه اضافه نمی‌شود.

نمایه کمکی (**Auxiliary Index**): یک نمایه وارونه کوچک، غیرفشرده (یا با فشرده‌سازی بسیار سبک) و کاملاً پویا است که منحصراً در حافظه اصلی (**RAM**) نگهداری می‌شود. تمامی اخبار و اسناد جدیدی که وارد سیستم می‌شوند، بلافاصله و با هزینه زمانی **O(1)** به این نمایه کمکی اضافه می‌شوند.

بخش 2) منطق اجرای پرسمان در سیستم دوگانه

وقتی کاربر پرسمانی (مثلاً **Q**) را جستجو می‌کند، سیستم دیگر نمی‌تواند فقط به یک نمایه رجوع کند. پردازشگر پرسمان، درخواست را به صورت موازی (**Parallel**) برای هر دو نمایه ارسال می‌کند:

- جستجو در نمایه اصلی و استخراج نتایج (**R_{main}**).
- جستجو در نمایه کمکی (موجود در **RAM**) و استخراج نتایج اخبار جدید (**R_{aux}**).
- سیستم این دو مجموعه نتیجه را با هم ترکیب و رتبه‌بندی کرده و خروجی نهایی یعنی **R_{main} ∪ R_{aux}** را به کاربر نمایش می‌دهد.

بخش 3) چالش ادغام (Index Merging) و هزینه آن

نمایه کمکی به دلیل قرارگیری در **RAM**، با محدودیت جدی فضا مواجه است. وقتی حافظه اختصاص‌یافته به این نمایه پر شود، سیستم باید عملیات سنگینی به نام ادغام (**Merge**) را آغاز کند.

در این مرحله، سیستم نمی‌تواند نمایه اصلی را درجا آپدیت کند. بلکه باید نمایه اصلی قبلی را به طور کامل از دیسک بخواند، آن را با نمایه کمکی (از **RAM**) به صورت ترتیبی ادغام کند، و یک **نمایه اصلی کاملاً جدید** روی دیسک بنویسد. پس از اتمام این عملیات **I/O** سنگین، نمایه کمکی در **RAM** پاک (**Flush**) شده و سیستم به نمایه اصلی جدید سوئیچ می‌کند.

بخش 4) رویکرد مدیریت اسناد حذف‌شده (Deletion Handling)

چالش حیاتی بعدی، حذف یک خبر (مثلاً به دلیل دستور قضایی یا اشتباه محتوایی) است. از آنجا که نمایه اصلی فقط‌خواندنی و فشرده است، پاک کردن فیزیکی شناسه آن خبر (**DocID**) از داخل لیست‌های رخداد دیسک امکان‌پذیر نیست.

برای رفع این چالش، معماران سیستم از تکنیک **بردار ابطال (Invalidation Bit Vector)** استفاده می‌کنند:

ساختار: سیستم یک آرایه بیتی بسیار سریع در حافظه اصلی (**RAM**) ایجاد می‌کند. هر بیت در این آرایه نماینده یک شناسه سند در سیستم است.

منطق حذف: اگر سندی (مثلاً **DocID_x**) حذف شود، سیستم هیچ تغییری در نمایه‌ها ایجاد نمی‌کند، بلکه صرفاً بیت مربوط به **DocID_x** را در آرایه ابطال به مقدار **1 (True)** تغییر می‌دهد.

منطق جستجو: هنگام اجرای هر پرسمان، پس از اینکه نتایج از نمایه اصلی و کمکی استخراج شد، سیستم پیش از نمایش نتایج به کاربر، آن‌ها را از فیلتر آرایه ابطال عبور می‌دهد. هر سندی که بیت آن **1** باشد، به عنوان سند مرده (**Tombstone**) شناخته شده و از نتایج نهایی دور ریخته می‌شود.

$$\text{Results} = (R_{\text{main}} \cup R_{\text{aux}}) - D$$

نتیجه‌گیری ریاضی معماری: با در نظر گرفتن مجموعه اسناد حذف‌شده با متغیر **D**، منطق ریاضی این معماری برای تولید خروجی نهایی در زمان جستجو همواره به شکل رابطه بالا فرمول‌بندی می‌شود که توازن بی‌نظیر میان سرعت آپدیت و کارایی جستجو برقرار می‌سازد.

پاسخنامه مسئله 1.18: استراتژی پردازش پرسمان ترکیبی و تحلیل هزینه

مقدمه و طرح مسئله

در پردازش پرسمان‌های بازبایی اطلاعات، مفسر پرسمان (Query Parser) در حالت پیش‌فرض از قوانین تقدم عملگرهای ریاضی پیروی می‌کند؛ به این معنا که همواره عبارات داخل پرانتز را در اولویت محاسبه قرار می‌دهد. اما زمانی که با توزیع نامتقارن کلمات (یک کلمه به شدت نادر و دو کلمه به شدت پرتکرار) مواجه هستیم، این تقدم ریاضی می‌تواند منجر به فروپاشی کارایی سیستم شود. در این نقطه، ورود ماژول **بهینه‌ساز پرسمان (Query Optimizer)** برای تغییر استراتژی پردازش (**Execution Plan**) الزامی است.

بخش 1) تحلیل رویکرد سنتی: آیا محاسبه زودهنگام (B OR C) منطقی است؟

پاسخ قطعی به این سوال «**خیر**» است. از منظر معماری سیستم‌های مقیاس‌بزرگ، اجرای این پرانتز در گام نخست یک خطای فاحش (**Anti-pattern**) محسوب می‌شود.

تحلیل هزینه (Cost Analysis): عملگر **OR** (اجتماع) در ذات خود یک عملگر «بسط‌دهنده» است. کلمات **B** و **C** هر کدام 1 میلیون شناسه سند دارند. اگر سیستم بخواهد حاصل (**B OR C**) را محاسبه کند، باید دو لیست عظیم را با الگوریتم ادغام پیمایش کند که پیچیدگی زمانی آن از مرتبه $O(|B| + |C|)$ خواهد بود.

سربار حافظه: سیستم مجبور است یک «لیست میانی» (**Intermediate List**) با حداکثر 2 میلیون عنصر را در حافظه اصلی (RAM) ایجاد کند. سپس در گام دوم، باید این لیست 2 میلیونی را با لیست کوچک **A** (که تنها 10 عنصر دارد) اشتراک بگیرد. این رویکرد، میلیون‌ها عملیات مقایسه‌ای و تخصیص حافظه بی‌فایده را به پردازنده تحمیل می‌کند، در حالی که خروجی نهایی به دلیل اشتراک با لیست نادر **A**، هرگز نمی‌تواند بیشتر از 10 سند باشد!

بخش 2) تغییر پارادایم در بهینه‌سازی: رویکرد جایگزین و بهینه

بهینه‌ساز پرسمان با بررسی آمار فرکانس اسناد (**Document Frequency**)، متوجه می‌شود که لیست **A** با طول 10، محدودکننده‌ترین بخش این پرسمان است. بنابراین، قانون طلایی بهینه‌سازی را اعمال می‌کند: «**لیست نادر همواره باید به عنوان لیست پیشران (Driving List) در نظر گرفته شود.**»

برای پردازش سریع‌تر این پرسمان، بهینه‌ساز از محاسبه اجتماع (**B OR C**) به طور کامل صرف‌نظر کرده و استراتژی «**پیمایش روی لیست کوچک، جستجو در لیست‌های بزرگ**» را جایگزین می‌کند.

بخش 3) اجرای مرحله‌به‌مرحله استراتژی جایگزین (Probing Strategy)

در این رویکرد، الگوریتم به جای استفاده از دو اشاره‌گر همگام (**Standard Merge**)، از مکانیزم جستجوی دودویی (**Binary Search**) یا پرشگرها (**Skip Pointers**) بهره می‌گیرد:

1. **پیمایش لیست پیشران:** سیستم حلقه پردازش را صرفاً روی عناصر لیست 10 تایی **A** باز می‌کند.

2. **آزمون شرط (Probing):** برای هر شناسه سند (**DocID**) موجود در لیست **A** (مثلاً سند شماره **X**):

ابتدا با استفاده از جستجوی دودویی (**Binary Search**) چک می‌کند که آیا سند **X** در لیست **B** وجود دارد؟ اگر وجود داشت، ارزیابی موفق است و سند **X** به لیست خروجی اضافه می‌شود. سیستم مستقیماً سراغ عنصر بعدی در لیست **A** می‌رود (الگوریتم میان‌بر منطقی یا **Short-circuit evaluation**). اگر در لیست **B** نبود، با جستجوی دودویی چک می‌کند که آیا سند **X** در لیست **C** وجود دارد؟ اگر بود، به خروجی اضافه می‌شود و در غیر این صورت کنار گذاشته می‌شود.

بخش 4) مقایسه پیچیدگی زمانی (تفاوت فاجعه تا بهینگی)

هزینه الگوریتم مبتنی بر پرانتز (سنتی): حدود 2,000,000 عملیات مقایسه.

هزینه الگوریتم جایگزین (بهینه شده):

برای هر یک از 10 عنصر لیست A، در بدترین حالت دو بار جستجوی دودویی (یکی در B و یکی در C) انجام می‌شود. پیچیدگی زمانی جستجوی دودویی در لیستی به طول 1 میلیون برابر است با:

$$\text{Cost(Binary Search)} \approx \log_2(1,000,000) \approx 20$$

بنابراین کل عملیات مقایسه برای این پرسمان برابر است با:

$$\text{Total Probing Cost} = 10 \times (\approx 20 + \approx 20) \approx 400$$

نتیجه‌گیری ساختاری: با تغییر زاویه دید از «حل معادلات جبری» به «انتخاب ساختار داده هوشمند»، زمان اجرای پرسمان از 2 میلیون واحد محاسباتی به تنها 400 مقایسه کاهش می‌یابد. در موتورهای جستجوی واقعی، بهینه‌سازها با بازنویسی فیزیکی نحوه اجرای پرسمان‌ها (Physical Execution Plan)، از تشکیل لیست‌های میانی غول‌پیکر برای عملگر OR جلوگیری می‌کنند.

پاسخنامه مسئله 1.19: تحلیل پیچیدگی زمانی اشتراک چندگانه و ارزیابی جدول درهم‌سازی

مقدمه

در پردازش پرسمان‌هایی با چندین عبارت محدودکننده (مانند $T_1 \text{ AND } T_2 \text{ AND } \dots \text{ AND } T_K$)، انتخاب الگوریتم مناسب برای یافتن اشتراک K لیست رخداد، مرز بین یک سیستم بازیابی بلادرنگ (Real-time) و یک سیستم ناکارآمد را مشخص می‌کند. در این مسئله، هزینه الگوریتم استاندارد متوالی بررسی شده و سپس ساختار جدول درهم‌سازی به عنوان یک راهکار جایگزین احتمالی نقد می‌شود.

بخش 1) تحلیل پیچیدگی زمانی الگوریتم ادغام متوالی (Sequential Merge)

الگوریتم استاندارد برای ادغام K لیست (با میانگین طول N) شامل دو فاز است: مرتب‌سازی اولیه لیست‌ها و سپس اشتراک‌گیری دوبه‌دو.

فاز اول (مرتب‌سازی بر اساس DF): سیستم ابتدا فرکانس سند (DF) یا همان طول هر K لیست را خوانده و آن‌ها را به صورت صعودی مرتب می‌کند. پیچیدگی زمانی این فاز برابر با $O(K \log K)$ است.

فاز دوم (اشتراک‌گیری متوالی): ادغام از دو لیست کوچک‌تر آغاز می‌شود. هزینه اشتراک دو لیست اول با الگوریتم دو اشاره‌گره، در بدترین حالت $O(N + N) = O(N)$ است.

بدترین حالت ممکن (Worst-case Scenario): بدترین حالت زمانی رخ می‌دهد که اشتراک اسناد بین لیست‌ها کامل باشد (Complete Overlap)؛ یعنی طول «لیست میانی» تولید شده پس از هر بار اشتراک، کاهش نیابد و همچنان N باقی بماند. در این شرایط، سیستم باید $(K - 1)$ بار عملیات ادغام را انجام دهد که در هر بار، یک لیست به طول N با لیست بعدی (که آن هم در بدترین حالت طول N دارد) مقایسه می‌شود. بنابراین هزینه کل فاز دوم برابر خواهد بود با:

$$O((K-1) \times N) \approx O(K \times N)$$

مرتبه زمانی کل: با ترکیب دو فاز، پیچیدگی زمانی کلی برابر با $O(K \log K + K N)$ خواهد بود. از آنجا که در سیستم‌های واقعی تعداد کلمات پرسمان (K) بسیار کوچکتر از طول لیست‌ها (N) است ($K \ll N$)، بخش $K \log K$ نادیده گرفته شده و مرتبه زمانی الگوریتم در بدترین حالت $O(K N)$ تقریب زده می‌شود.

بخش 2) آیا جدول درهم‌سازی (Hash Table) جایگزین بهتری است؟

پاسخ کوتاه به این ادعا «خیر» است.

ایده استفاده از جدول درهم‌سازی (ساخت **Hash** از کوچکترین لیست و جستجوی عناصر لیست‌های دیگر در آن با زمان $O(1)$ برای هر عنصر)، روی کاغذ جذاب به نظر می‌رسد و به پیچیدگی زمانی ظاهری $O(K N)$ می‌رسد. اما در معماری موتورهای جستجو و مهندسی سیستم، این رویکرد به دلایل ساختاری زیر کاملاً مطرود (**Anti-pattern**) است:

بخش 3) معایب استفاده از Hash Table در ادغام لیست‌های ادعا

الف) افت شدید کارایی حافظه نهان (Cache Locality Failure):

بزرگترین نقطه ضعف جدول درهم‌سازی، نحوه دسترسی آن به حافظه است. الگوریتم دو اشاره‌گره (**Standard Merge**) روی آرایه‌ها حرکت می‌کند. پردازنده‌ها برای خواندن آرایه‌ها به شدت بهینه‌سازی شده‌اند و به دلیل ویژگی «مکان‌گزینی فضایی» (**Spatial Locality**)، بلوک‌های بعدی لیست را از پیش در حافظه نهان (**CPU Cache**) بارگذاری می‌کنند (**Prefetching**). اما جستجو در جدول درهم‌سازی نیازمند دسترسی تصادفی به حافظه (**Random Memory Access**) است که منجر به خطاهای پیاپی کش (**Cache Miss**) شده و سرعت پردازنده را برای واکشی داده از **RAM** به شدت کاهش می‌دهد. در عمل، زمان پنهان یک **Cache Miss** بسیار بیشتر از چند مقایسه ساده عددی است.

ب) پیچیدگی فضایی و سربر حافظه (Memory Overhead):

الگوریتم استاندارد دو اشاره‌گره به صورت درجا (**In-place**) عمل کرده و به فضای کمکی (**Auxiliary Space**) نیازی ندارد؛ پیچیدگی فضایی آن $O(1)$ است. اما ساخت یک جدول درهم‌سازی برای لیست‌های میلیونی، نیازمند تخصیص حافظه جانبی عظیم از مرتبه $O(N)$ است. علاوه بر این، برای جلوگیری از تصادم (**Collisions**)، ضریب بار (**Load Factor**) جدول باید پایین نگه داشته شود که به معنای تخصیص حافظه‌ای به مراتب بیشتر از تعداد خود شناسه‌هاست.

ج) از دست رفتن ترتیب اسناد (Loss of Sorting):

در سیستم‌های بازیابی، خروجی پرسمان باید همواره بر اساس شناسه‌های سند (**DocID**) مرتب باشد تا در صورت نیاز به ترکیب با عملگرهای دیگر (نظیر فیلترهای زمانی، یا پرسمان‌های عبارتی) قابل استفاده باشد. جدول درهم‌سازی ترتیب اعداد را از بین می‌برد. اگر سیستم از **Hash** استفاده کند، در انتهای کار مجبور است لیست خروجی را مجدداً با زمان $O(R \log R)$ (که R تعداد نتایج است) مرتب کند که این امر سربر زمانی اضافه‌ای را تحمیل می‌سازد.

د) ضرایب ثابت پنهان (High Constant Factors):

گرچه زمان جستجو در جدول درهم‌سازی با $O(1)$ نمایش داده می‌شود، اما این $O(1)$ شامل محاسبه تابع درهم‌سازی (**Hash Function**) و مدیریت تصادم (**Collision Resolution**) برای هر تک‌تک شناسه‌هاست. محاسبه این تابع برای پردازنده، بسیار پرهزینه‌تر از یک عملگر مقایسه ساده ($>$ یا $<$) در الگوریتم دو اشاره‌گره است.

نکته کاربردی برای ارزیابی دانشجویان: در جلسات دفاع یا تصحیح برگه‌ها، اگر دانشجویی پیشنهاد استفاده از جدول

درهم‌سازی را داد، از او بخواهید تفاوت زمان تئوری (O) و زمان واقعی سیستم (با در نظر گرفتن معماری CPU و

کش) را توضیح دهد. این سوال معیار بسیار دقیقی برای سنجش میزان تسلط دانشجو بر مفاهیم

Systems Programming در کنار الگوریتم‌هاست.

پاسخنامه مسئله 1.20: تحلیل خطای پیاده‌سازی عملگر AND و تداخل مدل‌های بازیابی

مقدمه و طرح مسئله

در مهندسی سیستم‌های بازیابی اطلاعات، تفکیک دقیق میان پارادایم‌های مختلف جستجو از اهمیت بالایی برخوردار است. خطای فاحش تیم دانشجویی در این پروژه، ناشی از عدم درک مرزهای تئوریک میان «مدل بازیابی بولی کلاسیک» (Classic Boolean Retrieval) و «مدل‌های بازیابی رتبه‌بندی‌شونده» (Ranked Retrieval Models) است. آن‌ها به جای پیاده‌سازی منطق مجموعه‌ها، یک تابع امتیازدهی خام را در بستری اشتباه پیاده‌سازی کرده‌اند.

بخش 1) چرا این پیاده‌سازی در مدل Boolean از اساس غلط است؟

مدل بازیابی بولین بر دو پایه اساسی استوار است: **نظریه مجموعه‌ها (Set Theory)** و مفهوم **تطابق دقیق (Exact Match)**.

ارزیابی دودویی (Binary Evaluation) در برابر **فرکانس**: در مدل بولین کلاسیک، تنها شاخص معنادار، «حضور یا عدم حضور» (Term Presence) یک کلمه در سند است. سیستم برای هر کلمه در یک سند تنها مقادیر 1 (وجود دارد) یا 0 (وجود ندارد) را می‌پذیرد. فرکانس کلمه (Term Frequency)، یعنی اینکه کلمه 1 بار در متن آمده یا 1000 بار، در این مدل فاقد ارزش محاسباتی است.

نقض منطق عملگر AND (اشتراک مجموعه‌ها): عملگر AND به معنای تقاطع قطعی لیست‌های رخداد ($A \cap B$) است. وقتی دانشجویان به جای اشتراک، فرکانس کلمات را جمع می‌زنند، در واقع ماهیت محدودکننده AND را از بین برده و آن را تبدیل به یک عملگر OR با وزن‌دهی غلط کرده‌اند. در این پیاده‌سازی اشتباه، فقدان کامل یک کلمه می‌تواند توسط تکرار بیش از حد کلمه‌ای دیگر جبران شود، که این امر نقض صریح منطق جبر بول است.

بخش 2) بررسی یک مثال نقض برای اثبات فروپاشی الگوریتم دانشجویان

فرض کنید کاربر پرسمان عبارتی **Security AND Algorithm** را وارد کرده است. سیستم باید اسنادی را برگرداند که هر دو کلمه در آن‌ها وجود داشته باشد.

سند شماره 1: یک مقاله کاملاً نامرتب درباره «انواع قفل‌های امنیتی درب». کلمه **Security** در این متن 100 بار تکرار شده است، اما کلمه **Algorithm** اصلاً وجود ندارد (تکرار = 0).

سند شماره 2: یک مقاله کاملاً مرتبط درباره «الگوریتم‌های رمزنگاری». کلمه **Security** 2 بار و کلمه **Algorithm** 3 بار در متن آمده است.

رفتار الگوریتم دانشجویی:

جمع فرکانس در سند اول: $100 + 0 = 100$

جمع فرکانس در سند دوم: $2 + 3 = 5$

الگوریتم دانشجویان سند کاملاً نامرتب اول را با امتیاز 100 به عنوان بهترین نتیجه برمی‌گرداند! زیرا سند اول با وجود نداشتن نیمی از پرسمان کاربر، به دلیل جمع بالای فرکانس کلمه دیگر، سند دوم را شکست داده است. این در حالی است که در مدل بولین صحیح، خروجی سند اول دقیقاً برابر با 0 ($1 \text{ \& } 0 = 0$) و خروجی سند دوم برابر با 1 ($1 \text{ \& } 1 = 1$) است.

بخش 3) مفهوم واردشده به اشتباه: تداخل با مدل فضای برداری (Vector Space Model)

این گروه دانشجویی، به صورت ناخودآگاه و ناقص، دو مفهوم کلیدی از سیستم‌های رتبه‌بندی مدرن را وارد سیستم بولین کرده است:

مدل کیسه کلمات (Bag of Words): آن‌ها به جای نگاه باینری (ماتریس تلاقی)، به متن به عنوان ظرفی از کلمات نگاه کرده‌اند که تکرار در آن مجاز و معنادار است.

وزن‌دهی بر اساس فرکانس کلمه (Term Frequency Weighting - TF): جمع زدن تکرار کلمات، ابتدایی‌ترین شکل از امتیازدهی در مدل فضای برداری (Vector Space Model) است.

نتیجه‌گیری ساختاری

اما خطای مضاعف آن‌ها این است که این امتیازدهی (**Scoring**) را به صورت ایزوله و خام اعمال کرده‌اند. در مدل‌های رتبه‌بندی استاندارد، استفاده از فرکانس کلمه (**TF**) به تنهایی فاجعه‌بار است و حتماً باید با فاکتورهای متعادل‌کننده‌ای نظیر **فرکانس معکوس سند (IDF - Inverse Document Frequency)** ترکیب شود (فرمول **TF-IDF**) و سپس توسط معیارهایی مانند **شباهت کسینوسی (Cosine Similarity)** نرمال‌سازی (**Normalize**) گردد تا طول اسناد و اهمیت کلمات نایب در محاسبه نهایی لحاظ شود. ترکیب کردن منطق بولین با فرکانس‌های خام، نه دقت مدل اول را دارد و نه هوشمندی مدل دوم را.

پاسخنامه مسئله 1.21: تحلیل شکست تجاری مدل بولین و گذر به بازیابی رتبه‌بندی شده

مقدمه

طراحی سیستم‌های جستجو در پلتفرم‌های تجارت الکترونیک (**E-commerce**)، نیازمند درک عمیق از رفتار کاربر و انعطاف‌پذیری سیستم است. در این سناریو، فروشگاه با یکی از مخرب‌ترین خطاهای سیستم‌های بازیابی به نام **«قحطی اطلاعات» (Information Famine)** مواجه شده است؛ وضعیتی که سیستم به جای ارائه بهترین جایگزین‌های ممکن، خروجی تهی (**Null Result**) برمی‌گرداند و منجر به شکست قیف فروش می‌شود. در ادامه، ریشه‌های تئوریک این شکست و راهکار معماری آن تحلیل می‌گردد.

بخش الف) تحلیل تئوریک بن‌بست در مدل Boolean و Extended Boolean

ریشه این شکست تجاری، در ماهیت ریاضیاتی و سخت‌گیرانه مدل بولین نهفته است. مدل بولین بر پایه مفهوم **«تطابق دقیق» (Exact Match)** استوار است. در این پارادایم، ارزیابی اسناد به صورت کاملاً دودویی (**Binary**) انجام می‌شود و هیچ درکی از «درجه ارتباط» (**Degree of Relevance**) یا «نزدیکی معنایی» وجود ندارد.

زمانی که سیستم در حال پردازش پرسمانِ مقیدشده با عملگر **AND** است، شرط لازم و کافی برای بازیابی یک سند، حضور **تمامی** کلمات پرسمان در آن سند است.

رفتار سیستم را برای یک لپ‌تاپ «گیمینگ ایسوس 32 گیگابایت» (که یک جایگزین عالی است) بررسی می‌کنیم:

کلمه **laptop** در سند موجود است 1

کلمه **gaming** در سند موجود است 1

کلمه **asus** در سند موجود است 1

ارزیابی نهایی خروجی سیستم برای این سند بر اساس جبر بول به شکل زیر خواهد بود:

$$1 \text{ AND } 1 \text{ AND } 1 \text{ AND } 0 = 0$$

دلیل شکست تجاری: با وجود اینکه این سند 75٪ با نیاز کاربر هم‌پوشانی دارد، اما مدل بولین به دلیل شکنندگی ذاتی خود (Brittleness)، با غیبت تنها یک کلمه (16GB)، کل سند را بی‌اعتبار دانسته و به آن امتیاز 0 می‌دهد. مدل **Extended Boolean** نیز (حتی با وجود عملگرهای مجاورت)، در نهایت بر پایه همین منطق حذفی کار می‌کند. در دنیای واقعی، کاربران معمولاً شرایط سخت‌گیرانه‌ای ندارند و آماده پذیرش جایگزین‌های نزدیک (مثل برند دیگر یا حافظه بالاتر) هستند؛ اما مدل بولین این شانس را از کاربر و کسب‌وکار می‌گیرد.

بخش ب) گذار معماری به Free-text Query و Ranked Retrieval

برای جلوگیری از این بن‌بست، موتورهای جستجوی مدرن از منطق حذفی فاصله‌گرفته و به سمت ارزیابی پیوسته حرکت می‌کنند. این تغییر معماری نیازمند دو گام اساسی است:

1. حذف عملگرهای بولی و استفاده از Free-text Query

در سیستم جدید، کاربر نیازی به وارد کردن عملگرهای منطقی ندارد. پرسمان از حالت یک «معادله شرطی» خارج شده و به یک «کیسه کلمات» (**Bag of Words**) تبدیل می‌شود:

[laptop, gaming, asus, 16GB]

در این حالت، سیستم کلمات را به عنوان «ترجیحات کاربر» (**Soft Constraints**) می‌بیند، نه «شروط الزام‌آور» (**Hard Constraints**).

2. استفاده از مکانیزم تطابق نسبی (Partial Match) در Ranked Retrieval

در مدل‌های رتبه‌بندی‌شونده (مانند مدل فضای برداری - **Vector Space Model**)، ارزیابی اسناد دیگر خروجی 0 و 1 ندارد، بلکه تابع ارزیابی یک امتیاز پیوسته (**Score**) را برمی‌گرداند. نحوه برخورد مفهومی سیستم جدید با این پرسمان به شرح زیر است:

وزن‌دهی (Weighting): سیستم به هر کلمه در پرسمان و اسناد یک وزن عددی اختصاص می‌دهد (مثلاً با الگوریتم **TF-IDF**).

محاسبه شباهت (Similarity Calculation): سیستم میزان شباهت هر کالا را با پرسمان کاربر می‌سنجد.

کالای الف (لپ‌تاپ گیمینگ ایسوس 32GB): 3 کلمه از 4 کلمه پرسمان (**laptop, gaming, asus**) را داراست. بنابراین امتیاز بالایی (مثلاً **0.75**) دریافت می‌کند.

کالای ب (لپ‌تاپ گیمینگ لنوو 16GB): این کالا نیز 3 کلمه از 4 کلمه پرسمان (**laptop, gaming, 16GB**) را داراست و امتیاز مشابهی (مثلاً **0.70**) می‌گیرد.

رتبه‌بندی و نمایش (Ranking): سیستم اسناد را بر اساس امتیاز کسب‌شده به صورت نزولی مرتب می‌کند.

نتیجه‌گیری ساختاری: در سیستم **Ranked Retrieval**، با وجود اینکه کالای کاملاً منطبق (100٪) در انبار وجود ندارد، خروجی سیستم تهی نخواهد بود. سیستم بهترین جایگزین‌های موجود (لپ‌تاپ‌های ایسوس 32 گیگابایتی و لنوو 16

گیگابایتی) را در صدر نتایج (Top-K) به کاربر نمایش می‌دهد. این امر با ارضای نسبی نیاز کاربر، پدیده قحطی اطلاعات را حل کرده و مستقیماً باعث نجات نرخ تبدیل (Conversion Rate) فروشگاه می‌شود.

پاسخنامه مسئله 1.22: طراحی سناریوی بدترین حالت در اشاره‌گرهای پرشی

(Worst-case Scenario Design in Skip Pointers)

مقدمه و طرح مسئله

در پیاده‌سازی سیستم‌های بازیابی اطلاعات، استفاده از پرشگرها (Skip Pointers) با هدف کاهش پیچیدگی زمانی در ادغام لیست‌های طولانی (عملگر AND) پیشنهاد می‌شود. ایده اصلی این است که با پرش از روی بخش‌هایی از لیست که قطعاً در اشتراک نقشی ندارند، تعداد مقایسه‌ها کاهش یابد. اما پرشگرها یک هزینه پنهان (Hidden Cost) دارند: **سربار ارزیابی شرط پرش**.

الگوریتم برای انجام یک پرش، باید علاوه بر مقایسه اصلی دو عنصر، بررسی کند که آیا «مقصد پرشگر همچنان کوچکتر یا مساوی با مقدار فعلی در لیست دوم است یا خیر». اگر سیستم مدام این شرط را بررسی کند اما پرشی صورت نگیرد، الگوریتم از حالت خطی استاندارد ($O(x+y)$) نیز کندتر خواهد شد.

بخش 1) طراحی سناریو و مثال نقض عددی (Worst-case Scenario)

بدترین حالت برای پرشگرها زمانی رخ می‌دهد که شناسه‌های اسناد (DocIDs) در دو لیست، به شدت متراکم (Dense) و درهم‌تنیده (Interleaved) باشند. در این حالت، مقادیر دو لیست یکی‌درمیان از یکدیگر بزرگتر می‌شوند و عملاً هیچ شکاف (Gap) بزرگی برای پرش وجود ندارد.

برای اثبات این موضوع، دو لیست زیر را در نظر می‌گیریم:

لیست L_1 : [1, 3, 5, 7, 9, 11] (طول لیست: 6)

لیست L_2 : [2, 4, 6, 8, 10, 12]

تنظیمات پرشگر برای لیست L_1 :

بر اساس فرمول استاندارد، طول گام پرش (Skip Step) معمولاً برابر با جذر طول لیست ($\sqrt{N}$) در نظر گرفته می‌شود. برای لیست L_1 ، طول گام $2 \approx \sqrt{6}$ است. بنابراین در هر دو خانه، یک پرشگر قرار می‌دهیم:

از اندیس 0 (مقدار 1) پرشگر به اندیس 2 (مقدار 5) اشاره می‌کند.

از اندیس 2 (مقدار 5) پرشگر به اندیس 4 (مقدار 9) اشاره می‌کند.

از اندیس 4 (مقدار 9) پرشگر به انتهای لیست متصل است.

بخش 2) ردیابی گام‌به‌گام و اثبات افت کارایی (Trace)

اکنون رفتار الگوریتم دارای پرشگر را روی این دو لیست ردیابی می‌کنیم. فرض کنید P_1 اشاره‌گر لیست اول و P_2 اشاره‌گر لیست دوم است:

گام اول ($P_1 = 1$ و $P_2 = 2$):

مقایسه اصلی: $1 < 2$ (نتیجه: 1 مقایسه).

بررسی شرط پرش: چون L_1 کوچکتر است و پرشگر دارد، سیستم چک می‌کند که آیا مقدار مقصد پرشگر ($value = 5$) کوچکتر یا

مساوی با P_2 (value = 2) است؟ **خیر** ($2 \leq 5$).

نتیجه: پرش لغو می‌شود. P_1 یک خانه عادی به جلو می‌رود. (1 مقایسه بی‌فایده به پردازنده تحمیل شد).

گام دوم ($P_1 = 3$ و $P_2 = 2$):

مقایسه اصلی: $3 > 2$ (نتیجه: 1 مقایسه).

چون L_2 کوچکتر است، P_2 جلو می‌رود.

گام سوم ($P_1 = 3$ و $P_2 = 4$):

مقایسه اصلی: $3 < 4$ (نتیجه: 1 مقایسه). در اینجا پرشگری وجود ندارد. P_1 عادی جلو می‌رود.

گام چهارم ($P_1 = 5$ و $P_2 = 4$):

مقایسه اصلی: $5 > 4$ (نتیجه: 1 مقایسه). P_2 جلو می‌رود.

گام پنجم ($P_1 = 5$ و $P_2 = 6$):

مقایسه اصلی: $5 < 6$ (نتیجه: 1 مقایسه).

بررسی شرط پرش: پرشگر به عدد 9 اشاره دارد. آیا $9 \leq 6$ است؟ **خیر**.

نتیجه: پرش لغو می‌شود. P_1 عادی به جلو می‌رود. (1 مقایسه بی‌فایده دیگر).

نتیجه‌گیری بخش ردیابی: اگر این دو لیست را با الگوریتم استاندارد (بدون پرش) ادغام می‌کردیم، برای رسیدن به انتهای لیست اول دقیقاً به **11 مقایسه** نیاز داشتیم. اما با فعال بودن پرشگرها، علاوه بر آن 11 مقایسه اصلی، سیستم مجبور شد شرط‌های پرش لغوشده را نیز بررسی کند که تعداد کل مقایسه‌ها را افزایش داد (به عنوان مثال **13 مقایسه** یا بیشتر). در مقیاس‌های میلیونی، این شاخه‌های شرطی بی‌فایده (**Branch Prediction Misses**) باعث افت شدید سرعت CPU می‌شوند.

بخش 3) تحلیل تاثیر طول گام پرش (Impact of Skip Step Size)

در این حالت بدبینانه (لیست‌های درهم‌تنیده)، تغییر طول گام پرش چه تاثیری دارد؟

حالت الف: گام‌های کوتاه (تعداد پرشگرهای زیاد - مثلاً پرش هر 1 یا 2 خانه):

این **فاجعه‌بارترین** سناریو است. هرچه گام‌ها کوتاه‌تر باشند، تعداد گره‌های دارای پرشگر بیشتر می‌شود. در نتیجه، الگوریتم در هر قدم تلاش می‌کند پرش کند، شرط پرش با شکست مواجه می‌شود، و سیستم بالاترین حد از مقایسه‌های زائد و سربار محاسباتی را متحمل می‌گردد.

حالت ب: گام‌های بلند (تعداد پرشگرهای کم - مثلاً پرش هر 1000 خانه):

در این حالت، تعداد گره‌های دارای پرشگر در لیست کاهش می‌یابد. بنابراین الگوریتم کمتر با شرط‌های پرش برخورد می‌کند و تعداد خطاهای بررسی (**Overhead**) کمتر می‌شود. در این سناریو، رفتار سیستم تا حد زیادی به همان ادغام خطی استاندارد نزدیک می‌شود (هرچند باز هم اندکی کندتر از آن است).

خلاصه معماری

این مسئله اثبات می‌کند که پرشگرها یک راهکار جادویی برای افزایش سرعت نیستند. این ساختارها تنها زمانی مفیدند که با «لیست‌هایی با طول به شدت نامساوی» مواجه باشیم یا توزیع اعداد دارای «شکاف‌های بزرگ» (**Large Gaps**) باشد تا پرش‌های موفق بتوانند هزینه مقایسه‌های سربار را جبران کنند.

پاسخنامه مسئله 1.23: معماری نمایه پویا

(Dynamic Indexing)

مقدمه و طرح چالش معماری

در موتورهای جستجو و سیستم‌های بازیابی اطلاعات، نمایه وارونه (**Inverted Index**) یک ساختار داده به شدت فشرده و به هم پیوسته است. هرگونه تلاش برای درج مستقیم (**In-place Insertion**) یک سند جدید در نمایه‌ای که روی دیسک (**Disk**) قرار دارد، نیازمند بازنویسی بخش‌های عظیمی از فایل‌های روی دیسک است. این عملیات، سربار ورودی/خروجی (**I/O Overhead**) بسیار سنگینی دارد و معماری سیستم را در مواجهه با ده‌ها هزار آپدیت روزانه (نظیر خبرگزاری‌ها) فلج می‌کند.

برای حل این تضاد میان «حجم عظیم داده‌های ایستا» و «نیاز به آپدیت‌های بلادرنگ»، مهندسی نرم‌افزار رویکردی استاندارد تحت عنوان **معماری نمایه دوگانه (Dual Index Architecture)** را ارائه می‌دهد.

بخش 1) ساختار معماری نمایه دوگانه

در این رویکرد، سیستم نمایه‌سازی از یکپارچگی خارج شده و به دو جزء با عملکردهای متفاوت و سلسله‌مراتب حافظه (**Memory Hierarchy**) مجزا تقسیم می‌گردد:

الف) نمایه اصلی (Main Index):

این نمایه حاوی آرشیو کلان خبرگزاری است. ساختار آن کاملاً فشرده، ایستا و **فقط‌خواندنی (Read-only)** است که منحصراً روی دیسک سخت (**HDD/SSD**) نگهداری می‌شود. هیچ سند جدیدی به صورت مستقیم و در لحظه به این نمایه اضافه نمی‌گردد.

ب) نمایه کمکی (Auxiliary Index):

یک نمایه وارونه کوچک، پویا و غیرفشرده است که به طور کامل در **حافظه اصلی (RAM)** سیستم مستقر می‌شود. از آنجا که عملیات در **RAM** با سرعت پردازنده انجام می‌گیرد، هر خبر جدیدی که وارد خبرگزاری می‌شود، در زمان **O(1)** به این نمایه کمکی اضافه شده و فوراً قابل جستجو می‌گردد.

بخش 2) منطق اجرای پرسمان (Query Execution)

زمانی که کاربر پرسمانی را در این خبرگزاری جستجو می‌کند، موتور جستجو به جای یک سیستم، به دو سیستم موازی متوسل می‌شود:

1. پردازشگر پرسمان، درخواست را همزمان به نمایه اصلی (دیسک) و نمایه کمکی (**RAM**) ارسال می‌کند.

2. نتایج آرشیو قدیمی از نمایه اصلی استخراج می‌شود (**R_{main}**).

3. نتایج اخبار ثانیه‌های اخیر از نمایه کمکی استخراج می‌شود (**R_{aux}**).

4. سیستم در نهایت این دو مجموعه را ادغام (**Merge**) و بر اساس الگوریتم‌های رتبه‌بندی مرتب کرده و خروجی یکپارچه یعنی **R_{main} ∪ R_{aux}** را به کاربر بازمی‌گرداند.

بخش 3) مدیریت سرریز حافظه و ادغام دوره‌ای (Periodic Index Merging)

ظرفیت حافظه **RAM** محدود است و نمایه کمکی نمی‌تواند تا ابد رشد کند. وقتی حجم نمایه کمکی به آستانه مشخصی (**Threshold**) رسید، سیستم باید فاز سنگین «ادغام» را آغاز کند:

در این مرحله، سیستم در پس‌زمینه (**Background Thread**)، نمایه اصلی قدیمی را از دیسک می‌خواند، آن را با نمایه کمکی ادغام می‌کند، و یک **نمایه اصلی کاملاً جدید** روی دیسک می‌نویسد. پس از تکمیل این فرآیند، نمایه کمکی از **RAM** پاک (**Flush**) شده و سیستم به نمایه اصلی جدید سوئیچ می‌کند.

بخش 4) چالش حذف اخبار (Deletion) و بردار ابطال

اگر خبری به دلایل حقوقی یا اشتباه محتوایی حذف شود، امکان پاک کردن آن از نمایه اصلی فقط خواندنی وجود ندارد. راهکار معماری برای این چالش، استفاده از **بردار ابطال (Invalidation Bit Vector)** است.

سیستم یک آرایه بیتی بسیار سریع در حافظه **RAM** نگه می‌دارد که هر بیت آن نماینده شناسه یک سند (**DocID**) است. با حذف یک خبر، سیستم تنها بیت مربوط به شناسه آن خبر را در این آرایه برابر با **1** قرار می‌دهد (نشانه سند مرده یا **Tombstone**).

در زمان اجرای پرسمان، فرمول خروجی نهایی به شکل منطقی زیر اصلاح می‌گردد تا اسناد حذف‌شده (مجموعه **D**) پیش از نمایش به کاربر، از نتایج فیلتر شوند:

$$\text{Results} = (R_{\text{main}} \cup R_{\text{aux}}) - D$$

نتیجه‌گیری ساختاری: این الگو که در دیتابیس‌های پیشرفته نظیر Elasticsearch و موتورهای جستجوی وب استفاده می‌شود، با انتقال هوشمندانه هزینه از «زمان درج» (**Write-Time**) به «زمان ادغام دوره‌ای»، تضمین می‌کند که پیچیدگی زمانی ذخیره و بازیابی اخبار جدید در پایین‌ترین سطح ممکن باقی بماند.